دليل هندسي شامل — من الصفر إلى الإنتاج

المحوّلات والنماذج اللغوية الكبيرة

من النظرية إلى بناء أنظمة ذكية

Transformer · Attention · MoE · Flash-3 · VLM · DPO/GRPO



48 فصلًا · 10 أجزاء · 9 ملاحق · 33 مخطّطًا · +364 كتلة كود

BookAI

الطبعة الأولى — 2026

ARABIC · ENGINEERING · 2026

كتاب عملي • إصدار **2026**

المحوّلات والنماذج اللغوية الكبيرة

من النظرية إلى بناء أنظمة ذكية

م. أحمد صالح المغز

الطبعة الأولى — 2026 · 572~ ⏱ · BookAI دقيقة قراءة · 48 فصلًا · 10 أجزاء · 9 ملاحق · 33 مخططًا

فهرس الكتاب

أعمال تأسيسية

الهداء 4

المقدمة 5

خطة الكتاب 8

الجزء الأول: الأساسيات — من الشبكات العصبية إلى التعلّم العميق 11

الفصول 1–3

الفصل 1: الشبكات العصبية من الصفر — كيف تتعلم الآلة أن "تخمّن" بشكل أفضل 12

الفصل 2: مفاهيم التعلّم العميق للتدريب والتقييم — كيف تعرف أن نموذجك "يتعلم" فعلًا؟ 24

الفصل 3: بنية التدريب الحديثة — من GPU واحدة إلى عنقود 39

الجزء الثاني: التضمينات والتمثيلات — كيف تتحول النصوص إلى أرقام 51

الفصول 4–6

الفصل 4: الترميز إلى وحدات (Tokenization) — كيف يقرأ النموذج ما لا تراه عينك 52

الفصل 5: تضمينات الكلمات (Word2Vec وGloVe) — كيف تتحول الكلمة إلى إحداثيات في فضاء المعنى 66

الفصل 6: تضمينات الجُمل — RNN وLSTM وGRU: كيف تتذكر الشبكة ما نسيته؟ 81

الجزء الثالث: المحوّلات — القلب النابض للنماذج اللغوية 95

الفصول 7–11

الفصل 7: الدافع وراء آلية الانتباه الذاتي — اللحظة التي توقفت فيها الشبكة عن النسيان 96

الفصل 8: بنية المحوّل — من كتلة واحدة إلى معمارية كاملة 108

الفصل 9: التنويعات الكبرى — BERT وGPT وT5: ثلاث فلسفات في بناء العقل 119

الفصل 10: ما بعد المحوّل الكلاسيكي — FlashAttention وMLA وMoE وMamba 130

الفصل 11: تسريع العمليات الحسابية — عندما تصبح السرعة قرارًا هندسيًا 140

الجزء الرابع: بناء LLM من الصفر — خط الأنابيب الكامل 150

الفصول 12–19

الفصل 12: بناء LLM من الصفر — تخطيط المشروع وإعداد البيئة (مع كود كامل لكل خطوة) 151

الفصل 13: بناء Tokenizer وتجهيز البيانات بعمق — المطبخ الخفي الذي يحدد جودة النموذج 165

الفصل 14: بناء النموذج — من Embedding إلى GPT Block (كود كامل، مختبر، خطوة بخطوة) 180

الفصل 15: حلقة التدريب (Pre-training) — حيث يتحول النموذج من ورق إلى عقل 195

الفصل 16: الضبط الدقيق (SFT) والضبط الفعّال (LoRA/QLoRA) — كيف تعلّم نموذجًا أن يطيع دون أن تعيد بناءه 213

الفصل 17: ضبط التفضيلات (RLHF وDPO) — كيف تعلّم النموذج أن يختار "الأفضل" لا "الصحيح" فقط 225

الفصل 18: الاستدلال والتوليد — كيف يختار النموذج كلمته التالية؟ 237

الفصل 19: التكميم والتقطير — كيف تصنع نموذجًا أصغر لا يخسر عقله 250

الجزء الخامس: التقنيات المتقدمة — من هندسة الأوامر إلى الوكلاء 259

الفصول 20–24

الفصل 20: هندسة الأوامر (Prompt Engineering) — فن مخاطبة عقل لا تراه 260

الفصل 21: الضبط الدقيق الفعّال (PEFT) المتقدم — عندما يكون "الأقل" أكثر 272

الفصل 22: ضبط التفضيلات المتقدم — ORPO وKTO وSimPO وGRPO: عائلة DPO وأبناء العمومة 285

الفصل 23: التوليد المعزّز بالاسترجاع (RAG) — كيف تجعل النموذج يجيب من كتب لم يقرأها 299

الفصل 24: الوكلاء (Agents) واستخدام الأدوات — عندما يتحول النموذج من مُجيب إلى فاعل 314

الجزء السادس: النماذج متعددة الوسائط (Multimodal) (LLMs) 328

الفصول 25–28

الفصل 25: مدخل إلى النماذج متعددة الوسائط (Multimodal) — عندما يفتح النموذج عينيه 329

الفصل 26: معماريات Vision-Language المتقدمة — خمس فلسفات في رؤية واحدة 344

الفصل 27: بناء VLM عمليًا من نموذج موجود — من الصفر إلى نموذج يرى بالعربية 356

الفصل 28: تقييم النماذج متعددة الوسائط — كيف تعرف أن النموذج "يرى" فعلًا؟ 371

الجزء السابع: البناء انطلاقًا من نماذج موجودة — بدلًا من الصفر 382

الفصول 29–34

الفصل 29: لماذا لا تبدأ من الصفر؟ — التكلفة والوقت والبيانات: حساب الصفقة 383

الفصل 30: التقطير (Knowledge Distillation) — كيف تعلّم نموذجًا صغيرًا أن يفكر مثل معلمه 393

الفصل 31: الدمج (Model Merging) — عندما يصبح 1+1 أكبر من 2 405

الفصل 32: التشذيب (Pruning) واستعادة الجودة — كيف تحذف من النموذج دون أن تفقد عقله 419

الفصل 33: التخصيص السريع (LoRA وAdapters) كنمط بناء — كيف تخدم مئة عميل بنموذج واحد 431

الفصل 34: Self-Improvement وSynthetic Data — كيف يتفوق النموذج على نفسه 443

الجزء الثامن: التقييم والاختبار والنشر 454

الفصول 35–40

الفصل 35: مقاييس التقييم الأساسية — الأرقام التي تخبرك الحقيقة (وأحيانًا تكذب) 455

الفصل 36: المعايير الحديثة — MMLU وBIG-Bench وHELM: كيف نقيس "الذكاء العام"؟ 466

الفصل 37: LLM-as-a-Judge — عندما يصبح النموذج قاضيًا على نفسه 479

الفصل 38: تقييم النماذج متعددة الوسائط والوكلاء — كيف تقيس "الرؤية" و"الفعل"؟ 492

الفصل 39: الأمان والخصوصية والتحيز — عندما يصبح النموذج خطرًا دون أن يدري 504

الفصل 40: النشر (Deployment) — من المختبر إلى الإنتاج: كيف تبني نظامًا لا ينهار 517

الجزء التاسع: التطبيقات والمشاريع العملية 531

الفصول 41–45

الفصل 41: تحليل المشاعر بالعربية — عندما تصبح "ما شاء الله" سخرية 532

الفصل 42: الترجمة الآلية — عندما تخون الكلمة مترجمها 545

الفصل 43: RAG للمستندات العربية — عندما تقرأ الآلة ما كتبه البشر بيدهم 556

الفصل 44: تلخيص المستندات الطويلة — عندما ينسى النموذج ما في منتصف الكتاب 568

الفصل 45: توليد الكود وتحليله — عندما يكتب النموذج كودًا يعمل، لكنه يقتل 581

الجزء العاشر: المقابلات والمسار المهني 593

الفصول 46–48

الفصل 46: أسئلة المقابلات — خريطة النجاة في بحر الأسئلة 594

الفصل 47: أنماط تصميم أنظمة LLM — عندما يصبح النموذج مجرد لبنة 606

الفصل 48: أفضل الممارسات الهندسية والمسار المهني — الرحلة لا تنتهي عند النشر 620

الملاحق

ملحق أ: مسرد المصطلحات (عربي / إنجليزي) 634

ملحق ب: الأوراق البحثية الأساسية (مرتبة حسب الموضوع) 640

ملحق ج: الأدوات والبيئات 644

ملحق د: الحلول والتلميحات للتمارين 649

ملحق هـ: مشروع تخرج متكامل 652

ملحق و: قوالب جاهزة 656

ملحق ز: خريطة تعلم مقترحة (12 أسبوعًا) 660

ملحق ح: قائمة تحقق نهائية (Checklist) 662

ملحق ط: موارد إضافية 664

تأسيسي

الهداء

إلى **كل من سأل "كيف يعمل؟"**، ولم يكتفِ بـ"ماذا يفعل".

إلى **كل من فتح دفتر Jupyter** في منتصف الليل، وظل يبحث عن الخطأ في `.view()` و `transpose`.

إلى **كل من قرأ ورقة بحثية**، ولم يفهم كل كلمة، لكنه لم يتوقف.

إلى **كل من درّب نموذجًا**، ورأى الخسارة تنخفض، فشعر أن العالم تغير.

إلى **كل من فشل**، ثم عاد، ثم نجح.

إلى **العربية**، التي ظُلمت في الترميز، والتقييم، والبيانات. عسى أن يكون هذا الكتاب خطوة صغيرة في رد الاعتبار.

إلى **كل معلم** علّم بلا مقابل، وكل مهندس شارك كودًا بلا شروط، وكل باحث نشر ورقة بلا حجب.

إلى **من سيأتي بعدنا**، ويكمل الطريق.

هذا الكتاب لكم.

تأسيسي

المقدمة

لماذا هذا الكتاب؟ ولماذا الآن؟

في عام 2017، نشر ثمانية باحثين في Google ورقة بحثية بعنوان "Attention is All You Need". كانت الورقة قصيرة (11 صفحة)، لكنها كانت قنبلة فكرية. اقترحت معمارية جديدة، أسمتها **المحوّل (Transformer)**، تتخلى عن التكرار (Recurrence) والالتفاف (Convolution) تمامًا، وتعتمد على آلية واحدة: **الانتباه الذاتي (Self-Attention)**.

في ذلك الوقت، لم يدرك أحد أن هذه الورقة ستغير وجه الذكاء الاصطناعي. بعد خمس سنوات، أصبح المحوّل هو الأساس الذي تُبنى عليه كل النماذج اللغوية الكبيرة: Gemini، Claude، DeepSeek، Qwen، LLaMA، BERT، GPT. وبعد ثماني سنوات، أصبحت هذه النماذج جزءًا من حياتنا اليومية: في هواتفنا، في محركات البحث، في الترجمة، في البرمجة، في الطب، في التعليم.

لكن، ومع كل هذا الانتشار، ظهرت فجوة عميقة. **المعرفة موزعة**: الأوراق البحثية في arXiv، الدروس في المدونات، الأكواد في GitHub، النصائح في Twitter. لا يوجد **مرجع واحد** يجمع كل شيء: من الأساسيات الرياضية، إلى بناء نموذج من الصفر، إلى النشر في الإنتاج، إلى اجتياز مقابلة عمل. لا يوجد **دليل** يربط النظرية بالتطبيق، ويروي **لماذا** كل قطعة موجودة، و**متى** تفشل.

هذا الكتاب هو محاولة لسد تلك الفجوة.

لمن هذا الكتاب؟

هذا الكتاب **ليس** لكل شخص. إنه لكل من:

- **يريد أن يفهم** كيف تعمل النماذج اللغوية الكبيرة، وليس فقط كيف يستخدمها.
- **يريد أن يبني** نموذجًا من الصفر، وليس فقط استدعاء API.
- **يريد أن ينشر** نظامًا في الإنتاج، وليس فقط تجربة في دفتر Jupyter.
- **يريد أن يجتاز** مقابلة عمل في LLM، وليس فقط حفظ الإجابات.
- **يريد أن يفهم** العربية في النماذج، وليس فقط الإنجليزية.

إن كنت تبحث عن "دليل سريع"، أو "قائمة أوامر جاهزة"، أو "حل سحري"، فهذا الكتاب ليس لك. أما إن كنت تريد **فهمًا عميقًا**، **تطبيقًا عمليًا**، و**حكمة هندسية**، فهذا الكتاب كُتب لك.

ما يميز هذا الكتاب؟

1. الرحلة الكاملة، وليس القطع المتناثرة

هذا الكتاب يأخذك في رحلة **متكاملة**: من الخلية العصبية الواحدة، إلى آلية الانتباه، إلى بناء MiniGPT، إلى الضبط الدقيق، إلى RAG، إلى الوكلاء، إلى الوسائط المتعددة، إلى النشر، إلى المقابلات. كل فصل يبني على ما قبله. لا تبدأ من المنتصف. لا تقفز إلى النهاية. أنت **تبني**، خطوة بخطوة.

2. الكود، وليس فقط النظرية

كل مفهوم نظري في هذا الكتاب يقابله **كود قابل للتشغيل**. لن تكتفي بقراءة معادلة الانتباه. ستكتبها. لن تكتفي بفهم LoRA. ستنفذها. لن تكتفي بقراءة عن RAG. ستبني نظامًا كاملًا. الكود هو **الجسر** بين الفهم والتطبيق.

3. العربية، وليس الترجمة الحرفية

هذا الكتاب كُتب **بالعربية**، وليس مترجمًا. الأمثلة عربية. التحديات عربية. اللغة العربية ليست "إضافة"، بل **موضوع**. ستتعلم كيف تتعامل مع التهكم العربي، اللهجات العربية، الترميز العربي، والخط العربي. ستكتشف أن العربية ليست "لغة أخرى"، بل **تحدٍّ فريد** يستحق كتابًا خاصًا.

4. الأخطاء، وليس فقط الحلول

كل فصل يحتوي على **قائمة سوداء** بالأخطاء الشائعة: ما الذي يفعله المبتدئون؟ لماذا يفشل النموذج؟ كيف تتجنب الفخاخ؟ هذا الكتاب لا يخبرك فقط "ماذا تفعل"، بل "ماذا لا تفعل"، و"لماذا".

5. الحكايات، وليس فقط الجداول

هذا الكتاب لا يكتفي بالتعريفات. كل فصل يبدأ بحكاية: طاهٍ، نحات، راعٍ، موظف، مبرمج. الحكاية ليست زينة. إنها **طريقة للفهم**. "حكاية المهندس الذي اشترى Ferrari ليقود في زحام" تشرح مشكلة النشر أكثر من أي جدول. "حكاية المترجم الذي ترجم عين خطأ" تشرح مشكلة الترجمة الآلية أكثر من أي معادلة.

6. 2026، وليس 2020

هذا الكتاب **محدث**. يغطي: RAG، GRPO، SimPO، KTO، ORPO، DPO، MoE، Mamba، MLA، FlashAttention-3، الوكلاء، Multi-LoRA، VLM، والتقييم الحديث. لا يكتفي بالتقنيات القديمة. يواكب **ما يحدث الآن**.

كيف تقرأ هذا الكتاب؟

هناك طرق متعددة لقراءة هذا الكتاب، حسب هدفك:

إذا كنت مبتدئًا: اقرأ بالترتيب. من الفصل 1 إلى الفصل 48. لا تتخطَّ. كل فصل يبني على ما قبله.

إذا كنت خبيرًا: اذهب إلى الأجزاء التي تهمك. الجزء الرابع (بناء LLM)، الجزء السابع (البناء من نماذج موجودة)، الجزء الثامن (التقييم والنشر). استخدم الملاحق كمرجع.

إذا كنت مستعجلًا: اقرأ الجزء الثالث (المحوّلات)، الجزء الرابع (بناء LLM)، والجزء الثامن (النشر). هذه هي **الأساسيات** التي لا غنى عنها.

إذا كنت تحضر مقابلة: اقرأ الفصل 46 (أسئلة المقابلات)، ثم ارجع إلى الفصول التي تحتاج مراجعتها.

> 💡 **نصيحة أخيرة**: لا تقرأ هذا الكتاب كرواية. اقرأه **بيديك**. شغّل الكود. عدّل الأكواد. أخطئ. أصلح. هذا هو الطريق الوحيد للفهم.

شكر وتقدير

هذا الكتاب لم يكن ليُكتب لولا جهود **آلاف** الباحثين والمهندسين الذين شاركوا أعمالهم: فريق المحوّل الأصلي، فريق GPT، فريق LLaMA، فريق Qwen، فريق DeepSeek، فريق Anthropic، وكل من نشر ورقة، أو كتب مدونة، أو شارك كودًا.

شكر خاص لـ **المجتمع العربي** في الذكاء الاصطناعي: من درّب نماذج عربية، من كتب أدوات، من شارك معرفة. أنتم الأساس.

وأخيرًا، شكرًا **لك**، عزيزي القارئ. لأنك اخترت أن **تفهم**، وليس فقط أن **تستخدم**. لأنك اخترت أن **تبني**، وليس فقط أن **تستهلك**. هذا الكتاب كُتب لك.

فلنبدأ.

تأسيسي

خطة الكتاب

نظرة عامة

"المحوّلات والنماذج اللغوية الكبيرة: من النظرية إلى بناء أنظمة ذكية" هو دليل شامل، عملي، ومحدث (2026) يأخذك في رحلة متكاملة من **الأساسيات الرياضية** إلى **بناء أنظمة LLM في الإنتاج**.

الكتاب مقسم إلى **عشرة أجزاء**، **48 فصلًا**، و**تسعة ملاحق**، ويغطي:

- **النظرية**: كيف تعمل الشبكات العصبية، المحوّلات، الانتباه، التضمينات.
- **البناء**: كيف تبني MiniGPT من الصفر، تدربه، تضبطه، تنشره.
- **التقنيات الحديثة**: RAG، DPO، LoRA، الوكلاء، VLM، التكميم، التقطير، الدمج.
- **التقييم**: كيف تقيس الجودة، الهلوسة، الأمان.
- **التطبيقات**: المشاعر، الترجمة، RAG عربي، التلخيص، الكود.
- **المسار المهني**: كيف تجتاز المقابلات، تبني أنظمة، تنمو مهنيًا.

الأجزاء العشرة

الجزء I: الأساسيات (الفصول 1–3)

ماذا ستتعلم: كيف تعمل الشبكة العصبية من الصفر، كيف تتجنب الإفراط في التخصيص، كيف تدرب نموذجًا حديثًا (Mixed Precision، Gradient Accumulation، DDP/FSDP).

لماذا يهم: بدون هذه الأساسيات، سيبقى فهمك للمحوّلات سطحيًا. هذا الجزء هو **الأرضية** التي يقف عليها كل شيء آخر.

الجزء II: التضمينات (الفصول 4–6)

ماذا ستتعلم: كيف يتحول النص إلى أرقام (Tokenization)، كيف تتعلم الكلمات معانيها (Word2Vec، GloVe)، كيف تتعلم الجمل تسلسلها (RNN، LSTM، GRU).

لماذا يهم: التضمينات هي **الجسر** بين اللغة والحساب. فهمها يجعل فهم المحوّلات أعمق.

الجزء III: المحوّلات (الفصول 7–11)

ماذا ستتعلم: آلية الانتباه الذاتي، بنية المحوّل الكاملة، BERT/GPT/T5، المعماريات الحديثة (FlashAttention، MLA، MoE، Mamba)، تسريع العمليات الحسابية.

لماذا يهم: هذا هو **القلب** النابض للنماذج اللغوية. بدون فهمه، لن تفهم أي شيء آخر.

الجزء IV: بناء LLM من الصفر (الفصول 12–19)

ماذا ستتعلم: كيف تخطط لمشروع، تجهز البيانات، تبني MiniGPT، تدربه، تضبطه (SFT/LoRA)، تحسّن تفضيلاته (DPO)، تولّد النص، تضغطه (تكميم/تقطير).

لماذا يهم: هذا هو **الجزء العملي** الأهم. ستخرج منه بنموذج يعمل، وليس فقط معرفة نظرية.

الجزء V: التقنيات المتقدمة (الفصول 20–24)

ماذا ستتعلم: هندسة الأوامر، PEFT المتقدم (IA3، Prefix، Prompt Tuning)، ضبط التفضيلات المتقدم (ORPO، KTO، SimPO، GRPO)، RAG، الوكلاء.

لماذا يهم: هذه هي **الأدوات** التي تستخدمها في الإنتاج. بدونها، نموذجك لن يكون مفيدًا.

الجزء VI: الوسائط المتعددة (الفصول 25–28)

ماذا ستتعلم: كيف يرى النموذج الصور، معماريات VLM، بناء VLM عربي، تقييم VLM.

لماذا يهم: النماذج متعددة الوسائط هي **المستقبل**. من لا يعرفها، سيتخلف.

الجزء VII: البناء من نماذج موجودة (الفصول 29–34)

ماذا ستتعلم: لماذا لا تبدأ من الصفر، التقطير، الدمج (TIES، DARE)، التشذيب، Multi-LoRA، Self-Improvement.

لماذا يهم: 99% من المشاريع لا تحتاج إلى Pre-training من الصفر. هذا الجزء يخبرك **كيف تبني بذكاء**، وليس بتهور.

الجزء VIII: التقييم والنشر (الفصول 35–40)

ماذا ستتعلم: المقاييس الأساسية (PPL، F1، BLEU، ROUGE)، المعايير الحديثة (MMLU، BBH، HELM)، LLM-as-a-Judge، تقييم VLM والوكلاء، الأمان، النشر.

لماذا يهم: النموذج الذي لا يُقاس، لا يُعرف. والنموذج الذي لا يُنشر، لا يُستخدم.

الجزء IX: التطبيقات (الفصول 41–45)

ماذا ستتعلم: تحليل المشاعر العربي، الترجمة الآلية، RAG للمستندات العربية، تلخيص المستندات الطويلة، توليد الكود.

لماذا يهم: هذه هي **المشاريع الحقيقية** التي ستواجهها. كل فصل هو مشروع كامل، من البيانات إلى النشر.

الجزء X: المقابلات والمسار المهني (الفصول 46–48)

ماذا ستتعلم: أسئلة المقابلات، أنماط تصميم الأنظمة، أفضل الممارسات الهندسية، المسار المهني.

لماذا يهم: المعرفة وحدها لا تكفي. تحتاج إلى **إيصالها**، و**تطبيقها**، و**النمو**.

ما يميز هذا الكتاب

الميزة	التفصيل
شامل	من الأساسيات إلى الإنتاج، في كتاب واحد

الميزة	التفصيل
عملي	كود كامل، قابل للتشغيل، لكل مفهوم
محدث	يغطي تقنيات 2026 (FlashAttention-3، MLA، Mamba، GRPO)
عربي	أمثلة عربية، تحديات عربية، حلول عربية
نقدي	يذكر حدود كل تقنية، وأخطاءها الشائعة
قصصي	حكايات تجعل المفاهيم المجردة ملموسة
مرجعي	ملاحق شاملة: مسرد، أوراق، أدوات، قوالب

لمن هذا الكتاب؟

- **طلاب** الهندسة وعلوم الحاسوب.
- **مهندسو** البرمجيات الذين يريدون التحول إلى AI.
- **باحثو** NLP الذين يريدون فهمًا عمليًا.
- **مهندسو** ML الذين يريدون بناء LLM.
- **أي شخص** يريد اجتياز مقابلة في LLM.

ماذا ستتعلم في النهاية؟

بعد قراءة هذا الكتاب **وتطبيق الكود**، ستكون قادرًا على:

- ✅ بناء MiniGPT من الصفر، وتدريبه على بيانات عربية.
- ✅ ضبط نموذج موجود (LoRA/QLoRA) على مهمتك.
- ✅ تحسين تفضيلات النموذج (DPO/ORPO).
- ✅ بناء نظام RAG للمستندات العربية.
- ✅ بناء وكيل يستخدم أدوات بأمان.
- ✅ بناء VLM عربي بسيط.
- ✅ تقييم النماذج (آلي، بشري، LLM-as-a-Judge).
- ✅ نشر النموذج في الإنتاج (vLLM، FastAPI، مراقبة).
- ✅ حماية النظام (Prompt Injection، PII، تحيز).
- ✅ اجتياز مقابلة LLM بثقة.

هذا ليس كتابًا للقراءة. إنه كتاب للبناء.

01

الجزء الأول

الأساسيات — من الشبكات العصبية إلى التعلّم العميق

الفصول 1–3 · 3 فصولًا

المحوّلات والنماذج اللغوية الكبيرة · من النظرية إلى بناء أنظمة ذكية

الفصل 1 • الجزء 1

01

الشبكات العصبية من الصفر — كيف تتعلم الآلة أن "تخمّن" بشكل أفضل

افتتاحية: حكاية الراعي الذي لم يكن يعرف الرياضيات

في قرية صغيرة على سفح جبل، كان هناك راعٍ اسمه "سالم". لم يدرس الرياضيات يومًا، لكنه كان يعرف تمامًا متى ستمطر السماء. كان ينظر إلى السحاب، ويشمّ الهواء، ويلاحظ سلوك الأغنام، ويقول: "الليلة مطر". وكان غالبًا على حق.

سأله أحدهم: "كيف تعرف؟" أجاب: "لا أعرف. لكن كلما رأيت سحابًا رماديًا كثيفًا، ورائحة تراب، وأغنامًا تتجمع، أعرف أن المطر قادم".

سالم، دون أن يدري، كان يشغّل **شبكة عصبية** في رأسه. المدخلات: لون السحاب، الرائحة، سلوك الأغنام. المخرج: "مطر" أو "لا مطر". والأوزان التي يمنحها لكل إشارة تعلمها من **تجربته** عبر السنين. كلما أخطأ، عدّل أوزانه الداخلية. هذا بالضبط ما نسميه في التعلّم العميق: **الانتشار الأمامي (Forward Propagation)** ثم **الانتشار العكسي (Backpropagation)** ثم **تحديث الأوزان (Weight Update)**.

هذا الفصل هو محاولة لفهم ما فعله سالم في رأسه، لكن بلغة الرياضيات والكود. سنبدأ من **الخلية العصبية الواحدة**، ثم نبني شبكة كاملة، ثم ندربها. وسترى أن الفكرة كلها ليست سحرًا، بل **حساب تفاضلي** مع قليل من الصبر.

1.1 الخلية العصبية الواحدة: أبسط وحدة تعلم

الفكرة في سطر واحد

الخلية العصبية (Neuron) تأخذ مجموعة أرقام، تضرب كل رقم بوزن، تجمعها، تضيف انحيازًا (bias)، ثم تمرر الناتج عبر دالة تنشيط. هذا كل شيء.

رياضيًا:

$$z = \Sigma_{i=1}^{n} w_i x_i + b = w^T x + b$$

$$a = \sigma(z)$$

حيث:

- x: المدخلات (مثلًا: [طول السحاب، كثافته، رطوبة الهواء]).
- w: الأوزان (كم يهم كل مدخل؟).
- b: الانحياز (ميل الخلية الافتراضي).
- σ: دالة التنشيط (مثل Sigmoid أو ReLU).

- a: المخرج (تنبؤ الخلية).

تشبيه من الحياة

تخيل أنك تريد تقييم مطعم. لديك ثلاثة معايير: الطعام (وزن 0.5)، الخدمة (وزن 0.3)، السعر (وزن 0.2). إذا كان الطعام 9/10، والخدمة 7/10، والسعر 6/10، فالتقييم = $0.5 \times 9 + 0.3 \times 7 + 0.2 \times 6 = 4.5 + 2.1 + 1.2 = 7.8$. هذه "خلية عصبية" بدون دالة تنشيط. الأوزان هنا قررتها أنت يدويًا. في الشبكات العصبية، **النموذج يتعلمها بنفسه**.

دالة التنشيط: لماذا لا نستغني عنها؟

سؤال يستحق التوقف. لو كانت الخلية مجرد $z = w^T x + b$، فسيكون النموذج **خطيًا** بالكامل. وحتى لو رصصت مليون طبقة خطية فوق بعضها، فالناتج النهائي يبقى دالة خطية واحدة (لأن تركيب الدوال الخطية خطي). هذا يعني أن النموذج لن يستطيع تعلم أي علاقة غير خطية، مثل "XOR" أو "هذه الصورة قطة".

دالة التنشيط تكسر الخطية. أشهرها:

الدالة	الصيغة	متى تستخدمها	تحذير
Sigmoid	$(1)/(1+e^{-z})$	مخرج ثنائي (0/1) في الطبقة الأخيرة	تعاني من تلاشي التدرج (Vanishing Gradient) في الطبقات العميقة
Tanh	$(e^{z} - e^{-z})/(e^{z} + e^{-z})$	طبقات مخفية قديمة	نفس مشكلة Sigmoid لكن متمركزة حول الصفر
ReLU	$\max(0, z)$	الخيار الافتراضي للطبقات المخفية	قد "تموت" بعض الخلايا (Dead ReLU) إذا كانت المدخلات سالبة دائمًا
Leaky ReLU	$\max(0.01z, z)$	بديل لـ ReLU لتجنب موت الخلايا	معامله الصغير قد لا يكفي أحيانًا
GELU	$z \cdot \Phi(z)$	المحوّلات الحديثة	أغلى حسابيًا قليلًا من ReLU
SiLU/Swish	$z \cdot \sigma(z)$	نماذج LLaMA وSwiGLU	شائع جدًا في 2024–2026

نصيحة: في نماذج المحوّلات الحديثة، **GELU** أو **SiLU** هي الخيارات السائدة. ReLU ما زالت رائعة في الشبكات التلافيفية (CNNs) والمهام التقليدية. لا تستخدم Sigmoid في الطبقات المخفية إلا إذا كنت تختبر شيئًا محددًا.

1.2 من خلية واحدة إلى شبكة: الطبقات والانتشار الأمامي

الشبكة كسلسلة من التحويلات

الشبكة العصبية هي ببساطة **تكديس لطبقات**. كل طبقة تأخذ مخرجات الطبقة السابقة، وتحوّلها:

$$a^{(1)} = \sigma(W^{(1)} x + b^{(1)})$$

$$a^{(2)} = \sigma(W^{(2)} a^{(1)} + b^{(2)})$$

$$\vdots$$

$$\hat{y} = \text{softmax}(W^{(L)}\, a^{(L-1)} + b^{(L)})$$

هذه العملية تسمى **الانتشار الأمامي**. نمرر المدخلات عبر الطبقات، ونحصل على تنبؤ.

كود: شبكة بسيطة بـ NumPy

سنبني شبكة من طبقة مخفية واحدة، للتصنيف الثنائي. لا PyTorch، لا TensorFlow. فقط NumPy. الهدف: أن ترى **كل رقم** يتحرك.

```python
import numpy as np

# ===== 1. توليد بيانات اصطناعية (مشكلة XOR) =====
# XOR: مخرج = 1 إذا اختلف المدخلان، و0 إذا اتفقا
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=np.float32)
y = np.array([[0], [1], [1], [0]], dtype=np.float32)

# ===== 2. تهيئة الأوزان =====
np.random.seed(42)
n_input = 2
n_hidden = 4
n_output = 1

# تهيئة Xavier/Glorot: تقلل مشاكل التدرج
W1 = np.random.randn(n_input, n_hidden) * np.sqrt(1.0 / n_input)
b1 = np.zeros((1, n_hidden))
W2 = np.random.randn(n_hidden, n_output) * np.sqrt(1.0 / n_hidden)
b2 = np.zeros((1, n_output))

# ===== 3. دوال التنشيط =====
def sigmoid(z):
    # clip لتجنب overflow
    z = np.clip(z, -500, 500)
    return 1.0 / (1.0 + np.exp(-z))

def sigmoid_derivative(a):
    # a هو المخرج بعد sigmoid
    return a * (1 - a)

# ===== 4. الانتشار الأمامي =====
def forward(X, W1, b1, W2, b2):
    z1 = X @ W1 + b1
    a1 = sigmoid(z1)
    z2 = a1 @ W2 + b2
    a2 = sigmoid(z2)
    return z1, a1, z2, a2

# ===== 5. دالة الخسارة (Binary Cross-Entropy) =====
def bce_loss(y_true, y_pred):
    eps = 1e-8  # لمنع log(0)
    y_pred = np.clip(y_pred, eps, 1 - eps)
    return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

# ===== 6. الانتشار العكسي =====
def backward(X, y, z1, a1, z2, a2, W2):
    m = X.shape[0]  # حجم الدفعة

    # تدرج الخسارة بالنسبة لمخرجات الطبقة الأخيرة
    dz2 = a2 - y                          # (m, 1)
    dW2 = a1.T @ dz2 / m                  # (n_hidden, 1)
```

```
    db2 = np.sum(dz2, axis=0, keepdims=True) / m

    # تدرج الطبقة المخفية
    da1 = dz2 @ W2.T                    # (m, n_hidden)
    dz1 = da1 * sigmoid_derivative(a1)  # (m, n_hidden)
    dW1 = X.T @ dz1 / m                 # (n_input, n_hidden)
    db1 = np.sum(dz1, axis=0, keepdims=True) / m

    return dW1, db1, dW2, db2

# ===== 7. حلقة التدريب =====
learning_rate = 0.5
epochs = 5000

for epoch in range(epochs):
    z1, a1, z2, a2 = forward(X, W1, b1, W2, b2)
    loss = bce_loss(y, a2)
    dW1, db1, dW2, db2 = backward(X, y, z1, a1, z2, a2, W2)

    W1 -= learning_rate * dW1
    b1 -= learning_rate * db1
    W2 -= learning_rate * dW2
    b2 -= learning_rate * db2

    if epoch % 1000 == 0:
        print(f"Epoch {epoch:5d} | Loss: {loss:.6f}")

# ===== 8. النتائج =====
_, _, _, preds = forward(X, W1, b1, W2, b2)
print("\nالتنبؤات بعد التدريب:")
for i in range(len(X)):
    print(f"  {X[i]} -> {preds[i][0]:.4f} (المتوقع: {y[i][0]})")
```

ماذا يحدث في هذا الكود؟

1. **البيانات**: مشكلة XOR، وهي المشكلة الكلاسيكية التي **لا يمكن** لشبكة خطية حلها. هذا اختبار مهم: إذا كانت شبكتك تحلها، فهي بالفعل غير خطية.
2. **التهيئة**: استخدمنا Xavier، الذي يضبط تباين الأوزان بناءً على عدد المدخلات، لتجنب انفجار أو تلاشي التدرجات منذ البداية.
3. **الخسارة**: Binary Cross-Entropy، الأنسب للتصنيف الثنائي.
4. **الانتشار العكسي**: لاحظ `dz2 = a2 - y`. هذا تبسيط جميل ينتج من اشتقاق BCE مع Sigmoid. لو استخدمت دالة خسارة أخرى، ستحتاج اشتقاقًا مختلفًا.
5. **التحديث**: `W -= lr * dW`. لاحظ علامة الناقص: نتحرك **عكس** اتجاه التدرج، لأننا نريد تقليل الخسارة.

> 🤔 **سؤال يستحق التفكير**: لماذا قسمنا على `m` (حجم الدفعة) في حساب التدرجات؟ الجواب: لأن دالة الخسارة تحسب **المتوسط**، فالاشتقاق يجب أن يعكس ذلك. لو نسيت القسمة، فإن معدل التعلم الفعلي سيتضاعف مع حجم الدفعة، وقد ينفجر التدريب.

1.3 الانتشار العكسي: القصة الكاملة

لماذا هذا الجزء "مخيف" لكنه منطقي جدًا؟

الانتشار العكسي هو مجرد **قاعدة السلسلة (Chain Rule)** من حساب التفاضل، مطبقة على رسم بياني حسابي. لا أكثر. لكن المبتدئين يخافون منه لأنه يبدو كأنه "سحر أسود" من الأوراق البحثية.

الفكرة:

1. نحسب الخسارة في نهاية الشبكة.
2. نسأل: **كيف تتغير الخسارة إذا تغير كل وزن قليلًا؟**
3. نبني الإجابة طبقةً طبقة، من النهاية إلى البداية.

مخطط بسيط

```text
X المدخلات
   ↓
[1 الطبقة] → a1
   ↓
[2 الطبقة] → a2 (التنبؤ)
   ↓
L الخسارة

الانتشار العكسي:
L → dL/da2 → dL/dz2 → dL/dW2 → dL/da1 → dL/dz1 → dL/dW1
```

كل سهم في الاتجاه العكسي يستخدم **قاعدة السلسلة**: إذا كان L يعتمد على a، وa يعتمد على z، وz يعتمد على w، فإن:

$$\partial L/\partial w = \partial L/\partial a \cdot \partial a/\partial z \cdot \partial z/\partial w$$

كود: التحقق من التدرجات (Gradient Checking)

هذه واحدة من **أهم العادات** التي تميز المهندس المحترف عن الهاوي. الفكرة: احسب التدرج تحليليًا (كما فعلنا)، ثم احسبه عدديًا بتقريب صغير، وقارن. إذا اختلفا بشكل كبير، فهناك خطأ في الكود.

```python
def numerical_gradient(f, x, eps=1e-6):
    """تقريب التدرج عدديًا باستخدام الفروق المركزية."""
    grad = np.zeros_like(x)
    it = np.nditer(x, flags=['multi_index'], op_flags=['readwrite'])
    while not it.finished:
        idx = it.multi_index
        old = x[idx]
        x[idx] = old + eps
        fx_plus = f(x)
        x[idx] = old - eps
        fx_minus = f(x)
        x[idx] = old
        grad[idx] = (fx_plus - fx_minus) / (2 * eps)
        it.iternext()
    return grad

# استخدامه للتحقق من W1
```

```
def loss_for_W1(W1_val):
    W1_tmp = W1_val.copy()
    _, _, _, a2 = forward(X, W1_tmp, b1, W2, b2)
    return bce_loss(y, a2)

# التدرج التحليلي (الذي حسبناه في الكود السابق)
z1_, a1_, z2_, a2_ = forward(X, W1, b1, W2, b2)
dW1_analytic, _, _, _ = backward(X, y, z1_, a1_, z2_, a2_, W2)

# التدرج العددي
dW1_numeric = numerical_gradient(loss_for_W1, W1.copy())

diff = np.abs(dW1_analytic - dW1_numeric).max()
print(f"أقصى فرق بين التدرجين: {diff:.2e}")
# فالتدرج سليم، إذا كان الفرق < 1e-6
```

🏆 **نصيحة ذهبية**: قبل أن تبدأ تدريب أي نموذج مخصص، **تحقق من التدرجات**. خطأ واحد في الاشتقاق قد يجعلك تدرب لساعات وتتساءل لماذا لا يتعلم النموذج. هذا اختبار يستغرق دقيقتين ويوفر عليك أيامًا.

⚠️ **تحذير**: التحقق العددي مكلف جدًا (يحسب الدالة مرتين لكل معامل). استخدمه فقط على شبكات صغيرة أو على عينة من المعاملات، ثم أوقفه.

1.4 دوال الخسارة: كيف نقيس "الخطأ"؟

الخسارة ليست مجرد رقم

دالة الخسارة تخبر النموذج **ماذا يعني أن يكون مخطئًا**. اختيار خاطئ لدالة الخسارة يمكن أن يجعل النموذج يتعلم شيئًا مختلفًا تمامًا عما تريد.

أشهر دوال الخسارة

الدالة	الاستخدام	الصيغة	ملاحظات
MSE (Mean Squared Error)	الانحدار (Regression)	$1/n\Sigma(y - \hat{y})^2$	حساسة للقيم الشاذة
MAE (Mean Absolute Error)	الانحدار	$1/n\Sigma \lvert y - \hat{y} \rvert$	أقل حساسية للشواذ
BCE (Binary Cross-Entropy)	التصنيف الثنائي	$-1/n\Sigma[y\log\hat{y} + (1-y)\log(1-\hat{y})]$	الأفضل مع Sigmoid
CCE (Categorical Cross-Entropy)	التصنيف المتعدد	$-\Sigma\, y_i \log\hat{y}_i$	الأفضل مع Softmax
Focal Loss	تصنيف غير متوازن	$-\alpha\,(1-\hat{y})^{\gamma}\log\hat{y}$	تركز على العينات الصعبة
KL Divergence	التقطير وتوزيعات الاحتمال	$\Sigma\, p \log(p/q)$	ليست متماثلة!

❌ **خطأ شائع**: استخدام MSE مع مخرجات Softmax في التصنيف. النتيجة: تدرجات صغيرة جدًا وتدريب بطيء. استخدم CCE (أو `CrossEntropyLoss` في PyTorch، التي تجمع Softmax وCCE معًا).

الفرق بين BCE وCCE في سطر

- **BCE**: كل عينة يمكن أن تنتمي إلى فئة أو أكثر (multi-label).
- **CCE**: كل عينة تنتمي إلى فئة واحدة فقط (multi-class).

اختيار خاطئ بينهما سيسبب تدريبًا غير مستقر أو تفسيرات خاطئة.

1.5 الانتقال إلى PyTorch: نفس الفكرة، بأدوات أفضل

لماذا لا نبقى في NumPy؟

لأن PyTorch يقدم:

1. **الحساب التلقائي للتدرجات** (Autograd): لا تكتب الاشتقاق يدويًا.
2. **GPU**: تسريع هائل.
3. **محسّنات جاهزة**: Adam, AdamW, SGD مع الزخم.
4. **وحدات جاهزة**: طبقات، دوال خسارة، تحميل بيانات.

لكن **لا تنسَ NumPy**. من فهم الفكرة في NumPy، سيفهم PyTorch كأداة تسهيل، لا كصندوق أسود.

كود: نفس مشكلة XOR بـ PyTorch

PYTHON

```python
import torch
import torch.nn as nn
import torch.optim as optim

# البيانات
X = torch.tensor([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=torch.float32)
y = torch.tensor([[0], [1], [1], [0]], dtype=torch.float32)

# النموذج
class XORNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(2, 4),
            nn.Sigmoid(),
            nn.Linear(4, 1),
            nn.Sigmoid(),
        )

    def forward(self, x):
        return self.net(x)

model = XORNet()

# الخسارة والمحسّن
```

```
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.05)

# التدريب
for epoch in range(2000):
    optimizer.zero_grad()
    preds = model(X)
    loss = criterion(preds, y)
    loss.backward()       # Autograd يحسب التدرجات
    optimizer.step()      # تحديث الأوزان

    if epoch % 500 == 0:
        print(f"Epoch {epoch:4d} | Loss: {loss.item():.6f}")

# النتائج
with torch.no_grad():
    print("\nالتنبؤات:")
    print(model(X).detach().numpy().round(3))
```

ماذا تغيّر؟

- `loss.backward()` استبدلت `backward()` اليدوية.
- `optimizer.step()` استبدلت `W -= lr * dW`.
- `optimizer.zero_grad()` ضرورية، لأن PyTorch **يجمع** التدرجات افتراضيًا. نسيانها يعني تراكم التدرجات عبر الدفعات، وهذا قد يكون مقصودًا أحيانًا (Gradient Accumulation) لكنه خطأ في الحالة العادية.

❌ **خطأ شائع جدًا**: نسيان `optimizer.zero_grad()`. النتيجة: التدرجات تتراكم، والتدريب ينفجر أو يتصرف بغرابة. إذا وجدت نموذجك "يتعلم" ثم "يفقد ما تعلمه"، فهذا أول ما تفحصه.

⚖️ **مقارنة سريعة**: `SGD` مع `momentum=0.9` لا يزال قويًا في الرؤية الحاسوبية. `AdamW` / `Adam` هما الافتراضي في NLP والمحوّلات. `AdamW` يفصل تسوس الوزن (weight decay) عن تحديث التدرج، وهو الأفضل في النماذج الكبيرة.

1.6 الأخطاء الشائعة: قائمة "لا تفعل"

هذا القسم خلاصة سنوات من الأخطاء التي يقع فيها الجميع، بما فيهم أنا.

1. عدم تطبيع المدخلات

إذا كانت مدخلاتك بأحجام مختلفة جدًا (مثلًا: العمر من 0 إلى 100، والدخل من 0 إلى 1,000,000)، فستكون التدرجات غير متوازنة. بعض الأوزان ستتحدث بعنف، وبعضها بالكاد.

الحل: Normalize أو Standardize.

PYTHON

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)  # استخدم نفس المحول!
```

⚠️ **تحذير**: لا ت-fit على بيانات الاختبار. هذا "تسريب بيانات" (Data Leakage)، ويعطي نتائج مضللة.

2. معدل تعلم ثابت

معدل تعلم كبير جدًا: يتذبذب التدريب أو ينفجر. معدل صغير جدًا: يتقارب ببطء شديد، أو يعلق في منطقة مسطحة.

الحل: جدولة معدل التعلم (Learning Rate Schedule).

PYTHON

```
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs)
# أو
scheduler = optim.lr_scheduler.OneCycleLR(
    optimizer, max_lr=1e-3, total_steps=epochs * len(dataloader)
)
```

💡 **تلميح**: في المحوّلات، استخدم **Warmup** (تصاعد تدريجي في البداية) ثم **Cosine Decay**. هذا ما تفعله معظم النماذج المفتوحة.

3. تهيئة الأوزان بالأصفار

إذا بدأت كل الأوزان بصفر، فكل الخلايا في الطبقة نفسها ستحسب نفس الشيء، وستحصل على نفس التدرج، وستبقى متطابقة إلى الأبد. الشبكة لن تتعلم شيئًا.

الحل: تهيئة عشوائية مناسبة. `Xavier/Glorot` للـ Tanh/Sigmoid، `He/Kaiming` للـ ReLU، والتهيئة القياسية في PyTorch عادة جيدة.

4. عدم فصل بيانات التدريب والتحقق والاختبار

إذا دربت وراقبت على نفس البيانات، فسترى خسارة تنخفض بشكل جميل، ثم تتفاجأ بأن النموذج فاشل على بيانات جديدة. هذا **الإفراط في التخصيص (Overfitting)**.

الحل: قسّم بياناتك: 70–80% تدريب، 10–15% تحقق، 10–15% اختبار. **لا تلمس** مجموعة الاختبار إلا في النهاية.

5. تجاهل توازن الفئات

إذا كانت 95% من بياناتك من فئة واحدة، فقد يتعلم النموذج أن يقول دائمًا "الفئة الأولى" ويحقق دقة 95%، لكنه عديم الفائدة.

الحل: استخدم مقاييس مناسبة (F1، Precision/Recall، AUC)، و/أو أوزان فئات، و/أو إعادة عينات.

6. عدم ضبط البذرة العشوائية (Seed)

تريد إعادة إنتاج النتائج؟ اضبط البذور.

```python
import random, numpy as np, torch
def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
```

📌 **ملاحظة**: حتى مع البذور، قد تختلف النتائج قليلًا على GPU بسبب عمليات غير حتمية. لا تنزعج.

1.7 أفضل الممارسات: ما يفعله المحترفون فعلًا

1. **ابدأ بخط أساس بسيط (Baseline)**. قبل بناء شبكة عميقة، جرّب الانحدار اللوجستي أو شجرة قرار. إذا لم يتفوق التعلم العميق، فالمشكلة ليست في النموذج.
2. **راقب أكثر من الخسارة**. راقب الدقة، F1، توزيع التدرجات، وقت كل دفعة. لوحة مراقبة بسيطة توفر عليك ساعات.
3. **اكتب اختبارات للكود**. اختبار أن شكل المخرجات صحيح، أن الخسارة تنخفض على دفعة صغيرة (Overfit a single batch)، أن التدرجات ليست NaN.
4. **Overfit a single batch أولًا**. إذا لم يستطع نموذجك حفظ 8 عينات، فهناك خطأ في الكود، وليس في البيانات.
5. **استخدم `torch.no_grad()` في التقييم**. يوفر ذاكرة ووقتًا كبيرين.
6. **لا تعِد اختراع العجلة**. استخدم `nn.CrossEntropyLoss`، `nn.LayerNorm`، إلخ. لكن **افهم** ما تفعله.
7. **وثّق تجاربك**. اسم المجلد، البذرة، المعاملات، النتيجة. "لماذا هذا النموذج أفضل؟" سؤال ستحتاج الإجابة عليه لاحقًا.

1.8 تمرين ختامي: من XOR إلى مشكلة حقيقية

التمرين 1 (مبتدئ)

عدّل كود NumPy لتصنيف ثلاث فئات بدلًا من فئتين. استخدم Softmax وCross-Entropy. تحقق من التدرجات عدديًا.

التمرين 2 (متوسط)

خذ مجموعة بيانات Iris (3 فئات، 4 ميزات). ابنِ شبكة بـ PyTorch، وقارن بين:

- بدون تطبيع.
- مع StandardScaler.
- مع MinMaxScaler.

أي إعداد يتقارب أسرع؟ لماذا؟

التمرين 3 (متقدم)

ابنِ شبكة بثلاث طبقات مخفية. راقب توزيع التدرجات في كل طبقة أثناء التدريب. هل ترى **تلاشي التدرج**؟ جرّب:

- Sigmoid مقابل ReLU مقابل GELU.

- تهيئة Xavier مقابل He.
- بدون BatchNorm مقابل مع LayerNorm.

سجّل النتائج في جدول. ماذا تلاحظ؟

التمرين 4 (تحدٍّ)

استخدم `torch.autograd.gradcheck` للتحقق من دالة خسارة مخصصة كتبتها. ثم استخدم `torch.profiler` لمعرفة أين يقضي النموذج معظم الوقت. هل يمكنك تسريعه؟

خلاصة الفصل: ما يجب أن يبقى في ذهنك

1. **الشبكة العصبية = دالة قابلة للضبط**. الأوزان هي ما نتعلمه، والباقي (البنية، التنشيط، الخسارة) هو ما نصممه.
2. **الانتشار العكسي = قاعدة السلسلة**. لا سحر. إذا فهمت قاعدة السلسلة، فهمت Backprop.
3. **دالة التنشيط ضرورية للا خطية**. بدونها، الشبكة العميقة = نموذج خطي.
4. **دالة الخسارة تحدد "ماذا يعني أن تكون مخطئًا"**. اخترها بحكمة.
5. **PyTorch تسهّل، لا تلغي الفهم**. ابدأ بـ NumPy، ثم انتقل إلى PyTorch.
6. **الأخطاء الشائعة قاتلة**: عدم التطبيع، نسيان `zero_grad`، عدم فصل البيانات، معدل تعلم ثابت. تجنبها من البداية.
7. **تحقق من التدرجات**. هذه العادة وحدها قد توفر عليك أيامًا.

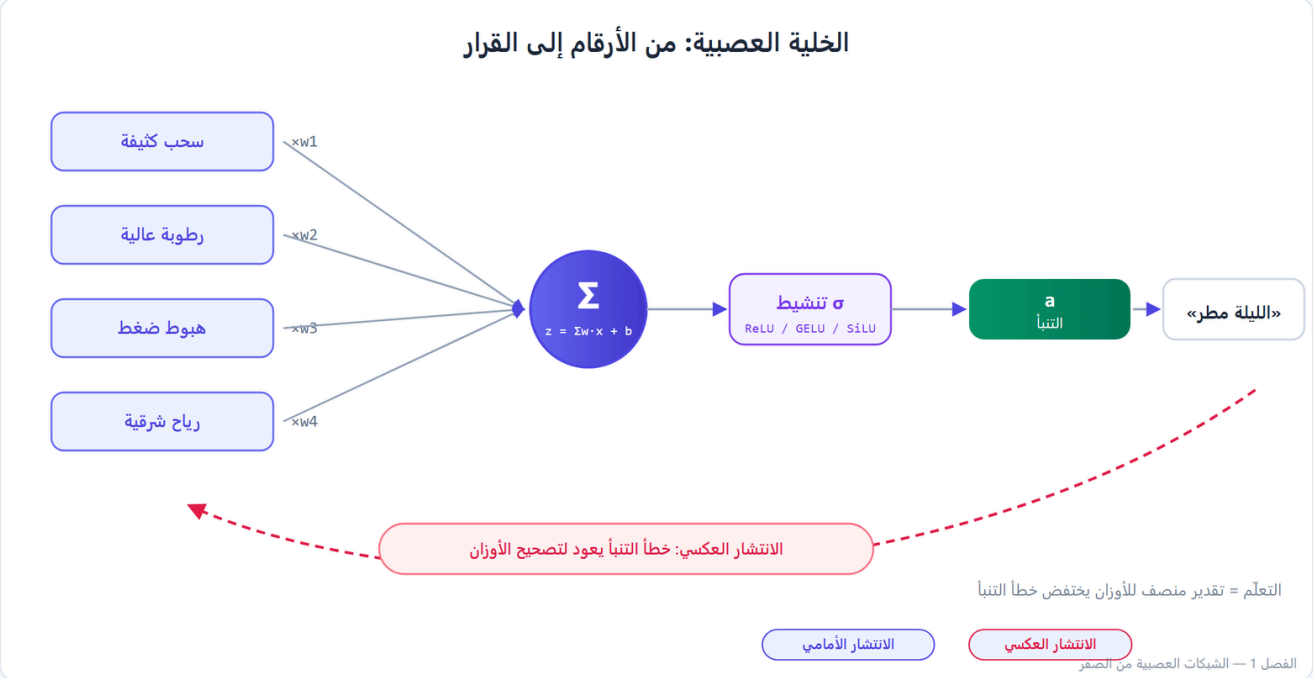
افتتاحية الفصل القادم: "مفاهيم التعلّم العميق للتدريب والتقييم"

في الفصل التالي، سنغادر XOR الصغيرة، ونواجه أسئلة أكبر: كيف نعرف أن نموذجنا **يتعلم فعلًا** وليس يحفظ؟ ما الفرق بين Overfitting وUnderfitting، وكيف نعالج كلًا منهما؟ ما هو Dropout، ولماذا يعمل رغم أنه يبدو "تخريبًا متعمدًا"؟ وكيف نختار معدل التعلم المناسب دون تخمين أعمى؟

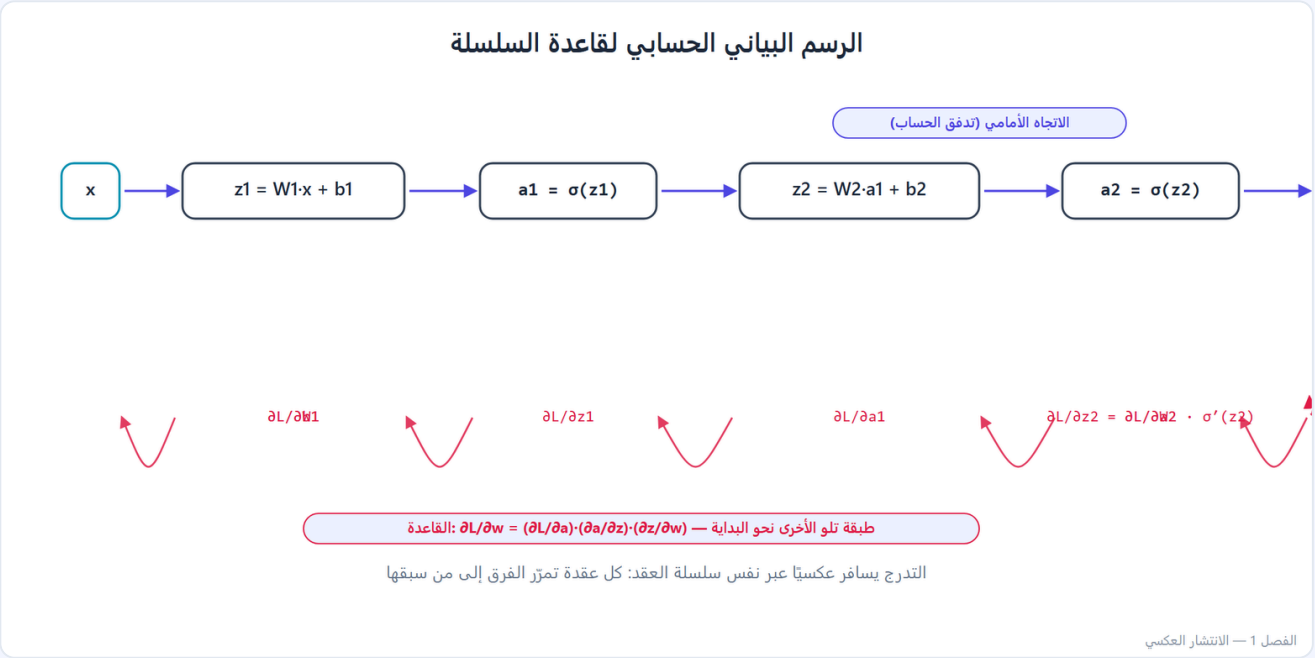
سنتعلم أيضًا كيف **نقرأ منحنيات التدريب** كما يقرأ الطبيب تخطيط القلب: كل انحناءة تحكي قصة. وكيف نبني **خط أساس** قبل أن نبني نموذجًا معقدًا، وكيف نتجنب "فخ الدقة" عندما تكون بياناتنا غير متوازنة.

> ❓ **سؤال يفتح الشهية**: هل تعلم أن شبكة عصبية بسيطة يمكنها حفظ بيانات عشوائية تمامًا (حتى لو كانت تسمياتها عشوائية)؟ ما معنى ذلك لأفكارنا عن "التعميم"؟ سنكتشف ذلك في الفصل القادم.

المخطط التوضيحي للفصل



شكل 1.1 — الخلية العصبية: من المدخلات إلى التنشيط



شكل 1.2 — الرسم البياني الحسابي لقاعدة السلسلة

02

مفاهيم التعلّم العميق للتدريب والتقييم — كيف تعرف أن نموذجك "يتعلم" فعلًا؟

افتتاحية: حكاية الطالب الذي حفظ الأسئلة

في إحدى المدارس، كان هناك طالب اسمه "كريم". كان كريم ذكيًا، لكنه كسول قليلًا. قبل الامتحان، حصل على نماذج الأسئلة السابقة، فحفظها كلها عن ظهر قلب. دخل الامتحان، ورأى نفس الأسئلة، فأجاب بسرعة ودقة مذهلة. حصل على 100%. احتفل الجميع بنبوغه.

لكن بعد شهر، جاء امتحان جديد، بأسئلة مختلفة قليلًا، لكنها تختبر نفس المفاهيم. كريم رسب. لماذا؟ لأنه **حفظ**، ولم **يتعلم**. حفظ الشكل، لا الجوهر. حفظ الإجابة، لا المنطق.

هذه هي قصة **الإفراط في التخصيص (Overfitting)** بالضبط. النموذج يحفظ بيانات التدريب، فيبدو رائعًا عليها، لكنه يفشل على أي بيانات جديدة. والمشكلة أنه **لا يخبرك** بأنه يحفظ. ستظن أنك أنجزت شيئًا عظيمًا، حتى تختبره في العالم الحقيقي.

هذا الفصل يدور حول سؤال واحد: **كيف نعرف أن نموذجنا يتعلم فعلًا، وليس يحفظ؟** وكيف نجعله يتعلم بدلًا من أن يحفظ؟

2.1 مفارقة التعميم: ما نريده حقًا

الهدف الحقيقي ليس تقليل خطأ التدريب

لو كان هدفنا الوحيد هو تقليل خطأ التدريب، فالحل سهل: اجعل النموذج ضخمًا بما يكفي ليحفظ كل شيء. لكن هذا بالضبط ما لا نريده. نريد نموذجًا يعمل جيدًا على بيانات **لم يرها من قبل**. هذه القدرة تسمى **التعميم (Generalization)**.

الفرق بين خطأ التدريب وخطأ الاختبار هو مقياسنا الحقيقي:

```
Generalization Gap = Test Error - Train Error
```

كلما كبرت هذه الفجوة، زاد الإفراط في التخصيص.

تشبيه: خريطة المدينة

تخيل أنك تريد رسم خريطة لمدينة. إذا رسمت كل شجرة وكل حجر، فستحصل على خريطة "دقيقة" لما رأيته، لكنها عديمة الفائدة لشخص يريد التنقل. الخريطة الجيدة هي **تبسيط ذكي**: تحتفظ بالشوارع الرئيسية، وتتجاهل التفاصيل. النموذج الجيد يفعل الشيء نفسه: يلتقط الأنماط، ويتجاهل الضوضاء.

ثلاث حالات يجب أن تميزها

الحالة	خطأ التدريب	خطأ الاختبار	التشخيص	الحل
Underfitting (نقص التخصيص)	مرتفع	مرتفع	النموذج بسيط جدًا أو التدريب غير كافٍ	زد السعة، أطل التدريب، حسّن الميزات
Overfitting (إفراط التخصيص)	منخفض جدًا	مرتفع	النموذج يحفظ الضوضاء	تنظيم، بيانات أكثر، تبسيط
Good Fit (تخصيص جيد)	منخفض	منخفض	الهدف المنشود	استمر

🤔

سؤال يستحق التفكير: ماذا لو كانت كل من خسارة التدريب وخسارة الاختبار منخفضة، لكن النموذج يفشل في الإنتاج؟ هذه حالة رابعة خطيرة، تسمى **Distribution Shift**: بيانات الإنتاج تختلف عن بيانات التدريب. لا علاقة لها بالـ Overfitting، لكنها قاتلة. سنتناولها في فصل التقييم.

2.2 منحنى التدريب: قراءة "تخطيط القلب" للنموذج

لماذا المنحنى أهم من الرقم النهائي؟

لأن الرقم النهائي يخبرك **أين وصلت**، لكن المنحنى يخبرك **كيف وصلت**. وهناك فرق هائل بين من وصل بالقفز ومن وصل بالمشي. المنحنى يكشف المشاكل قبل أن تتفاقم.

الأشكال الأساسية لمنحنيات التدريب

1. المنحنى المثالي (تحقق ينخفض باستمرار، تدريب ينخفض، فجوة صغيرة)

TEXT

```
Loss
 |
 |\
 | \\
 |   \\\
 |      \\\\_____________  ← train
 |           \\\\\\\\\____  ← val
 └──────────────────────→ Epochs
```

2. إفراط في التخصيص (فجوة تتسع)

TEXT

```
Loss
 |
 |\
 | \\
 |   \\_______________  ← train (يستمر في الانخفاض)
 |     \\
 |       \\___
 |           \\_______  ← val (يبدأ في الارتفاع!)
 └──────────────────────→ Epochs
```

3. نقص في التخصيص (كلاهما مرتفع ومسطح)

TEXT

```
Loss
 │
 │  \
 │   \_____________   ← train
 │   \_____________   ← val (متقاربان، لكن مرتفعان)
 └──────────────────→ Epochs
```

4. معدل تعلم كبير جدًا (تذبذب عنيف)

TEXT

```
Loss
 │   /\   /\
 │  /  \ /  \
 │ /    V    \___
 │/
 └──────────────────→ Epochs
```

5. معدل تعلم صغير جدًا (انخفاض بطيء جدًا)

TEXT

```
Loss
 |\
 | \___
 |     \___
 |         \___   ← يبدو أنه لا يتقدم
 └──────────────→ Epochs
```

> **نصيحة**: في التدريب الحقيقي، ارسم **خسارة التدريب وخسارة التحقق على نفس المحور**. أي انحراف بينهما هو إشارة. أفضل أداة سريعة: `tensorboard` أو `wandb`. لا تعتمد على ذاكرتك.

كود: تسجيل منحنيات التدريب

PYTHON

```python
import matplotlib.pyplot as plt

history = {"train_loss": [], "val_loss": [], "val_acc": []}

for epoch in range(num_epochs):
    # ... حلقة التدريب ...
    train_loss = ...  # متوسط خسارة التدريب
    val_loss, val_acc = evaluate(model, val_loader)

    history["train_loss"].append(train_loss)
    history["val_loss"].append(val_loss)
    history["val_acc"].append(val_acc)

# رسم
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
ax1.plot(history["train_loss"], label="Train")
ax1.plot(history["val_loss"], label="Validation")
ax1.set_xlabel("Epoch"); ax1.set_ylabel("Loss")
ax1.legend(); ax1.set_title("Loss Curves")

ax2.plot(history["val_acc"])
ax2.set_xlabel("Epoch"); ax2.set_ylabel("Accuracy")
```

```
ax2.set_title("Validation Accuracy")

plt.tight_layout()
plt.savefig("training_curves.png", dpi=150)
```

⚠️ **تحذير**: لا ترسم خسارة كل **دفعة (batch)** فقط. هي صاخبة جدًا وقد تخفي الاتجاه الحقيقي. ارسم متوسط الخسارة لكل **حقبة (epoch)** أو استخدم المتوسط المتحرك.

2.3 التنظيم (Regularization): كيف نمنع الحفظ الأعمى؟

التنظيم هو أي تقنية تجعل النموذج "أقل حرية" في حفظ البيانات، دون أن تمنعه من التعلم. الفكرة الفلسفية: **نضيف عقوبة على التعقيد**، فيفضل النموذج الحلول البسيطة.

2.3.1 Dropout: التخريب المتعمد الذي ينجح

تخيل أنك تدرّب فريقًا. إذا كان أحد الأعضاء هو الذي يقوم بكل العمل دائمًا، فالباقي لن يتعلموا. في يوم ما، يغيب هذا العضو، فينهار الفريق. Dropout يفعل شيئًا مشابهًا: **في كل خطوة تدريب، يعطّل عشوائيًا نسبة من الخلايا**، فيضطر الباقي إلى تعلم تمثيلات مفيدة.

PYTHON

```
# في PyTorch
self.dropout = nn.Dropout(p=0.3)  # 30% من الخلايا تُعطّل عشوائيًا
# ...
x = self.dropout(x)
```

كيف يعمل بالضبط؟

- في **التدريب**: كل خلية تُضرب في قناع (mask) عشوائي من 0 و1، ثم يُقسم الناتج على 1-p للحفاظ على التوقع (Inverted Dropout).
- في **التقييم**: لا شيء يُعطّل، ويُستخدم النموذج كاملًا.

❌ **خطأ شائع**: نسيان استدعاء `model.eval()` قبل التقييم. إذا نسيت، فسيظل Dropout نشطًا، وستحصل على نتائج متقلبة وغير صحيحة. وبالمثل، `model.train()` قبل التدريب.

PYTHON

```
model.train()   # قبل التدريب
# ... training loop ...

model.eval()    # قبل التقييم
with torch.no_grad():
    pass  # ... evaluation ...
```

متى يكون Dropout فعالًا؟

- فعال جدًا في الشبكات الكبيرة (وخصوصًا في FFN في المحوّلات).

- أقل فعالية في الشبكات الصغيرة أو عند وجود تنظيم آخر قوي.
- في المحوّلات الحديثة، القيم الشائعة 0.1 للـ FFN و0.0 للانتباه (في النماذج الكبيرة، غالبًا لا حاجة له).

2.3.2 Weight Decay (L2 Regularization): العقوبة على الأوزان الكبيرة

الأوزان الكبيرة جدًا تعني أن النموذج "متأكد جدًا"، وهذا غالبًا علامة على الحفظ. نضيف عقوبة تتناسب مع مربع الأوزان:

$$L_{total} = L_(data) + \lambda \Sigma_i w_i^2$$

النتيجة: النموذج يفضل أوزانًا صغيرة، فيكون أكثر سلاسة وأقل حساسية للضوضاء.

```python
# في PyTorch: عبر المحسّن
optimizer = optim.AdamW(model.parameters(), lr=3e-4, weight_decay=0.01)
```

❌ **فرق مهم**: `Adam` مع `weight_decay` ليس نفس `AdamW`. في `Adam`، تسوس الوزن يُضاف إلى التدرج، فيتداخل مع تحديث Adam. في `AdamW`، يُطبق بشكل منفصل، وهو الأصح. النماذج الحديثة تستخدم `AdamW`.

كم قيمة `weight_decay`؟

- في المحوّلات: 0.1 شائع (مثلًا في LLaMA).
- في الشبكات الصغيرة: 1e-4 إلى 1e-2.
- جرّب قيمًا لوغاريتمية: 0.1, 1e-2, 1e-3, 1e-4.

2.3.3 Early Stopping: توقف قبل أن تفسد

بدلًا من تخمين عدد الحقب، راقب خسارة التحقق. إذا لم تتحسن لعدد معين من الحقب (الصبر/patience)، فتوقف. هذا أبسط أشكال التنظيم وأكثرها فعالية.

```python
class EarlyStopping:
    def __init__(self, patience=5, min_delta=0.0):
        self.patience = patience
        self.min_delta = min_delta
        self.best_loss = None
        self.counter = 0
        self.should_stop = False

    def __call__(self, val_loss):
        if self.best_loss is None or val_loss < self.best_loss - self.min_delta:
            self.best_loss = val_loss
            self.counter = 0
        else:
            self.counter += 1
            if self.counter >= self.patience:
                self.should_stop = True
```

نصيحة: احفظ أفضل نقطة (checkpoint) وليس الأخيرة. النموذج في الحقبة الأخيرة قد يكون قد بدأ في الإفراط في التخصيص.

2.3.4 Data Augmentation: أفضل تنظيم هو بيانات أكثر

لا شيء يعالج الإفراط في التخصيص مثل **بيانات حقيقية أكثر**. لكن إذا لم تستطع، فـ **توليد بيانات معدّلة** من الموجودة يساعد. في الصور: دوران، قص، تغيير سطوع. في النصوص: استبدال مرادفات، حذف عشوائي، ترجمة عكسية (back-translation).

تحذير: في النصوص، لا تبالغ في الـ augmentation. قد تغير المعنى أو تنتج جملًا غير نحوية، فيتعلم النموذج ضوضاء بدلًا من الإشارة. اختبر دائمًا على مجموعة تحقق حقيقية.

2.4 التطبيع (Normalization): لماذا LayerNorm وليس BatchNorm في المحوّلات؟

القصة: مشكلة "الانحراف الداخلي"

في الشبكات العميقة، توزيع مدخلات كل طبقة يتغير أثناء التدريب (لأن الأوزان تتغير). هذا يسمى **Internal Covariate Shift**. النتيجة: الطبقات اللاحقة تعاني من عدم استقرار، والتدريب يصبح صعبًا.

BatchNorm يحل ذلك بتطبيع مخرجات كل طبقة على مستوى **الدفعة (batch)**: لكل ميزة، نطرح متوسط الدفعة ونقسم على انحرافها المعياري.

```python
x_norm = (x - x.mean(dim=0)) / x.std(dim=0)
```

المشكلة: هذا يعتمد على **الدفعة**. في التقييم، نستخدم إحصاءات مجمعة (running mean/var) من التدريب. لكن في النماذج اللغوية:

1. حجم الدفعة قد يكون صغيرًا (أو 1 في الاستدلال)، فتكون الإحصاءات غير دقيقة.
2. التسلسلات مختلفة الطول، والحشو (padding) يفسد الإحصاءات.
3. في التوليد (token by token)، لا يوجد "دفعة" أصلًا.

LayerNorm يحل ذلك بتطبيع **كل عينة على حدة**، عبر الميزات:

```python
x_norm = (x - x.mean(dim=-1, keepdim=True)) / x.std(dim=-1, keepdim=True)
```

النتيجة: مستقل عن حجم الدفعة، ومناسب للتسلسلات، وهو الخيار السائد في المحوّلات.

مقارنة شاملة

	BatchNorm	LayerNorm
البُعد المطبع	الدفعة (لكل ميزة)	الميزات (لكل عينة)
يعتمد على حجم الدفعة	نعم	لا
مناسب للتسلسلات	لا	نعم
مناسب للاستدلال الفوري	نعم (بإحصاءات مجمعة)	نعم (بدون إحصاءات إضافية)
الاستخدام السائد	CNN	المحوّلات
بدائل حديثة	—	RMSNorm

RMSNorm: تبسيط LayerNorm

RMSNorm يحذف خطوة الطرح (المتوسط)، ويقسم فقط على الجذر التربيعي لمتوسط المربعات:

$$\text{RMSNorm}(x) = (x)/(\sqrt{(1/d\Sigma\, x_i^2 + \epsilon)}) \cdot \gamma$$

النتيجة: أسرع، وأداؤه مماثل أو أفضل في النماذج الكبيرة. تستخدمه LLaMA وQwen وDeepSeek.

PYTHON

```python
class RMSNorm(nn.Module):
    def __init__(self, dim, eps=1e-6):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(dim))
        self.eps = eps

    def forward(self, x):
        norm = x.float().pow(2).mean(-1, keepdim=True).add(self.eps).rsqrt()
        return (x.float() * norm).type_as(x) * self.weight
```

نصيحة: إذا كنت تبني محوّلًا من الصفر، ابدأ بـ LayerNorm. عندما تعمل، جرّب RMSNorm وقارن. الفرق عادة صغير، لكن في النماذج الكبيرة يصبح ذا معنى.

2.5 معدل التعلم: العامل الذي يدمر النماذج أكثر من أي شيء آخر

لماذا معدل التعلم خطير؟

لأنه **يبدو بسيطًا** (رقم واحد) لكنه يحدد سلوك التدريب كله. إذا كان كبيرًا، ينفجر. إذا كان صغيرًا، يتجمد. وأسوأ من ذلك: قد يبدو مقبولًا في البداية ثم يدمر النموذج لاحقًا.

تجربة فكرية

تخيل أنك تنزل جبلًا في الضباب. معدل التعلم هو **طول خطوتك**.

- خطوات كبيرة: قد تصل بسرعة، لكن قد تقفز فوق الوادي أو تتعثر.

- خطوات صغيرة: آمنة، لكنك قد تصل بعد أيام، أو تعلق في هضبة صغيرة.

جدولة معدل التعلم (LR Scheduling)

أفضل ممارسة: **ابدأ صغيرًا، ثم كبّر، ثم صغّر**.

1. **Warmup**: ابدأ بمعدل صغير جدًا، وارفعه تدريجيًا على مدى خطوات قليلة. هذا يمنع انفجار التدرجات في البداية.
2. **Cosine Decay**: بعد الوصول إلى الذروة، أنزل المعدل بسلاسة حتى النهاية.

PYTHON

```python
import math

def get_lr(step, warmup_steps, max_steps, max_lr, min_lr=1e-5):
    if step < warmup_steps:
        return max_lr * step / warmup_steps
    progress = (step - warmup_steps) / (max_steps - warmup_steps)
    cosine_decay = 0.5 * (1 + math.cos(math.pi * progress))
    return min_lr + (max_lr - min_lr) * cosine_decay
```

أو باستخدام PyTorch:

PYTHON

```python
scheduler = optim.lr_scheduler.OneCycleLR(
    optimizer,
    max_lr=3e-4,
    total_steps=total_steps,
    pct_start=0.03,       # 3% warmup
    anneal_strategy="cos",
)
```

> 🏆 **نصيحة ذهبية**: في المحوّلات، **Warmup ضروري**. بدونها، قد ينفجر التدريب في أول 100 خطوة. القيم الشائعة: 1%–5% من إجمالي الخطوات، أو 2000 خطوة في النماذج الكبيرة.

كيف تجد معدل تعلم مناسب دون تخمين؟

طريقة LR Range Test (من Leslie Smith): ابدأ بمعدل صغير جدًا، وارفعه تدريجيًا أثناء التدريب، وسجّل الخسارة. اختر المعدل الذي تكون عنده الخسارة في أسرع انخفاض.

PYTHON

```python
lrs = np.logspace(-6, 0, 100)
losses = []
for lr in lrs:
    model = rebuild_model()  # أعد التهيئة
    optimizer = optim.Adam(model.parameters(), lr=lr)
    # درّب لخطوة أو خطوتين
    losses.append(train_a_few_steps(model, optimizer))

# ابحث عن النقطة التي تبدأ فيها الخسارة في الارتفاع
# اختر معدلًا أقل بقليل من تلك النقطة
```

⚠️ **تحذير**: هذا الاختبار لا يعطيك المعدل الأمثل عالميًا، بل نقطة بداية معقولة. بعدها، جرّب قيمًا حولها (2×، 0.5×) وراقب.

2.6 الإفراط في التخصيص في العصر الحديث: مفارقة مزدوجة

هل ما زال Overfitting مشكلة في 2026؟

نعم، لكن بشكل مختلف. في الماضي، كانت النماذج صغيرة والبيانات قليلة، فكان الإفراط في التخصيص هو القاعدة. اليوم، النماذج ضخمة والبيانات هائلة، لكن ظهرت ظاهرتان جديدتان:

1. Double Descent (الانحدار المزدوج)

في النماذج الكبيرة جدًا، قد ترى سلوكًا غريبًا: الخسارة تنخفض، ثم ترتفع (منطقة الإفراط الكلاسيكي)، ثم **تنخفض مرة أخرى** عندما يتجاوز حجم النموذج عدد البيانات. النموذج "يحفظ" ثم "يعمم" بشكل أفضل. هذا يخالف الحكمة التقليدية، وأسبابه ما زالت موضوع بحث.

2. Grokking (الفهم المفاجئ)

في بعض المهام (خصوصًا الحسابية والرمزية)، يتدرب النموذج لآلاف الحقب، ويبدو أنه يحفظ فقط (خطأ تدريب منخفض جدًا، خطأ اختبار مرتفع). ثم فجأة، في لحظة ما، **ينخفض خطأ الاختبار بشكل حاد**. كأن النموذج "يفهم" فجأة. هذه الظاهرة تذكرنا بأن التدريب العميق لا يزال مليئًا بالأسرار.

🕰️ **درس عملي**: لا توقف التدريب مبكرًا جدًا فقط لأن خطأ التحقق ارتفع قليلًا. أحيانًا، الصبر يكشف عن تحسن مفاجئ. لكن أيضًا، لا تنتظر إلى الأبد. راقب وتعلّم.

2.7 خط الأساس (Baseline): لماذا تبدأ بسيطًا؟

القصة: مطرقة ضخمة لمسمار صغير

جاء مهندس شاب إلى فريقه وقال: "سأبني نموذجًا بـ Transformer لتصنيف رسائل البريد العشوائي". سأله القائد: "هل جرّبت Naive Bayes؟" قال: "لا، لكن Transformer أحدث". بعد أسبوع من العمل، حصل على دقة 97%. ثم جرّب زميله Naive Bayes في ساعة واحدة، وحصل على 96.5%. الفرق لم يكن يستحق أسبوعًا كاملًا.

خط الأساس يعني: أبسط حل معقول لمشكلتك. الغرض منه:

1. التأكد من أن البيانات قابلة للتعلم.
2. توفير مرجع للمقارنة.
3. كشف المشاكل في البيانات قبل استثمار الوقت في نموذج معقد.

خطوط أساس شائعة حسب المهمة

المهمة	خط الأساس
تصنيف نصوص	Naive Bayes، Logistic Regression، TF-IDF + SVM
تصنيف صور	Logistic Regression على ميزات HOG، أو k-NN
ترجمة	قاموس + قواعد بسيطة، أو حتى نسخ المدخلات
تلخيص	أخذ أول جملتين، أو الجملة الأطول
RAG	BM25 (بحث كلمات مفتاحية)

> **نصيحة**: إذا كان خط الأساس البسيط يحقق 90%، فقد تحتاج فقط إلى تحسين طفيف. وإذا لم يحقق سوى 50%، فالمشكلة في البيانات، وليس في النموذج. لا تبنِ قصرًا على مستنقع.

2.8 البيانات غير المتوازنة: عندما تكون الدقة كذبة

القصة: نموذج يكره النادرة

تخيل تشخيصًا طبيًا لمرض يصيب 1% من الناس. نموذج يقول دائمًا "سليم" سيحقق دقة 99%. رائع، أليس كذلك؟ لكنه **لا يشخص أي مريض**. هذه هي مشكلة البيانات غير المتوازنة.

المقاييس الصحيحة

المقياس	الصيغة	متى تستخدمه
Accuracy	(TP+TN)/(الكل)	بيانات متوازنة فقط
Precision	TP/(TP+FP)	عندما تكون الإيجابية الكاذبة مكلفة (بريد عشوائي)
Recall	TP/(TP+FN)	عندما تكون السلبية الكاذبة مكلفة (تشخيص مرض)
F1	2·P·R/(P+R)	توازن بين الدقة والاستدعاء
AUC-ROC	المساحة تحت المنحنى	تقييم عام للتصنيف الثنائي
PR-AUC	المساحة تحت منحنى Precision-Recall	أفضل للبيانات غير المتوازنة جدًا

تقنيات المعالجة

1. **Class Weights**: اجعل الخسارة أكبر للفئة النادرة.

```python
weights = torch.tensor([1.0, 10.0])  # الفئة النادرة أثقل
criterion = nn.CrossEntropyLoss(weight=weights)
```

1. **Oversampling**: كرر عينات الفئة النادرة (أو استخدم SMOTE).
2. **Undersampling**: احذف بعض عينات الفئة الشائعة (قد تفقد معلومات).

3. **Focal Loss**: تركز على العينات الصعبة.

```python
def focal_loss(logits, targets, alpha=0.25, gamma=2.0):
    p = torch.sigmoid(logits)
    ce = F.binary_cross_entropy_with_logits(logits, targets, reduction="none")
    p_t = p * targets + (1 - p) * (1 - targets)
    return (alpha * (1 - p_t) ** gamma * ce).mean()
```

⚠️ **تحذير**: لا تطبق oversampling قبل تقسيم البيانات! قد تتسرب عينات مكررة إلى مجموعة الاختبار، فتحصل على نتائج مضللة. قسّم أولًا، ثم عالج التدريب فقط.

2.9 التحقق المتقاطع (Cross-Validation): عندما تكون البيانات قليلة

لماذا؟

تقسيم واحد 80/20 قد يكون محظوظًا أو مؤسفًا. التحقق المتقاطع (K-Fold) يقسم البيانات إلى K أجزاء، ويدرب K مرات، في كل مرة يستخدم جزءًا للتحقق والباقي للتدريب. النتيجة: تقدير أكثر استقرارًا للأداء.

```python
from sklearn.model_selection import KFold
import numpy as np

kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = []

for fold, (train_idx, val_idx) in enumerate(kf.split(X)):
    model = build_model()
    train_loader = make_loader(X[train_idx], y[train_idx])
    val_loader = make_loader(X[val_idx], y[val_idx])

    train(model, train_loader)
    score = evaluate(model, val_loader)
    scores.append(score)
    print(f"Fold {fold+1}: {score:.4f}")

print(f"Mean: {np.mean(scores):.4f} ± {np.std(scores):.4f}")
```

💡 **نصيحة**: في التعلم العميق مع بيانات كبيرة، K-Fold مكلف. استخدم تقسيمًا واحدًا مع تحقق، وربما K=3 فقط. في البيانات الصغيرة أو الطبية، K=5 أو 10 يستحق التكلفة.

⚠️ **تحذير**: في البيانات الزمنية (Time Series)، لا تستخدم K-Fold العشوائي! استخدم **TimeSeriesSplit**، حيث التدريب دائمًا على الماضي والتحقق على المستقبل. وإلا، فأنت "تتسرب" من المستقبل إلى الماضي، والنتائج مضللة.

2.10 الأخطاء الشائعة: القائمة السوداء

1. تقييم النموذج على بيانات التدريب

"نموذجي يحقق 99.9%!" — على بيانات درّبته عليها. هذا ليس إنجازًا، هذا حفظ. **دائمًا** قيّم على بيانات لم يرها النموذج.

2. تسريب البيانات (Data Leakage)

أخطر خطأ، وأصعب اكتشافًا. أمثلة:

- تطبيع البيانات قبل التقسيم (يستخدم إحصاءات الاختبار).
- إعادة عينات قبل التقسيم (تسرب العينات المكررة).
- ميزات تحتوي على معلومات من المستقبل (مثلًا: "عدد الزيارات بعد الشراء" للتنبؤ بالشراء).
- في RAG: استخدام الإجابة نفسها في السياق.

الحل: ارسم خط أنابيب البيانات بالكامل، واسأل: "هل هذه المعلومة متاحة **وقت التنبؤ**؟" إذا لا، فهي تسريب.

3. الاعتماد على مقياس واحد

الدقة وحدها لا تكفي. راقب Recall، Precision، F1، مصفوفة الالتباس. في بعض الحالات، الخطأ "الثمين" أهم من الخطأ "الرخيص".

4. عدم ضبط البذور وإعادة التجارب

إذا لم تستطع إعادة تجربة، فأنت لا تعرف ما الذي نجح. اضبط البذور، سجّل الإصدارات، احفظ الإعدادات.

5. مقارنة نماذج بإعدادات مختلفة

"النموذج A أفضل من B" — لكن A استخدم بيانات أكثر، أو تدريبًا أطول، أو معدل تعلم أفضل. المقارنة العادلة تعني **تثبيت كل شيء آخر**. هذا يسمى **Ablation Study**.

6. تجاهل التكلفة

نموذج بدقة 95% يستغرق 10 ثوانٍ للاستدلال قد يكون أسوأ من نموذج بدقة 93% يستغرق 50 مللي ثانية. في الإنتاج، **الدقة ليست كل شيء**. زمن الاستجابة، الذاكرة، التكلفة، كلها عوامل.

7. عدم اختبار الحالات الحدية

بياناتك فيها حالات شاذة (نصوص فارغة، صور مشوهة، مدخلات طويلة جدًا). اختبر عليها. النموذج الذي يعمل على "المتوسط" فقط قد يفشل كارثيًا في الإنتاج.

2.11 أفضل الممارسات: قائمة المحترف

1. **قسّم البيانات أولًا**، قبل أي معالجة. واحفظ مجموعة الاختبار "مقدسة" حتى النهاية.
2. **ابنِ خط أساس بسيطًا** قبل أي نموذج معقد.
3. **راقب منحنيات التدريب والتحقق** في كل تجربة، لا فقط الرقم النهائي.

4. **استخدم Early Stopping** مع حفظ أفضل نقطة.
5. **اضبط معدل التعلم**، ولا تعتمد على قيمة عشوائية. جرّب Range Test.
6. **استخدم Warmup + Cosine Decay** في المحوّلات.
7. **تحقق من التدرجات** قبل التدريب الطويل (كما في الفصل الأول).
8. **راقب** `grad_norm`: انفجاره المفاجئ يعني مشكلة.
9. **اضبط البذور** وسجّل كل تجربة (اسم، تاريخ، إعدادات، نتائج).
10. **اكتب اختبارات**: Overfit a single batch، شكل المخرجات، عدم وجود NaN، إعادة إنتاجية.
11. **لا تقارن نماذج بإعدادات مختلفة** دون Ablation Study.
12. **فكر في التكلفة**، وليس فقط الدقة.
13. **اختبر على الحالات الحدية**، وليس فقط على المتوسط.
14. **وثّق افتراضاتك**: ماذا تعني الفئة؟ كيف جُمعت البيانات؟ ما القيود؟

2.12 تمرين شامل: مختبر التشخيص

السيناريو

أعطيتك مجموعة بيانات تصنيف ثنائي (يمكنك توليدها بـ `sklearn.datasets.make_classification` مع `n_samples=1000` , `n_features=20` , `n_informative=5` , `flip_y=0.1`). ابنِ نموذجًا، ثم شخّص حالته.

المطلوب

1. **قسّم** البيانات إلى تدريب/تحقق/اختبار (70/15/15). طبّع باستخدام إحصاءات **التدريب فقط**.
2. **ابنِ خط أساس**: Logistic Regression. سجّل F1 على الاختبار.
3. **ابنِ MLP** بطبقتين مخفيتين (64، 32) مع ReLU. درّب 100 حقبة. ارسم منحنيات التدريب والتحقق.
4. **شخّص**: هل هناك Overfitting؟ Underfitting؟ كيف عرفت؟
5. **عالج**: جرّب (أ) Dropout=0.3، (ب) Weight Decay=1e-3، (ج) Early Stopping. أيها حسّن أداء التحقق أكثر؟
6. **قارن**: هل تفوق MLP على Logistic Regression؟ بكم؟ هل الفرق يستحق التعقيد؟
7. **اختبر الحالات الحدية**: ما أداء النموذج على عينات ذات قيم شاذة (مثلًا، ضرب بعض الميزات في 100)؟ هل ينهار؟
8. **اكتب تقريرًا قصيرًا** (صفحة واحدة): ما المشكلة؟ ما الحل الذي اخترته؟ لماذا؟ ما القيود؟

تمرين إضافي (متقدم)

طبّق **Focal Loss** بدلًا من Cross-Entropy، وقارن F1 على الفئة النادرة (اجعل 10% فقط من العينات من الفئة 1). ثم جرّب **Class Weights**. أيهما أفضل؟ لماذا؟

تمرين بحثي (للمهتمين)

ابحث عن ظاهرة **Grokking**: درّب نموذجًا صغيرًا على مهمة حسابية بسيطة (مثل جمع أعداد صحيحة modulo 97). راقب خطأ التدريب والاختبار لآلاف الحقب. هل ترى لحظة "الفهم المفاجئ"؟ صف ما حدث.

خلاصة الفصل: ما يجب أن يبقى في ذهنك

1. **التعميم هو الهدف**، وليس تقليل خطأ التدريب. الفجوة بين الاثنين هي مؤشرك.
2. **المنحنى أهم من الرقم**. شكل المنحنى يخبرك بالمشكلة قبل أن تتفاقم.
3. **التنظيم ليس خيارًا**. Early Stopping، Weight Decay، Dropout، والمزيد من البيانات، كلها أدوات.
4. **LayerNorm وRMSNorm** هما الخيار في المحوّلات، وليس BatchNorm. والسبب: الاستقلال عن الدفعة.
5. **معدل التعلم هو العامل الأكثر خطورة**. ابدأ بـ Warmup، واستخدم Cosine Decay، وجرّب Range Test.
6. **ابدأ بخط أساس بسيط**. لا تبنِ Transformer لتصنيف بريد عشوائي قبل تجربة Naive Bayes.
7. **البيانات غير المتوازنة تحتاج مقاييس مختلفة**. الدقة كذبة في هذه الحالة.
8. **تسريب البيانات هو العدو الخفي**. راجع خط أنابيبك بالكامل.
9. **قارن بعدل**: تثبيت الإعدادات، Ablation Study، تسجيل كل شيء.
10. **العصر الحديث يحمل مفاجآت**: Double Descent وGrokking يذكراننا بأن التعلم العميق لا يزال مليئًا بالأسرار.

افتتاحية الفصل القادم: "بنية التدريب الحديثة"

في الفصل التالي، سنغادر عالم المفاهيم النظرية، وندخل ورشة العمل الحقيقية. كيف ندرّب نموذجًا على عدة GPUs دون أن نعيد كتابة الكود؟ ما هو **Mixed Precision**، ولماذا يمكن أن يضاعف السرعة دون خسارة دقة؟ كيف يعمل **Gradient Accumulation**، ومتى يكون ضروريًا؟ ما الفرق بين **DDP** و**FSDP**، وأيهما تختار؟ وكيف نتجنب أخطاء `NaN` التي تدمر التدريب في منتصف الليل؟

> ❓ **سؤال يفتح الشهية**: هل تعلم أن استخدام `bf16` بدلًا من `fp16` قد يغير استقرار تدريبك بالكامل، وليس فقط السرعة؟ ولماذا يفضل المحترفون `bf16` في النماذج الكبيرة رغم أنه "أقل دقة"؟ سنكتشف ذلك في الفصل القادم.

المخطط التوضيحي للفصل



شكل 2.1 — منحنيات التدريب: خمسة أنماط تُقرأ قبل الأرقام



شكل 2.2 — التنظيم: أدوات منع الحفظ الأعمى

03

بنية التدريب الحديثة — من GPU واحدة إلى عنقود

افتتاحية: حكاية المهندس الذي درّب نموذجًا في ليلة واحدة

في إحدى الشركات الناشئة، كان هناك مهندس اسمه "طارق". طُلب منه تدريب نموذج تصنيف نصوص على مجموعة بيانات كبيرة. نظر طارق إلى حاسوبه المحمول، ثم إلى GPU واحدة في الخادم، وقال: "سأدرّب النموذج الليلة، وأعرض النتائج غدًا صباحًا".

بدأ التدريب. بعد ساعتين، توقف الجهاز. نفدت الذاكرة (OOM). فكّر طارق: "سأقلل حجم الدفعة (batch size)". قلّلها. بدأ التدريب من جديد. بعد ثلاث ساعات، ظهرت رسالة غامضة: `loss: nan`. توقف. فكّر: "سأقلل معدل التعلم". قلّله. بدأ من جديد. بعد خمس ساعات، اكتمل التدريب، لكن الخسارة كانت أعلى بكثير من المتوقع. لم يكن يعرف أن **معدل التعلم الفعلي** قد تغيّر بسبب تقليل حجم الدفعة، وأن **الدقة المختلطة** كانت تسبب مشاكل عددية، وأنه كان يستخدم **fp16** بدلًا من **bf16**، وأن **التحقق من التدرجات** كان يمكن أن يكشف المشكلة في دقيقتين.

طارق لم يكن سيئًا. كان فقط يجهل أن التدريب الحديث ليس مجرد `loss.backward()` و `optimizer.step()`. هناك بنية تحتية كاملة، وتفاصيل دقيقة، وأخطاء قاتلة تبدو صغيرة.

هذا الفصل هو دليل النجاة الذي كان طارق يحتاجه.

3.1 لماذا التدريب الحديث معقد؟

ثلاث حقائق مؤلمة

1. **النماذج أكبر من الذاكرة**. نموذج بـ 7 مليارات معامل يحتاج إلى ~28 غيغابايت في FP32 فقط لتخزين الأوزان، دون احتساب التدرجات وحالات المحسّن والتنشيطات. وحدة GPU واحدة قد لا تكفي.
2. **التدريب يستهلك وقتًا هائلًا**. نموذج صغير قد يستغرق أيامًا. النماذج الكبيرة تستغرق أسابيع أو أشهرًا على عناقيد كاملة.
3. **الأخطاء العددية صامتة**. `NaN` لا يخبرك من أين جاء. قد يكون من تدرج منفجر، أو قسمة على صفر، أو دقة غير كافية. وأنت تكتشف ذلك بعد ساعات من التدريب.

الأعمدة الثلاثة للتدريب الحديث

لحسن الحظ، المجتمع طوّر ثلاث تقنيات أساسية تحل هذه المشاكل:

التقنية	الهدف	الأداة
Mixed Precision	تقليل الذاكرة وزيادة السرعة	`torch.amp`
Gradient Accumulation	محاكاة دفعات كبيرة بذاكرة صغيرة	حلقة يدوية

التقنية	الهدف	الأداة
Distributed Training	توزيع العمل على عدة GPUs	`DDP` / `FSDP`

سنتناول كلًّا منها بالتفصيل، مع الأكواد، والأخطاء الشائعة، والمقارنات.

3.2 الدقة المختلطة (Mixed Precision): عندما تصبح الأرقام أرخص

الفكرة في سطر واحد

بدلًا من تخزين كل الأرقام بدقة 32 بت (FP32)، استخدم دقة 16 بت (FP16 أو BF16) في معظم العمليات، واحتفظ بنسخة FP32 فقط حيث تحتاجها (مثل تحديث الأوزان). النتيجة: **ذاكرة أقل، سرعة أعلى، ودقة مقبولة**.

لماذا **16** بت وليس **8** بت؟

- 32 :**FP32** بت، دقة عالية، لكن ذاكرة كبيرة وبطيئة.
- 16 :**FP16** بت، 5 بتات للأس (exponent) و10 بتات للعشري (mantissa). المدى محدود جدًا (أقصى قيمة ~65504)، لكن الدقة جيدة.
- 16 :**BF16** بت، 8 بتات للأس و7 بتات للعشري. المدى **نفس FP32** تقريبًا، لكن الدقة أقل. هذا يجعله أكثر استقرارًا عدديًا.
- 8 :**FP8** بت، نمطان (E4M3 وE5M2). تستخدمه النماذج الحديثة جدًا (مثل DeepSeek-V3) لتسريع التدريب، لكنه يحتاج إلى معايرة دقيقة.

FP16 أم **BF16**؟ هذه ليست مسألة سرعة فقط

هذا سؤال يستحق التوقف. الاختلاف بينهما ليس مجرد "دقة أعلى" أو "أقل". إنه **فلسفة مختلفة في التعامل مع الأرقام**.

	FP16	BF16
بتات الأس	5	8
بتات العشري	10	7
المدى الأقصى	~65,504	~3.4e38 (مثل FP32)
دقة الأرقام الصغيرة	أفضل	أقل
خطر التجاوز (Overflow)	مرتفع جدًا	منخفض جدًا
خطر التلاشي (Underflow)	متوسط	أعلى قليلًا
الاستقرار في التدريب	يحتاج Loss Scaling	مستقر افتراضيًا
الاستخدام السائد	بعض النماذج القديمة	**الخيار الافتراضي في النماذج الحديثة**

لماذا BF16 أكثر استقرارًا؟ لأن مداه الواسع يعني أن التدرجات الكبيرة جدًا أو الصغيرة جدًا لن تتجاوز الحد أو تتلاشى بسهولة. في FP16، إذا تجاوز التدرج 65504 (وهذا يحدث كثيرًا في المحوّلات)، فسيصبح `inf`، ثم `NaN`، وينهار التدريب. في BF16، نفس التدرج يبقى ممثلًا.

لكن، ألا تدفع ثمنًا في الدقة؟ نعم، BF16 لديه 7 بتات فقط للعشري (مقابل 10 في FP16). هذا يعني أن الأرقام الصغيرة جدًا تُقرّب بشكل خشن. لكن في التدريب العميق، **المدى أهم من الدقة الدقيقة**. معظم التدرجات في النماذج الكبيرة صغيرة جدًا، لكنها لا تحتاج إلى دقة عالية للتقارب. والأهم: BF16 لا يحتاج إلى Loss Scaling، مما يبسط الكود ويزيل مصدرًا كاملًا من الأخطاء.

> **نصيحة 2026**: إذا كنت تستخدم PyTorch 2.x وGPU من عائلة Ampere أو أحدث (A100، H100، RTX 30/40)، فاستخدم `torch.bfloat16` افتراضيًا. لا تفكر مرتين. إلا إذا كنت تعمل على أجهزة أقدم (V100، T4) لا تدعم BF16، فاستخدم FP16 مع Loss Scaling.

كود: الدقة المختلطة في PyTorch

```python
import torch
import torch.nn as nn
from torch.amp import autocast, GradScaler

device = "cuda"
model = MyModel().to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4)

# BF16: لا يحتاج GradScaler
use_bf16 = torch.cuda.is_bf16_supported()

# FP16: يحتاج GradScaler لمنع التلاشي
scaler = GradScaler(enabled=not use_bf16)

for batch in dataloader:
    optimizer.zero_grad(set_to_none=True)

    with autocast(device_type="cuda", dtype=torch.bfloat16 if use_bf16 else torch.float16):
        logits = model(batch["input_ids"])
        loss = criterion(logits, batch["labels"])

    if use_bf16:
        loss.backward()
        optimizer.step()
    else:
        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)  # للـ clipping
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        scaler.step(optimizer)
        scaler.update()
```

لماذا Loss Scaling في FP16؟

التدرجات في FP16 غالبًا صغيرة جدًا، وقد تتلاشى إلى صفر. Loss Scaling يضرب الخسارة في رقم كبير (مثل 65536) قبل `backward()`، فتتكبر التدرجات، ثم يقسمها `scaler` قبل تحديث الأوزان. النتيجة: تدرجات ممثلة بدقة، دون تجاوز.

> **خطأ شائع**: استخدام `autocast` دون `GradScaler` مع FP16. النتيجة: تدرجات صفرية، والنموذج لا يتعلم. إذا رأيت خسارة ثابتة لا تتحرك، فهذا أول ما تفحصه.

⚠️ **تحذير**: بعض العمليات لا يجب أن تُنفذ في دقة منخفضة، مثل `softmax` أو حساب الخسارة. `autocast` يتعامل معها تلقائيًا في معظم الحالات، لكن إذا كتبت عملية مخصصة، فتأكد من أنها تعمل في FP32 إذا لزم الأمر.

3.3 تراكم التدرجات (Gradient Accumulation): دفعات كبيرة بذاكرة صغيرة

المشكلة

تريد حجم دفعة (batch size) كبيرًا لاستقرار التدريب، لكن ذاكرة GPU لا تسمح. ماذا تفعل؟

الحل

قسّم الدفعة الكبيرة إلى دفعات صغيرة، واحسب التدرجات لكل دفعة صغيرة، ثم **اجمعها** قبل تحديث الأوزان. النتيجة: نفس تأثير الدفعة الكبيرة، بذاكرة دفعة صغيرة.

PYTHON

```python
accumulation_steps = 4
optimizer.zero_grad(set_to_none=True)

for step, batch in enumerate(dataloader):
    with autocast(device_type="cuda", dtype=torch.bfloat16):
        logits = model(batch["input_ids"])
        loss = criterion(logits, batch["labels"])
        loss = loss / accumulation_steps  # مهم جدًا!

    loss.backward()

    if (step + 1) % accumulation_steps == 0:
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        optimizer.zero_grad(set_to_none=True)
```

🛑 **خطأ شائع قاتل**: نسيان قسمة الخسارة على `accumulation_steps`. النتيجة: التدرجات تتضاعف بعدد الخطوات، ومعدل التعلم الفعلي يصبح أكبر بعدة مرات، وقد ينفجر التدريب. **دائمًا** اقسم.

التغييرات الخفية التي يسببها التراكم

هذه نقطة يغفل عنها الكثيرون. التراكم **لا يعادل** تمامًا دفعة كبيرة في كل شيء. هناك تفاصيل دقيقة:

1. **LayerNorm وBatchNorm**: إذا كانت تعتمد على إحصاءات الدفعة، فكل دفعة صغيرة تحسب إحصاءاتها الخاصة، وليس الدفعة الكبيرة. النتيجة: سلوك مختلف. في المحوّلات، LayerNorm لا تعتمد على الدفعة، فالمشكلة أقل. لكن في CNNs مع BatchNorm، قد تحتاج إلى `SyncBatchNorm` عبر الأجهزة.
2. **Dropout**: يولّد أقنعة عشوائية لكل دفعة صغيرة. مع التراكم، تحصل على تنوع أكبر في الأقنعة، وهذا قد يكون مفيدًا (تأثير تنظيم إضافي).
3. **معدل التعلم**: إذا كنت تستخدم جدولة تعتمد على "الخطوات"، فتذكر أن "الخطوة" الآن هي دورة تراكم كاملة، وليس كل دفعة صغيرة. اضبط `total_steps` بناءً على عدد دورات التراكم، وليس عدد الدفعات.

نصيحة: إذا كنت تستخدم `OneCycleLR` أو `CosineAnnealingLR`، فاحسب `total_steps` على أساس `len(dataloader) // accumulation_steps`. وإلا، ستنتهي الجدولة قبل نهاية التدريب، أو لن تصل إلى معدل التعلم الأدنى.

مقارنة: دفعة كبيرة حقيقية أم تراكم؟

	دفعة كبيرة حقيقية	تراكم التدرجات
الذاكرة	عالية	منخفضة
السرعة	أسرع (توازٍ أفضل)	أبطأ قليلًا (تسلسل)
دقة التدرج	مطابقة تمامًا	تقريبية (بسبب LayerNorm/Dropout)
الاستقرار	جيد	جيد، لكن قد يختلف قليلًا
متى تختاره	إذا كانت الذاكرة تسمح	إذا كانت الذاكرة محدودة

3.4 التدريب الموزع: DDP أم FSDP؟

المشكلة: نموذج لا يتسع في GPU واحدة

نموذج بـ 7B معامل في BF16 يحتاج إلى ~14 غيغابايت للأوزان فقط. أضف التدرجات (14 غيغابايت)، وحالات Adam (28 غيغابايت)، والتنشيطات (متغيرة). المجموع قد يتجاوز 80 غيغابايت. وحدة A100 بـ 80 غيغابايت قد تكون على الحد. وحدة RTX 4090 بـ 24 غيغابايت؟ مستحيل.

الحل: توزيع العمل

هناك استراتيجيتان رئيسيتان:

1. DDP (Distributed Data Parallel)

الفكرة: كل GPU تحتفظ بنسخة **كاملة** من النموذج. كل GPU تعالج دفعة مختلفة من البيانات. بعد حساب التدرجات، تتزامن جميع GPUs عبر `all-reduce`، فتحصل كلها على نفس التدرج المتوسط. ثم تحدّث كل GPU أوزانها (التي تبقى متطابقة).

الميزة: بسيط، فعال، ومدعوم جيدًا. إذا كان النموذج يتسع في GPU واحدة، فهذا هو الخيار.

العيب: لا يحل مشكلة "النموذج أكبر من GPU واحدة". كل GPU تحتاج إلى ذاكرة كافية للنموذج كاملًا.

```python
# تشغيل DDP (مبسط)
# كل عملية (process) تعمل على GPU مختلفة
torch.distributed.init_process_group(backend="nccl")
model = nn.parallel.DistributedDataParallel(model, device_ids=[local_rank])
```

2. FSDP (Fully Sharded Data Parallel)

الفكرة: بدلًا من نسخة كاملة على كل GPU، **قسّم** معاملات النموذج والتدرجات وحالات المحسّن عبر GPUs. كل GPU تحتفظ بجزء فقط. عند الحاجة إلى طبقة معينة، يتم تجميع أجزائها مؤقتًا (all-gather)، ثم تحريرها بعد الاستخدام.

الميزة: يمكنك تدريب نموذج أكبر بكثير من ذاكرة GPU واحدة. مثالي للنماذج الكبيرة (7B، 13B، 70B).

العيب: اتصال شبكي أكثر كثافة، وتعقيد أعلى. قد يكون أبطأ من DDP إذا كان النموذج صغيرًا.

```python
from torch.distributed.fsdp import FullyShardedDataParallel as FSDP

model = FSDP(model, use_orig_params=True)
```

مقارنة شاملة

	DDP	FSDP
نسخة النموذج	كاملة على كل GPU	مقسمة
الذاكرة المطلوبة لكل GPU	حجم النموذج كاملًا	جزء من النموذج
السرعة (نموذج صغير)	أسرع	أبطأ قليلًا
السرعة (نموذج كبير)	قد لا يعمل (OOM)	يعمل
التعقيد	منخفض	متوسط إلى مرتفع
متى تختاره	النموذج يتسع في GPU واحدة	النموذج أكبر من GPU واحدة

نصيحة: إذا كنت تستخدم `accelerate` أو `deepspeed` أو `torchrun`، فغالبًا سيتولى الأمر نيابة عنك. لا تحتاج إلى كتابة كود DDP/FSDP يدويًا في معظم الحالات. لكن **افهم** ما يحدث تحت الغطاء، لأن الأخطاء ستظهر، وعندها تحتاج إلى معرفة أين تبحث.

أخطاء شائعة في التدريب الموزع

1. **عدم ضبط البذور لكل عملية**: كل GPU يجب أن تحصل على بذرة مختلفة، وإلا ستعالج نفس البيانات (أو نفس التهيئة) وتفسد التدريب.
2. **عدم استخدام `DistributedSampler`**: إذا كان كل GPU يقرأ من `DataLoader` عادي، فستحصل على تداخل في البيانات، وقد ترى تكرارًا غير مقصود.
3. **نسيان `torch.distributed.barrier()` قبل التقييم**: قد تبدأ بعض العمليات التقييم قبل أن تنتهي أخرى، فتحصل على نتائج غير متسقة.
4. **عدم حفظ النموذج بشكل صحيح**: في FSDP، الأوزان مقسمة. تحتاج إلى `state_dict` كامل قبل الحفظ، أو حفظ الأجزاء بشكل منفصل.

3.5 الاستقرار العددي: العدو الصامت

لماذا `NaN` يظهر فجأة؟

`NaN` (Not a Number) هو نتيجة عملية غير معرّفة رياضيًا: قسمة على صفر، `log(0)`، `sqrt(-1)`، أو `inf - inf`. في التدريب العميق، يظهر عادةً بسبب:

1. **انفجار التدرجات**: تدرج كبير جدًا، يصبح `inf`، ثم `NaN`.
2. **تحت/فوق التدفق في FP16**: تدرج صغير جدًا يصبح صفرًا، أو كبير جدًا يصبح `inf`.
3. **خسارة غير مستقرة**: `log(0)` في Cross-Entropy، أو `softmax` على قيم كبيرة جدًا.
4. **معدل تعلم مرتفع جدًا**: يقفز بالنموذج إلى منطقة غير مستقرة.
5. **بيانات تالفة**: قيم `NaN` أو `inf` في المدخلات نفسها.

كيف تكتشف المشكلة مبكرًا؟

1. راقب `grad_norm`

هذا أهم مقياس بعد الخسارة. `grad_norm` هو معيار L2 لجميع التدرجات. إذا كان ينمو بشكل مطرد، أو يقفز فجأة، فهناك مشكلة.

```python
total_norm = 0.0
for p in model.parameters():
    if p.grad is not None:
        total_norm += p.grad.detach().data.norm(2).item() ** 2
total_norm = total_norm ** 0.5
print(f"grad_norm: {total_norm:.4f}")
```

> **قاعدة عملية**: إذا تجاوز `grad_norm` قيمة 1000 فجأة، فتوقف وأصلح المشكلة قبل المتابعة. `clip_grad_norm_` يمكن أن يساعد، لكنه يعالج الأعراض لا السبب.

2. استخدم `torch.autograd.set_detect_anomaly(True)` أثناء التصحيح

هذا الخيار يجعل PyTorch يتحقق من كل عملية، ويخبرك أين حدث `NaN` بالضبط. لكنه **بطيء جدًا**، فلا تستخدمه في التدريب العادي.

```python
with torch.autograd.set_detect_anomaly(True):
    loss.backward()
```

3. تحقق من المدخلات

قبل تمرير أي دفعة إلى النموذج، تحقق من عدم وجود `NaN` أو `inf`:

```python
assert not torch.isnan(batch["input_ids"]).any(), "NaN in input!"
assert not torch.isinf(batch["input_ids"]).any(), "Inf in input!"
```

4. استخدم `clip_grad_norm_` دائمًا

هذا ليس اختياريًا في المحوّلات. قصّ التدرجات إلى معيار أقصى (عادة 1.0) يمنع الانفجار.

```python
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

> ⚠ **تحذير:** `clip_grad_norm_` يجب أن يُستدعى **قبل** `optimizer.step()`، و**بعد** `backward()`. إذا استدعيته بعد `step()`، فلا فائدة له.

5. جرّب `bf16` بدلًا من `fp16`

إذا كنت تستخدم FP16 وتواجه `NaN` متكررًا، فجرّب BF16. في معظم الحالات، سيختفي `NaN` لأن المدى أوسع. هذا واحد من أكبر أسباب تفضيل BF16 في النماذج الحديثة.

3.6 حفظ النماذج الكبيرة: Checkpointing

لماذا الحفظ صعب؟

في DDP، كل GPU لديها نسخة كاملة. الحفظ بسيط، لكنه يستهلك مساحة كبيرة (نسخة واحدة تكفي). في FSDP، الأوزان مقسمة. تحتاج إلى تجميعها قبل الحفظ، أو حفظ الأجزاء. وإذا انهار التدريب في منتصف الحفظ، فقد تفقد كل شيء.

أفضل الممارسات

1. **احفظ النموذج والمحسّن و `GradScaler` معًا**. إذا استأنفت التدريب دون حالة المحسّن (مثل زخم Adam)، فستبدأ من حالة مختلفة، وقد تتعطل.
2. **احفظ `scheduler` أيضًا**.
3. **احفظ رقم الخطوة/الحقبة**.
4. **احفظ `RNG state`** لضمان إعادة إنتاجية Dropout والتهيئة.
5. **استخدم حفظًا غير متزامن (async)** إن أمكن، لتجنب توقف التدريب.

```python
def save_checkpoint(path, model, optimizer, scheduler, scaler, step, epoch):
    checkpoint = {
        "model": model.state_dict(),
        "optimizer": optimizer.state_dict(),
        "scheduler": scheduler.state_dict() if scheduler else None,
        "scaler": scaler.state_dict() if scaler else None,
        "step": step,
        "epoch": epoch,
        "rng_state": torch.get_rng_state(),
        "cuda_rng_state": torch.cuda.get_rng_state_all(),
    }
    torch.save(checkpoint, path)

def load_checkpoint(path, model, optimizer, scheduler, scaler):
    checkpoint = torch.load(path, map_location="cpu")
    model.load_state_dict(checkpoint["model"])
    optimizer.load_state_dict(checkpoint["optimizer"])
```

```
    if scheduler and checkpoint["scheduler"]:
        scheduler.load_state_dict(checkpoint["scheduler"])
    if scaler and checkpoint["scaler"]:
        scaler.load_state_dict(checkpoint["scaler"])
    torch.set_rng_state(checkpoint["rng_state"])
    torch.cuda.set_rng_state_all(checkpoint["cuda_rng_state"])
    return checkpoint["step"], checkpoint["epoch"]
```

نصيحة: احفظ checkpoint **دوريًا** (كل N خطوة)، وليس فقط في النهاية. واحفظ أفضل نموذج بناءً على خسارة التحقق. التخزين رخيص؛ إعادة التدريب من الصفر ليست كذلك.

3.7 الأخطاء الشائعة: القائمة السوداء للتدريب الحديث

1. استخدام FP16 دون Loss Scaling

الأعراض: خسارة ثابتة، `grad_norm` صفري، النموذج لا يتعلم.

الحل: استخدم `GradScaler`، أو انتقل إلى BF16.

2. نسيان قسمة الخسارة في Gradient Accumulation

الأعراض: تدرجات ضخمة، `NaN`، تدريب ينفجر بعد عدة دورات تراكم.

الحل: `loss = loss / accumulation_steps` قبل `backward()`.

3. نسيان `optimizer.zero_grad()`

الأعراض: تدرجات تتراكم عبر الدفعات، سلوك غير متوقع، خسارة تتذبذب بعنف.

الحل: استدع `zero_grad(set_to_none=True)` في بداية كل دورة تراكم.

4. جدولة معدل التعلم على "خطوات" خاطئة

الأعراض: معدل التعلم يصل إلى الحد الأدنى قبل نهاية التدريب، أو لا يصل أبدًا.

الحل: احسب `total_steps` بناءً على عدد دورات التراكم، وليس عدد الدفعات الصغيرة.

5. عدم استخدام `DistributedSampler`

الأعراض: كل GPU ترى نفس البيانات، التدريب يتوقف عن التعلم.

الحل: استخدم `DistributedSampler` مع `shuffle=True`.

6. عدم ضبط البذور لكل عملية في التدريب الموزع

الأعراض: نتائج غير قابلة لإعادة الإنتاج، سلوك عشوائي غريب.

الحل: `torch.manual_seed(seed + rank)`، وليس نفس البذرة للجميع.

7. عدم التحقق من `grad_norm`

الأعراض: `NaN` مفاجئ بعد ساعات من التدريب، دون سابق إنذار.

الحل: سجّل `grad_norm` في كل خطوة، وراقب الاتجاه.

8. استخدام `clip_grad_norm_` بعد `optimizer.step()`

الأعراض: لا فائدة من القص، التدرجات تنفجر رغم وجوده.

الحل: استدعِه **بين** `backward()` و `step()`.

9. حفظ النموذج دون المحسّن

الأعراض: عند استئناف التدريب، الخسارة ترتفع أو لا تتعافى.

الحل: احفظ `optimizer.state_dict()` و `scheduler` و `scaler` معًا.

10. عدم اختبار **"Overfit a single batch"** قبل التدريب الطويل

الأعراض: تدرّب لساعات، ثم تكتشف أن الكود فيه خطأ.

الحل: قبل التدريب الكامل، جرّب حفظ 8 عينات فقط. إذا لم تنخفض الخسارة إلى قريب من الصفر، فهناك خطأ.

3.8 أفضل الممارسات: قائمة المهندس المحترف

1. **استخدم BF16 افتراضيًا**، وFP16 فقط عند الضرورة مع Loss Scaling.
2. **راقب** `grad_norm` كأنه مؤشر حيوي. الانفجار المفاجئ إشارة حمراء.
3. **استخدم** `clip_grad_norm_` **دائمًا** في المحوّلات (max_norm=1.0 عادة).
4. **اقسم الخسارة** في Gradient Accumulation. لا تنسَ.
5. **اضبط** `total_steps` لجدولة معدل التعلم بناءً على دورات التراكم.
6. **اختبر Overfit a single batch** قبل أي تدريب طويل.
7. **احفظ checkpoint دوريًا** مع كل الحالة (نموذج، محسّن، جدولة، RNG).
8. **في DDP/FSDP، اضبط البذور لكل عملية**، واستخدم `DistributedSampler`.
9. **لا تستخدم** `set_detect_anomaly` **في التدريب العادي** (بطيء جدًا). استخدمه فقط للتصحيح.
10. **تحقق من المدخلات** من `NaN` و `inf` في مرحلة التطوير.
11. **استخدم** `torch.profiler` لمعرفة أين يقضي النموذج الوقت فعلًا. قد تكتشف أن عنق الزجاجة في `DataLoader` وليس في GPU.
12. **وثّق إعدادات التدريب**: الإصدارات، البذور، حجم الدفعة، معدل التعلم، الأجهزة. "لماذا هذا النموذج أفضل؟" سؤال ستحتاج الإجابة عليه.

3.9 تمرين شامل: مختبر التدريب الحديث

السيناريو

لديك نموذج تصنيف نصوص بسيط (مثل `DistilBERT` أو MLP على تمثيلات نصية). تريد تدريبه على مجموعة بيانات متوسطة الحجم (مثل 50,000 عينة). ذاكرتك محدودة (GPU واحدة بـ 16 غيغابايت). تريد استخدام دفعة فعالة كبيرة (256) لاستقرار التدريب.

المطلوب

1. **ابدأ بتدريب بسيط** (FP32، دفعة 16، بدون تراكم). سجّل الوقت والذاكرة والخسارة النهائية.
2. **فعّل BF16**. قارن الوقت والذاكرة. هل تحسنت الخسارة؟
3. **أضف Gradient Accumulation** (4 خطوات). قارن الوقت والذاكرة والأداء مع الدفعة 64 الحقيقية (إن أمكن).
4. **أضف** `clip_grad_norm_`. راقب `grad_norm` قبل وبعد. هل تغير شيء؟
5. **حاكِ انفجار التدرجات**: ارفع معدل التعلم إلى 1.0. ماذا يحدث؟ كيف يبدو `grad_norm`؟ كيف يبدو `NaN`؟
6. **أصلح المشكلة**: استخدم `clip_grad_norm_`، وأعد معدل التعلم إلى قيمة معقولة. هل تعافى التدريب؟
7. **جرّب جدولة** `OneCycleLR`. تأكد من أن `total_steps` صحيح مع التراكم.
8. **احفظ checkpoint** في منتصف التدريب، ثم استأنف. هل الخسارة تستمر من حيث توقفت؟
9. **اكتب تقريرًا**: ما الإعداد الذي كان الأسرع؟ الأكثر استقرارًا؟ الأفضل من حيث الذاكرة؟ ماذا تعلمت؟

تمرين إضافي (متقدم)

إذا كان لديك وصول إلى أكثر من GPU واحدة (أو يمكنك محاكاة ذلك بـ `torchrun` على CPU)، فجرّب DDP. قارن السرعة مع GPU واحدة. ثم جرّب FSDP إذا كان النموذج أكبر. ماذا تلاحظ؟

تمرين بحثي

اقرأ عن **Loss Scaling** في FP16. لماذا لا تحتاج إليه في BF16؟ اشرح الفرق من حيث مدى الأس (exponent range). ثم جرّب كتابة `GradScaler` يدويًا (بسيطًا) واختبره على نموذج صغير.

خلاصة الفصل: ما يجب أن يبقى في ذهنك

1. **BF16 هو الافتراضي** في النماذج الحديثة. FP16 يحتاج Loss Scaling، وقد يكون أقل استقرارًا. لا تختار FP16 إلا إذا كان جهازك لا يدعم BF16.
2. **Gradient Accumulation** يحاكي الدفعات الكبيرة، لكنه **لا يعادل** الدفعة الكبيرة تمامًا (بسبب LayerNorm/Dropout). واضبط معدل التعلم والجدولة.
3. **DDP** للنماذج التي تتسع في GPU واحدة. **FSDP** للنماذج الأكبر. لا تستخدم FSDP إذا لم تكن بحاجة إليه.
4. `grad_norm` **هو مؤشرك الحيوي**. راقبه دائمًا. الانفجار المفاجئ إشارة إلى مشكلة.
5. `clip_grad_norm_` **ضروري** في المحوّلات. استدعِه بين `backward()` و `step()`.
6. `NaN` **لا يأتي من العدم**. له سبب. ابحث عنه، لا تخفيه بـ `clip_grad_norm_` فقط.
7. **احفظ كل شيء**: النموذج، المحسّن، الجدولة، RNG. استئناف التدريب دون حالة المحسّن قد يدمر النتائج.

8. **اختبر Overfit a single batch** قبل أي تدريب طويل. هذه العادة وحدها قد توفر عليك أيامًا.
9. **الأخطاء الشائعة قاتلة**: نسيان القسمة في التراكم، نسيان `zero_grad`، جدولة على "خطوات" خاطئة، عدم استخدام `DistributedSampler`. احفظ القائمة.
10. **التدريب الحديث ليس سحرًا**. إنه هندسة. كل جزء له سبب، وكل خطأ له عرض. إذا فهمت البنية، ستشخّص المشاكل بسرعة.

افتتاحية الفصل القادم: "الترميز إلى وحدات (Tokenization)"

في الفصل التالي، سنغادر ورشة التدريب، وندخل عالم **التمثيل**. كيف تحوّل النص إلى أرقام؟ لماذا "القطة" و"قطة" قد لا تكون نفس الوحدة؟ ما الفرق بين BPE وWordPiece وSentencePiece؟ ولماذا اللغة العربية تمثل تحديًا خاصًا في الترميز؟

سنتعلم كيف تبني Tokenizer من الصفر، وكيف تقيس كفاءته، وكيف تتجنب الأخطاء التي تجعل النموذج يستهلك ضعف عدد الوحدات التي يحتاجها. وسنجيب على سؤال يبدو بسيطًا لكنه عميق: **هل الوحدة (token) هي الكلمة؟** الإجابة ستغير طريقة تفكيرك في النماذج اللغوية.

> ❓ **سؤال يفتح الشهية**: هل تعلم أن نفس النص الإنجليزي قد يستهلك 1.3 ضعف عدد الوحدات عند ترميزه بالعربية؟ ولماذا هذا مهم للتكلفة والسرعة؟ سنكتشف ذلك في الفصل القادم.

المخطط التوضيحي للفصل



شكل 3.1 — بنية التدريب الحديثة: دقة مختلطة + تراكم + توزيع

02

الجزء الثاني

التضمينات والتمثيلات — كيف تتحول النصوص إلى أرقام

الفصول 4–6 · 3 فصولًا

المحوّلات والنماذج اللغوية الكبيرة · من النظرية إلى بناء أنظمة ذكية

04

الترميز إلى وحدات (Tokenization) — كيف يقرأ النموذج ما لا تراه عينك

افتتاحية: حكاية المترجم الذي كسر النص

في مؤتمر للترجمة الآلية، وقف باحث شاب يعرض نتائجه. قال بثقة: "نموذجي يحقق 95% دقة في تصنيف المشاعر بالعربية". سأله أحد الحضور: "كم عدد الوحدات (tokens) التي يستهلكها النص العربي مقارنة بالإنجليزي؟" توقف الشاب. لم يكن يعرف. لم يكن قد فكر في الأمر أصلًا.

بعد الجلسة، جلس مع زميل له وأجرى تجربة سريعة. أخذ جملة إنجليزية بسيطة: "I love programming". رمّزها بـ GPT-4 tokenizer: **4 وحدات**. ثم أخذ ترجمتها العربية: "أنا أحب البرمجة". رمّزها بنفس الأداة: **9 وحدات**. أكثر من الضعف. ضحك الزميل وقال: "لقد كنت تدفع ضعف التكلفة، وتستهلك ضعف السياق، ولا تعرف".

هذه ليست حكاية عن سوء حظ. إنها حكاية عن **أحد أخطر الأخطاء الصامتة** في بناء أنظمة النماذج اللغوية: تجاهل الترميز. النموذج لا يرى الحروف. لا يرى الكلمات. يرى **أرقامًا**. والطريقة التي تتحول بها الحروف إلى أرقام تحدد كل شيء: التكلفة، السرعة، دقة التمثيل، وقدرة النموذج على فهم لغتك.

هذا الفصل هو رحلة إلى **ما قبل** النموذج. إلى اللحظة التي يتحول فيها النص إلى أرقام. وسنكتشف أن هذه اللحظة ليست تفصيلًا تقنيًا، بل قرار هندسي عميق.

4.1 لماذا لا نستخدم الحروف أو الكلمات مباشرة؟

الخيار الأول: الترميز على مستوى الحروف (Character-Level)

كل حرف يصبح رقمًا. في الإنجليزية، 26 حرفًا + علامات ترقيم + أرقام = مفردات صغيرة (مثلًا 100). الميزة: **لا توجد كلمة خارج المفردات (OOV)**. أي كلمة، حتى لو كانت جديدة، يمكن تمثيلها حرفًا بحرف. العيب: **التسلسلات تصبح طويلة جدًا**. جملة من 20 كلمة قد تصبح 100 حرف. والنموذج يستهلك وقتًا وذاكرة أكبر. والأسوأ: النموذج يجب أن يتعلم أن "c-a-t" تعني "cat"، وهذا يتطلب جهدًا إضافيًا.

الخيار الثاني: الترميز على مستوى الكلمات (Word-Level)

كل كلمة تصبح رقمًا. الميزة: تسلسلات قصيرة، وكل وحدة تحمل معنى. العيب: **المفردات تنفجر**. اللغة الإنجليزية وحدها فيها مئات الآلاف من الكلمات، بما فيها الأسماء والتصريفات. واللغة العربية أغنى صرفيًا، بملايين الصيغ الممكنة. والأسوأ: **الكلمات الجديدة** (أسماء علم، مصطلحات تقنية، أخطاء إملائية) لا يمكن تمثيلها إلا بـ `<UNK>` (رمز المجهول)، وهذا يعني فقدان المعلومات.

الخيار الثالث: الترميز الفرعي (Subword Tokenization) — الحل الوسط

بدلًا من حرف أو كلمة، نقسم النص إلى **وحدات فرعية** ذات معنى. "programming" قد تصبح "program" + "ming". "البرمجة" قد تصبح "ال" + "برمجة". الكلمات الشائعة تبقى وحدة واحدة، والكلمات النادرة تُقسّم إلى أجزاء معروفة. النتيجة: مفردات محدودة (مثلًا 32,000 أو 50,000)، لكن تغطية كاملة، وتسلسلات أقصر من الحروف.

هذا هو النهج السائد اليوم. كل النماذج الحديثة (GPT، LLaMA، BERT، T5، Qwen) تستخدم ترميزًا فرعيًا، لكن بطرق مختلفة. وسنتناول هذه الطرق بالتفصيل.

🤔

سؤال يستحق التفكير: لماذا لا نستخدم ترميزًا على مستوى "المقاطع الصوتية" أو "الجذور الصرفية" في العربية؟ الجواب: قد يكون فعالًا، لكنه يحتاج إلى موارد لغوية متخصصة (قواميس جذور، محاللات صرفية). الترميز الفرعي الإحصائي (مثل BPE) لا يحتاج إلى معرفة لغوية مسبقة، بل يتعلم من البيانات. هذا يجعله عامًا وقابلًا للتطبيق على أي لغة، حتى بدون موارد.

4.2 BPE: الخوارزمية التي بنت نصف نماذج اللغة

الفكرة في سطر واحد

Byte Pair Encoding (BPE) تبدأ بكل حرف كوحدة مستقلة، ثم تدمج repeatedly أكثر زوج من الوحدات تكرارًا في وحدة جديدة، حتى تصل إلى حجم المفردات المطلوب.

القصة: من ضغط البيانات إلى نماذج اللغة

BPE لم تُخترع للنماذج اللغوية. اخترعها Philip Gage في 1994 كخوارزمية **ضغط بيانات**. الفكرة: إذا تكرر زوج من البايتات كثيرًا، استبدله برمز جديد. هذا يقلل حجم الملف. في 2016، استخدمها Sennrich et al. في الترجمة الآلية العصبية لتمثيل الكلمات النادرة. ثم تبنتها (2019) GPT-2، وأصبحت المعيار.

كيف تعمل بالضبط؟ مثال يدوي

لنفترض أن لدينا نصًا صغيرًا: `low lower lowest`. أولًا، نقسم إلى حروف مع نهاية الكلمة (مثل `</w>`):

```txt
l o w </w>
l o w e r </w>
l o w e s t </w>
```

نحسب تكرارات الأزواج المتجاورة:

- `l o`: 3 مرات
- `o w`: 3 مرات
- `w </w>`: 1 مرة
- `w e`: 2 مرات
- `e r`: 1 مرة
- `e s`: 1 مرة

- `s t`: 1 مرة

أكثر زوج تكرارًا هو `l o` (3) أو `o w` (3). نختار أحدهما، لنقل `l o`، وندمجه في وحدة جديدة `lo`. النص يصبح:

```txt
lo w </w>
lo w e r </w>
lo w e s t </w>
```

ثم نكرر: أكثر زوج الآن هو `lo w` (3)، ندمجه في `low`. النص:

```txt
low </w>
low e r </w>
low e s t </w>
```

ثم `low </w>` (1)، `low e` (2)، `e r` (1)، `e s` (1)، `s t` (1). ندمج `low e` (2) في `lowe`. وهكذا. بعد عدد من الدمجات، تصبح المفردات: `low`, `lower`, `lowest`، أو أجزاء منها.

كود: BPE من الصفر

```python
from collections import Counter

def get_vocab(corpus):
    """يحول corpus إلى قاموس من الكلمات المقسومة إلى حروف."""
    vocab = Counter()
    for word in corpus:
        # نضيف </w> لنهاية كل كلمة
        chars = tuple(word) + ("</w>",)
        vocab[chars] += 1
    return vocab

def get_stats(vocab):
    """يحسب تكرارات كل زوج متجاور."""
    pairs = Counter()
    for word, freq in vocab.items():
        for i in range(len(word) - 1):
            pairs[(word[i], word[i+1])] += freq
    return pairs

def merge_vocab(pair, vocab):
    """يدمج pair في كل الكلمات."""
    new_vocab = {}
    bigram = " ".join(pair)
    replacement = "".join(pair)
    for word, freq in vocab.items():
        new_word = []
        i = 0
        while i < len(word):
            if i < len(word) - 1 and word[i] == pair[0] and word[i+1] == pair[1]:
                new_word.append(replacement)
                i += 2
            else:
                new_word.append(word[i])
                i += 1
        new_vocab[tuple(new_word)] = freq
    return new_vocab
```

```python
# مثال
corpus = ["low", "lower", "lowest", "new", "newer", "newest"]
vocab = get_vocab(corpus)
num_merges = 10
merges = []

for i in range(num_merges):
    pairs = get_stats(vocab)
    if not pairs:
        break
    best = max(pairs, key=pairs.get)
    vocab = merge_vocab(best, vocab)
    merges.append(best)
    print(f"Merge {i+1}: {best} (freq={pairs[best]})")

print("\nالمفردات النهائية:")
for word, freq in vocab.items():
    print(f"  {' '.join(word)}: {freq}")
```


ملاحظة: هذا BPE "كلاسيكي" يعمل على مستوى الحروف. GPT-2 يستخدم **Byte-Level BPE**، حيث تبدأ الوحدات كبايتات (0–255)، وليس حروف يونيكود. هذا يسمح بتمثيل **أي** نص، حتى الرموز التعبيرية واللغات غير اللاتينية، دون `<UNK>`. ثمن ذلك: بعض الرموز غير الإنجليزية تستهلك عدة بايتات، فتزيد عدد الوحدات. سنعود إلى هذا لاحقًا.

لماذا **BPE** فعال للغات؟

لأنه **يكتشف الأنماط الشائعة تلقائيًا**. في الإنجليزية، سيكتشف أن "ing" و"ed" و"tion" تظهر كثيرًا، فيدمجها. في العربية، سيكتشف أن "ال" (أداة التعريف) و"ية" (لاحقة النسب) و"ات" (جمع المؤنث) تظهر كثيرًا. النتيجة: الكلمات الشائعة تبقى وحدات كاملة، والكلمات النادرة تُقسّم إلى جذور ولواحق معروفة.



نصيحة: عندما تدرب BPE على بيانات عربية، تأكد من وجود **تشكيل** كافٍ (أو غيابه المتسق). إذا كانت بياناتك مختلطة (بعضها مشكول وبعضها لا)، فستحصل على وحدات مختلفة للكلمة نفسها، وتهدر مساحة المفردات. الأفضل: إما تشكيل كامل، أو إزالة التشكيل بالكامل، باستمرار.

4.3 WordPiece: BPE بمنطق احتمالي

الفرق الجوهري

WordPiece تشبه BPE في الفكرة (دمج الوحدات المتكررة)، لكنها تختار الزوج الذي **يزيد الاحتمالية اللغوية** بأكبر قدر، وليس الزوج الأكثر تكرارًا فقط.

رياضيًا، BPE تختار الزوج (a, b) الذي يعظم التكرار $\text{count}(a, b)$. WordPiece تختار الزوج الذي يعظم **الاحتمالية**:

$$\text{score}(a, b) = (\text{count}(a, b))/(\text{count}(a) \times \text{count}(b))$$

هذا يشبه **نقطة المعلومات المتبادلة (PMI)**. النتيجة: WordPiece تفضل دمج الأزواج التي تظهر معًا **أكثر مما هو متوقع بالصدفة**، وليس فقط الأزواج الشائعة. هذا ينتج مفردات أكثر "دلالية".

أين تُستخدم؟

- **BERT** (وجميع نماذج Google: ALBERT، DistilBERT، ELECTRA).
- بعض النماذج متعددة اللغات.

مثال عملي

WordPiece ترمز "unaffable" إلى `["un", "##aff", "##able"]`. لاحظ البادئة `##` التي تشير إلى أن الوحدة **ليست بداية كلمة**. هذا يساعد النموذج على التمييز بين "able" في بداية الكلمة و"able" في نهايتها.

> ⚖️ **مقارنة**: BPE عادة لا تستخدم `##`. GPT-2 يستخدم مسافة قبل الكلمة (`Ġ` في تمثيله) للإشارة إلى بداية كلمة جديدة. WordPiece تستخدم `##` للوحدات الداخلية. كلاهما يحاول حل نفس المشكلة: كيف نعرف أن هذه الوحدة ليست بداية كلمة؟

كود: WordPiece عبر HuggingFace

```python
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-multilingual-cased")
text = "unaffable programming"
tokens = tokenizer.tokenize(text)
print(tokens)
# ['una', '##ff', '##able', 'programming']
```

4.4 SentencePiece: الترميز المستقل عن اللغة

المشكلة التي حلها

BPE وWordPiece تفترضان أن النص **مقسم إلى كلمات مسبقًا** (عبر المسافات). لكن ماذا عن اللغات التي لا تستخدم المسافات؟ الصينية واليابانية والكورية (CJK) لا تفصل الكلمات بمسافات. وماذا عن العربية التي تستخدم المسافات لكن معقدة صرفيًا؟ وماذا عن اللغات التي تستخدم أنظمة كتابة مختلفة؟

(Kudo & Richardson, 2018) **SentencePiece** تعامل النص كـ **تدفق مستمر من الأحرف**، بما فيها المسافة، وتتعلم التقسيم من البيانات مباشرة. لا تحتاج إلى تقسيم مسبق.

الميزات الرئيسية

1. **تعمل على مستوى Unicode**، وليس البايت فقط.
2. **لا تحتاج إلى تقسيم مسبق للكلمات**.
3. تدعم **BPE** و**Unigram Language Model** كخوارزميتين.
4. تُستخدم في **T5، ALBERT، XLNet، LLaMA، Qwen، DeepSeek** (بدرجات متفاوتة).

Unigram Language Model: النهج الآخر

بدلًا من الدمج (Bottom-Up)، تبدأ Unigram بمفردات كبيرة، ثم **تحذف** الوحدات الأقل فائدة، حتى تصل إلى الحجم المطلوب. النتيجة: توزيع احتمالي على الوحدات، ويمكن توليد تقسيمات متعددة لنفس النص، مع اختيار الأكثر احتمالًا.

⚖️ **مقارنة**: BPE حتمية (تقسيم واحد لكل نص). Unigram احتمالية (قد يكون هناك تقسيمات متعددة، لكن النموذج يختار الأكثر احتمالًا). Unigram قد تكون أفضل للغات ذات الصرف الغني، لأنها تسمح بتمثيلات متعددة للكلمة الواحدة.

كود: **SentencePiece** للعربية

```python
import sentencepiece as spm

# تدريب tokenizer على ملف نصي
spm.SentenceTrainer.train(
    input="arabic_corpus.txt",
    model_prefix="arabic_sp",
    vocab_size=32000,
    model_type="bpe",  # أو "unigram"
    character_coverage=1.0,  # 1.0 للغات الكبيرة، 0.9995 لكل اللغات
    byte_fallback=True,  # للتعامل مع الرموز غير المعروفة
)

# الاستخدام
sp = spm.SentencePieceProcessor(model_file="arabic_sp.model")
text = "أنا أحب البرمجة"
pieces = sp.encode(text, out_type=str)
print(pieces)  # ['أنا', '▁أحب', '▁البرمجة']
ids = sp.encode(text, out_type=int)
print(ids)
```

📌 **لاحظ الرمز** `▁`. هذا رمز خاص في SentencePiece يشير إلى **بداية الكلمة** (أو مسافة). النص الأصلي يُستبدل فيه كل مسافة بـ `▁` قبل التقسيم. هذا يسمح للنموذج بمعرفة حدود الكلمات دون الاعتماد على المسافات في التقسيم نفسه.

💡 **نصيحة**: استخدم `byte_fallback=True` إذا كنت تريد ضمان تمثيل أي رمز، حتى لو لم يكن في المفردات. هذا يحل مشكلة `<UNK>` نهائيًا، لكنه قد يزيد عدد الوحدات للرموز النادرة.

4.5 الترميز بالعربية: تحديات خاصة

لماذا العربية صعبة الترميز؟

العربية ليست مجرد لغة أخرى. لها خصائص تجعل الترميز تحديًا فريدًا:

1. **الصرف الغني**: الكلمة الواحدة قد تحمل معنى جملة كاملة. "فاستقبلناها" = "ف" + "استقبل" + "نا" + "ها". هذا يعني أن عدد الصيغ الممكنة هائل، وأن الترميز على مستوى الكلمة سينفجر.

1. **التشكيل (Diacritics)**: الفتحة، الضمة، الكسرة، السكون، الشدة، التنوين. النص المشكول يستهلك وحدات أكثر بكثير من غير المشكول. والمشكلة: معظم النصوص العربية على الإنترنت غير مشكولة، لكن بعضها مشكول جزئيًا. هذا ينتج تمثيلات غير متسقة.

1. **الهمزات**: "أ" و"إ" و"آ" و"ا" قد تُعتبر حروفًا مختلفة أو نفس الحرف، حسب المعيار. هذا يضاعف عدد الوحدات إذا لم يتم التطبيع.

1. **التطويل (Kashida)**: حرف `ـ` الذي يُستخدم لتمديد النص جماليًا. لا يحمل معنى، لكنه يستهلك وحدة. يجب إزالته أو تجاهله.

1. **الأرقام والرموز المختلطة**: العربية غالبًا ما تُخلط مع الإنجليزية والأرقام، مما يزيد التعقيد.

أرقام حقيقية

في دراسة على tokenizers شائعة، وُجد أن النص العربي يستهلك **1.3 إلى 2.5 ضعف** عدد الوحدات مقارنة بالإنجليزي، حسب الـ tokenizer ونوع النص. هذا يعني:

- **تكلفة أعلى** (إذا كنت تدفع لكل وحدة).
- **سياق أقصر فعليًا** (نافذة السياق 128K لا تعني 128K كلمة عربية).
- **بطء في الاستدلال** (وحدات أكثر = خطوات توليد أكثر).

> **نصيحة عملية**: إذا كنت تبني نظامًا عربيًا، فدرّب tokenizer خاصًا بك على corpus عربي كبير. لا تعتمد على tokenizer GPT-4 أو LLaMA الافتراضي. النتيجة: عدد وحدات أقل بنسبة 30–50%، وسياق أطول فعليًا، وتكلفة أقل. هذا واحد من أعلى الاستثمارات مردودًا في NLP العربي.

كود: قياس كفاءة الترميز

```python
def tokens_per_word(text, tokenizer):
    tokens = tokenizer.encode(text)
    words = len(text.split())
    return len(tokens) / words

# مقارنة
from transformers import AutoTokenizer

texts = {
    "english": "I love programming and building AI systems.",
    "arabic": "أنا أحب البرمجة وبناء أنظمة الذكاء الاصطناعي.",
}

for name, tok_name in [("GPT-4", "cl100k_base"), ("LLaMA", "meta-llama/Llama-2-7b-hf")]:
    print(f"\n=== {name} ===")
    if tok_name == "cl100k_base":
        import tiktoken
        enc = tiktoken.get_encoding(tok_name)
        for lang, text in texts.items():
```

```
            tokens = enc.encode(text)
            print(f"  {lang}: {len(tokens)} tokens, {len(text.split())} words, ratio=
{len(tokens)/len(text.split()):.2f}")
    else:
        tok = AutoTokenizer.from_pretrained(tok_name)
        for lang, text in texts.items():
            tokens = tok.encode(text)
            print(f"  {lang}: {len(tokens)} tokens, {len(text.split())} words, ratio=
{len(tokens)/len(text.split()):.2f}")
```

تلميح: استخدم `ratio` كمقياس. إذا كان النص العربي يستهلك ضعف عدد وحدات النص الإنجليزي، فأنت تخسر نصف سياقك. وإذا كنت تستخدم API، فأنت تدفع ضعف التكلفة.

4.6 مقارنة شاملة بين Tokenizers

الخوارزمية	المفردات	التقسيم المسبق	`<UNK>`	الاستخدام	ملاحظات
BPE (Character)	10K–50K	نعم (مسافات)	ممكن	GPT-2، RoBERTa	بسيط، لكن قد يفشل مع اللغات غير اللاتينية
Byte-Level BPE	50K	نعم	لا	GPT-2/3/4، LLaMA 3	يمثل أي نص، لكن اللغات غير الإنجليزية تستهلك وحدات أكثر
WordPiece	30K	نعم	ممكن	BERT، DistilBERT	احتمالي، `##` للوحدات الداخلية
SentencePiece BPE	32K–50K	لا	لا (مع `byte_fallback`)	T5، ALBERT، Qwen	مستقل عن اللغة، `_` لبداية الكلمة
SentencePiece Unigram	32K–50K	لا	لا	XLNet، بعض نماذج T5	احتمالي، تقسيمات متعددة

أفضل ممارسة: لا تختر tokenizer بناءً على "ما يستخدمه النموذج الكبير". اختر بناءً على **لغتك وبياناتك**. إذا كنت تبني نموذجًا عربيًا، فدرّب SentencePiece على corpus عربي، وقارن `ratio` مع tokenizers الجاهزة. الفرق قد يصل إلى 40% في عدد الوحدات.

4.7 الرموز الخاصة: ما لا تراه في النص

لماذا نحتاج رموزًا خاصة؟

النموذج يحتاج إلى رموز لا تظهر في النص العادي، لكنها تحمل معنى بنيويًا:

الرمز	المعنى	متى يُستخدم
`<pad>`	حشو	لجعل كل التسلسلات بنفس الطول في الدفعة
`<unk>`	مجهول	لكلمة/رمز غير موجود في المفردات

الرمز	المعنى	متى يُستخدم
`<bos>`	بداية تسلسل	في نماذج التوليد (مثل GPT)
`<eos>`	نهاية تسلسل	لإيقاف التوليد
`<mask>`	قناع	في BERT (Masked Language Modeling)
`[SEP]`	فاصل	بين جملتين في BERT
`[CLS]`	تصنيف	في بداية التسلسل في BERT، لتمثيل الجملة

الأخطاء الشائعة

1. **عدم إضافة `<eos>` في نهاية كل تسلسل تدريبي**. النتيجة: النموذج لا يتعلم أين يتوقف، ويستمر في التوليد إلى ما لا نهاية.
2. **استخدام `<pad>` في حساب الخسارة**. النتيجة: النموذج يتعلم توليد الحشو. استخدم `ignore_index` في `cross_entropy`.
3. **عدم توحيد الرموز الخاصة بين التدريب والاستدلال**. النتيجة: النموذج يفسر `<eos>` كرمز عادي، أو لا يتوقف أبدًا.
4. **إضافة `<bos>` في نماذج لا تتوقعه**. بعض النماذج (مثل BERT) لا تستخدم `<bos>`. إضافته تربك النموذج.

> 💡 **نصيحة**: احفظ `special_tokens_map.json` مع tokenizer، واستخدمه دائمًا. لا تكتب الرموز يدويًا. واختبر أن `tokenizer.decode(tokenizer.encode(text)) == text` (مع استثناءات معروفة مثل التطبيع).

4.8 التطبيع (Normalization): عندما تتغير النصوص دون أن تدري

ما هو التطبيع؟

قبل التقسيم، غالبًا ما يُطبَّق **تطبيع** على النص: توحيد الحروف، إزالة التشكيل، تحويل الأرقام، إلخ. هذا يقلل التنوع ويحسن التمثيل. لكنه قد **يغير المعنى** إذا لم يُنفذ بحذر.

أمثلة على التطبيع

- **NFKC** (Unicode Normalization Form KC): يحول الأحرف المركبة إلى مركبة، ويوحد الأشكال. مثلًا، يحول "ﻻ" (ligature) إلى "لا".
- **إزالة التشكيل**: يحول "كَتَبَ" إلى "كتب". مفيد للبحث، لكنه يفقد معلومات نحوية.
- **توحيد الهمزات**: يحول "أ" و"إ" و"آ" إلى "ا". قد يخلط بين كلمات مختلفة (مثل "سأل" و"سال").
- **تحويل الأرقام العربية إلى إنجليزية**: "١٢٣" → "123".

> ⚠️ **تحذير**: التطبيع المفرط قد يمحو معلومات حاسمة. في الترجمة أو التحليل النحوي، قد تحتاج إلى الحفاظ على التشكيل والهمزات. في البحث أو التصنيف، قد يكون التطبيع مفيدًا. **اختر بناءً على المهمة، وليس بشكل أعمى.**

كود: تطبيع عربي بسيط

```python
import re
import unicodedata

def normalize_arabic(text, remove_diacritics=True, unify_alef=True, unify_teh_marbuta=True):
    # إزالة التطويل
    text = re.sub(r"\u0640", "", text)

    if remove_diacritics:
        # إزالة التشكيل (الفتحة، الضمة، الكسرة، السكون، الشدة، التنوين)
        text = re.sub(r"[\u064B-\u0652]", "", text)

    if unify_alef:
        text = re.sub(r"[أإآا]", "ا", text)

    if unify_teh_marbuta:
        text = re.sub(r"ه" ,"ة", text)

    # توحيد المسافات
    text = re.sub(r"\s+", " ", text).strip()

    return text
```

تلميح: لا تطبق التطبيع على النص **قبل** التقسيم فقط، بل **أيضًا** على النص في وقت الاستدلال. إذا دربت على نص مطبع لكن استخدمت نصًا غير مطبع في الاستدلال، فستحصل على تمثيلات مختلفة. **التطبيع يجب أن يكون متسقًا في كل مكان**.

4.9 اختيار حجم المفردات: لعبة التوازن

لماذا حجم المفردات مهم؟

- **مفردات صغيرة** (مثل 10K): تسلسلات أطول، لكن كل وحدة تحمل معنى أقل. قد تزيد التكلفة والوقت.
- **مفردات كبيرة** (مثل 100K): تسلسلات أقصر، لكن المفردات تستهلك ذاكرة أكبر (في طبقة Embedding)، وقد تزيد صعوبة التدريب (كل وحدة تحتاج إلى عينات كافية).

القاعدة العامة

- **الإنجليزية**: 32K–50K شائع.
- **متعدد اللغات**: 100K–250K (لتغطية كل اللغات).
- **العربية**: 32K–64K إذا كان النموذج عربيًا بشكل أساسي. إذا كان متعدد اللغات، فأنت بحاجة إلى مفردات أكبر، لكن تأكد من أن العربية تحصل على نصيب عادل.

مقارنة: LLaMA 2 يستخدم 32K، LLaMA 3 يستخدم 128K (لتحسين الترميز متعدد اللغات)، GPT-4 يستخدم ~100K (cl100k_base). Qwen يستخدم ~150K. النماذج متعددة اللغات تحتاج إلى مفردات أكبر، لكن النماذج أحادية اللغة قد تستفيد من مفردات أصغر وأكثر تركيزًا.

نصيحة: لا تختر حجم المفردات عشوائيًا. درّب tokenizers بأحجام مختلفة (16K، 32K، 64K)، وقس **عدد الوحدات لكل كلمة** و**جودة التمثيل** (عبر مهمة صغيرة). اختر الحجم الذي يعطي أفضل توازن.

4.10 الأخطاء الشائعة: القائمة السوداء للترميز

1. استخدام tokenizer دون التحقق من اللغة

العرض: tokenizer GPT-4 يرمز العربية بشكل سيئ (نسبة 2.0+). أنت تستخدمه على نص عربي، وتتساءل لماذا التكلفة مرتفعة.

الحل: قس `ratio` دائمًا. إذا كان مرتفعًا، درّب tokenizer خاصًا.

2. عدم إضافة `<eos>` في التدريب

العرض: النموذج لا يتوقف عن التوليد، أو يولّد نصًا لا نهائيًا.

الحل: أضف `<eos>` في نهاية كل تسلسل تدريبي، وتأكد من أن النموذج يتعلم إنتاجه.

3. نسيان `attention_mask` مع الحشو

العرض: النموذج يعطي اهتمامًا للحشو، وتتدهور الجودة.

الحل: استخدم `attention_mask`، وتأكد من أن `pad_token_id` صحيح.

```python
batch = tokenizer(texts, padding=True, truncation=True, return_tensors="pt")
# batch["attention_mask"] موجودة تلقائيًا
```

4. تقسيم النص قبل التطبيع

العرض: نفس الكلمة تظهر بتمثيلات مختلفة (بسبب التشكيل أو الهمزات).

الحل: طبّق التطبيع **قبل** التقسيم، وبشكل متسق.

5. عدم اختبار `decode(encode(text)) == text`

العرض: بعض الأحرف تُفقد أو تتغير دون أن تدري.

الحل: اختبر إعادة البناء على عينات متنوعة (عربية، إنجليزية، رموز، إيموجي). قد لا تكون المساواة تامة بسبب التطبيع، لكن يجب أن تكون قريبة.

6. استخدام tokenizer من نموذج آخر دون تعديل

العرض: تستخدم tokenizer BERT مع نموذج GPT. التوزيعات مختلفة، والأداء يتدهور.

الحل: استخدم tokenizer الذي دُرّب مع النموذج، أو أعد تدريب النموذج من الصفر مع tokenizer الجديد.

7. تجاهل `add_special_tokens` في الاستدلال

العرض: النموذج يتصرف بشكل مختلف بين التدريب والاستدلال.

الحل: تأكد من أن `add_special_tokens` متسق. بعض النماذج تحتاج إلى `<bos>` في البداية، وبعضها لا.

4.11 أفضل الممارسات: قائمة المهندس المحترف

1. **قس** `tokens_per_word` قبل أي شيء. إذا كان النص العربي يستهلك ضعف الإنجليزي، فأنت تخسر سياقًا وتكلفة.
2. **درّب tokenizer خاصًا** إذا كانت لغتك غير الإنجليزية. لا تعتمد على الافتراضي.
3. **استخدم SentencePiece مع** `byte_fallback=True` لضمان تمثيل أي رمز دون `<UNK>`.
4. **طبّق التطبيع بشكل متسق** في التدريب والاستدلال. لا تفترض أن النص "نظيف".
5. **أضف** `<eos>` **في نهاية كل تسلسل تدريبي**. لا تنسَ.
6. **استخدم** `attention_mask` مع الحشو، وتأكد من `pad_token_id`.
7. **اختبر** `decode(encode(text))` على عينات متنوعة، بما فيها العربية والإيموجي.
8. **احفظ** `special_tokens_map.json` واستخدمه، ولا تكتب الرموز يدويًا.
9. **لا تختر حجم المفردات عشوائيًا**. جرّب أحجامًا مختلفة وقس.
10. **راقب عدد الوحدات في الإنتاج**. إذا كان يرتفع فجأة، فقد يكون هناك تغيير في توزيع النص (لهجة جديدة، رموز جديدة).
11. **وثّق tokenizer** (الإصدار، التطبيع، الرموز الخاصة). "لماذا تغيرت النتائج؟" سؤال ستحتاج الإجابة عليه.

4.12 تمرين شامل: مختبر الترميز

السيناريو

لديك corpus عربي (يمكنك استخدام أي مجموعة نصوص عربية، مثل مقالات من ويكيبيديا العربية أو مجموعة بيانات مفتوحة). تريد بناء tokenizer عربي، ومقارنته مع tokenizers جاهزة.

المطلوب

1. **نظّف النص**: أزل التطويل، وطبّق تطبيعًا معقولًا (إزالة التشكيل، توحيد الهمزات). سجّل عدد الأحرف قبل وبعد.
2. **درّب SentencePiece BPE** بحجم 32K على 90% من النص. احتفظ بـ 10% للاختبار.
3. **احسب** `tokens_per_word` على مجموعة الاختبار باستخدام tokenizer الخاص بك، وGPT-4 tokenizer (عبر `tiktoken`)، وLLaMA tokenizer (عبر HuggingFace).
4. **قارن**: أي tokenizer يعطي أقل عدد وحدات؟ ما الفرق بالنسبة المئوية؟
5. **اختبر** `decode(encode(text))` على 100 عينة. هل هناك فقدان؟
6. **جرّب أحجام مفردات مختلفة** (16K، 32K، 64K). أيها يعطي أفضل توازن بين عدد الوحدات وحجم المفردات؟
7. **اكتب تقريرًا**: ما التطبيع الذي استخدمته؟ لماذا؟ ما حجم المفردات الذي اخترته؟ ما الفرق مع tokenizers الجاهزة؟ ما القيود؟

تمرين إضافي (متقدم)

قارن بين BPE وUnigram في SentencePiece على نفس corpus. أي منهما يعطي `tokens_per_word` أقل؟ وأيهما يعطي تمثيلات "أكثر دلالية" (يمكنك اختبار ذلك بمهمة تصنيف بسيطة: درّب نموذجًا صغيرًا على تمثيلات كل tokenizer وقارن الدقة).

تمرين بحثي

ابحث عن **"tokenization bias"** في النماذج متعددة اللغات. كيف يؤثر اختيار tokenizer على عدالة الأداء بين اللغات؟ هل يمكن تصميم tokenizer "عادل" يوزع المفردات بشكل متوازن بين اللغات؟ اقرأ عن **"vocabulary allocation"** في النماذج متعددة اللغات.

خلاصة الفصل: ما يجب أن يبقى في ذهنك

1. **النموذج لا يرى النص، يرى أرقامًا**. الترميز هو الجسر، وجودته تحدد كل شيء.
2. **BPE هي الخوارزمية السائدة**، لكن WordPiece (BERT) وSentencePiece (T5/LLaMA) لهما مكانهما. الاختيار يعتمد على اللغة والبيانات.
3. **العربية تمثل تحديًا خاصًا**: الصرف الغني، التشكيل، الهمزات، التطويل. الترميز الافتراضي قد يستهلك ضعف الوحدات.
4. **درّب tokenizer خاصًا بلغتك** إذا كنت جادًا. الفرق في التكلفة والسياق يستحق الاستثمار.
5. **الرموز الخاصة ليست تفصيلًا**: `<eos>`، `<pad>`، `<bos>`، `<mask>`. نسيانها يدمر التدريب.
6. **التطبيع ضروري لكنه خطير**. طبّقه بشكل متسق، واختر ما يناسب مهمتك.
7. **حجم المفردات لعبة توازن**. لا تختر عشوائيًا. جرّب وقس.
8. **اختبر دائمًا**: `ratio`، `decode(encode())`، `attention_mask`. هذه الاختبارات البسيطة تكشف مشاكل كبيرة.
9. **الأخطاء الشائعة قاتلة**: نسيان `<eos>`، تجاهل `attention_mask`، عدم التطبيع، استخدام tokenizer خاطئ. احفظ القائمة.
10. **الترميز ليس "ما قبل المعالجة"**. إنه قرار معماري. خذه بجدية.

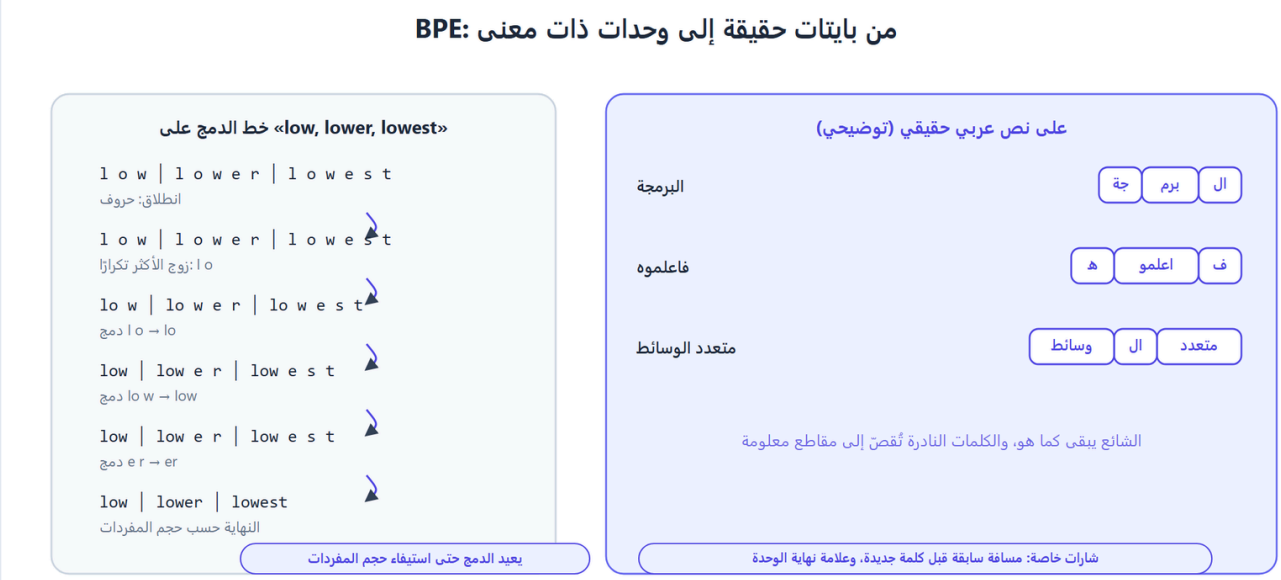
افتتاحية الفصل القادم: "تضمينات الكلمات (Word2Vec وGloVe)"

في الفصل التالي، سنغادر عالم الترميز، وندخل عالم **التمثيل المتجهي**. كيف تتحول الوحدة (token) إلى متجه يحمل معنى؟ لماذا "ملك" - "رجل" + "امرأة" ≈ "ملكة"؟ كيف يتعلم Word2Vec هذا دون أن يعرف أي شيء عن المعنى؟ وما حدود هذه التمثيلات التي أدت إلى ظهور المحوّلات؟

سنتعلم كيف نبني Word2Vec من الصفر، وكيف نقيس جودة التضمينات، ولماذا التضمينات السياقية (مثل BERT) تفوقت على التضمينات الثابتة. وسنجيب على سؤال يبدو بسيطًا: **هل يمكن أن يمثل متجه واحد كلمة "بنك" بكل معانيها؟** الإجابة ستغير فهمك للغة والنماذج.

> ❓ **سؤال يفتح الشهية**: هل تعلم أن Word2Vec يمكن أن تكتشف أن "طوكيو" بالنسبة لليابان مثل "باريس" بالنسبة لفرنسا، دون أن تعرف أي شيء عن الجغرافيا؟ كيف يحدث ذلك؟ سنكتشف في الفصل القادم.

المخطط التوضيحي للفصل



شكل 4.1 — BPE: من بايتات حقيقية إلى وحدات ذات معنى

05

تضمينات الكلمات (Word2Vec وGloVe) — كيف تتحول الكلمة إلى إحداثيات في فضاء المعنى

افتتاحية: حكاية العالم الذي رسم خريطة للكلمات

في عام 2013، كان توماس ميكولوف (Tomáš Mikolov) وفريقه في Google يعملون على مشكلة تبدو بسيطة: كيف نجعل الحاسوب يفهم أن "باريس" و"طوكيو" متشابهتان؟ لا شكليًا (الحروف مختلفة تمامًا)، ولا وظيفيًا (كلتاهما عاصمة). لكنهما متشابهتان **دلاليًا**. كيف نمثل هذا التشابه رقميًا؟

كان الحل الذي توصلوا إليه أنيقًا لدرجة أنه بدا سحرًا: **مروا كل كلمة إلى متجه، بحيث تكون الكلمات المتشابهة في المعنى قريبة في الفضاء**. لم يتوقف الأمر عند التشابه. اكتشفوا أن العلاقات الدلالية تصبح **عمليات رياضية**:

$$\text{vec("king")} - \text{vec("man")} + \text{vec("woman")} \approx \text{vec("queen")}$$

تخيل ذلك. متجه "ملك" ناقص متجه "رجل" زائد متجه "امرأة" يعطيك متجه "ملكة". وكأن المعنى له **هندسة**. وكأن اللغة يمكن أن تُرسم على خريطة، والاتجاهات على هذه الخريطة تحمل معاني: اتجاه "الجنس"، اتجاه "الزمن"، اتجاه "الملكية".

هذا الاكتشاف لم يكن مجرد فضول أكاديمي. كان الأساس الذي بُنيت عليه كل النماذج اللغوية الحديثة. حتى المحوّلات التي نستخدمها اليوم، بدأت من فكرة بسيطة: **الكلمة ليست رمزًا معزولًا، بل نقطة في فضاء، ونقطة تُفهم من جيرانها**.

هذا الفصل هو رحلة إلى ذلك الفضاء. كيف بُني؟ كيف نقرأه؟ وما حدوده التي أدت إلى ظهور المحوّلات؟

5.1 المشكلة: لماذا لا يكفي One-Hot Encoding؟

التمثيل الساذج

أبسط طريقة لتمثيل كلمة رقميًا هي **One-Hot Encoding**: متجه طوله يساوي حجم المفردات، فيه 1 في موضع الكلمة، و0 في كل مكان آخر.

```txt
king   = [0, 0, 0, 1, 0, 0, ..., 0]
queen  = [0, 0, 0, 0, 1, 0, ..., 0]
apple  = [0, 0, 1, 0, 0, 0, ..., 0]
```

المشكلة الأولى: **التمثيل ضخم**. مفردات بـ 50,000 كلمة تعني متجهًا بـ 50,000 بُعد. إذا كان لديك مليون كلمة في corpus، فهذا تريليون رقم. لا يمكن تخزينه، ولا معالجته.

المشكلة الثانية، وهي الأهم: **كل الكلمات متساوية المسافة**. المسافة بين "king" و"queen" هي نفس المسافة بين "king" و"apple". كلاهما $\sqrt{2}$. لا يوجد أي تمثيل للتشابه. النموذج لا يستطيع أن يعرف أن "king" أقرب إلى "queen" من "apple".

المشكلة الثالثة: **لا يوجد تعميم**. إذا تعلم النموذج شيئًا عن "king"، فلا يمكنه تطبيقه على "queen"، لأنهما متجهان مستقلان تمامًا.

> 🤔
> **سؤال يستحق التفكير**: لماذا لا نستخدم تمثيلًا يعتمد على خصائص يدوية (مثل: هل الكلمة اسم؟ هل هي مذكر؟ هل هي عاصمة؟)؟ الجواب: لأن الخصائص اليدوية تحتاج إلى خبراء لغويين، ولا تغطي كل شيء. والأهم: الكلمات تحمل معاني يصعب وصفها بالكلمات. "الشعر" و"الغزل" و"الحنين" — كيف تصفها بخصائص محدودة؟ التضمينات تتعلم هذه الخصائص من البيانات نفسها، دون تدخل بشري.

5.2 الفكرة الثورية: الكلمة تُعرف بجيرانها

فرضية التوزيع (Distributional Hypothesis)

في عام 1957، كتب اللغوي جون فيرث (J.R. Firth) جملة أصبحت مبدأً في معالجة اللغة:

> 📌
> "You shall know a word by the company it keeps." (ستعرف الكلمة من رفاقها.)

الفكرة: الكلمات التي تظهر في سياقات متشابهة، لها معاني متشابهة. "القطة" و"الكلب" يظهران مع "يجري"، "يأكل"، "ينام". "السيارة" و"الطائرة" يظهران مع "يسافر"، "سريع"، "محرك". إذًا، يمكننا تمثيل معنى الكلمة من خلال **توزيع سياقاتها**.

Word2Vec وGloVe هما مجرد طريقتان لتحويل هذه الفكرة إلى خوارزمية. الفكرة نفسها أقدم بكثير، لكن الأدوات الحسابية هي التي جعلتها ممكنة.

5.3 Word2Vec: الشبكة العصبية الصغيرة التي هزت العالم

الفكرة المعمارية

Word2Vec ليست نموذجًا واحدًا، بل **معماريتان**:

1. **CBOW** (Continuous Bag of Words): تنبأ بالكلمة من سياقها.
2. **Skip-gram**: تنبأ بالسياق من الكلمة.

كلتاهما شبكة عصبية بسيطة جدًا: طبقة إدخال، طبقة مخفية (وهي التضمين الذي نريده)، طبقة إخراج. لا دالة تنشيط في الطبقة المخفية. لا عمق. ومع ذلك، تنتج تمثيلات مذهلة.

5.3.1 CBOW: السياق يتنبأ بالكلمة

تخيل الجملة: "القطة ___ على السجادة". الكلمة المفقودة هي "جلست". CBOW تأخذ الكلمات المحيطة ("القطة"، "على"، "السجادة")، وتحاول التنبؤ بالكلمة المفقودة.

المعمارية:

- المدخل: متوسط متجهات الكلمات السياقية (أو مجموعها).
- الطبقة المخفية: بُعد d (مثلًا 300).
- المخرج: توزيع احتمالي على المفردات كلها.

PYTHON
```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class CBOW(nn.Module):
    def __init__(self, vocab_size, embed_dim):
        super().__init__()
        self.embeddings = nn.Embedding(vocab_size, embed_dim)
        self.output = nn.Linear(embed_dim, vocab_size)

    def forward(self, context_ids):
        # context_ids: (batch, context_size)
        embeds = self.embeddings(context_ids)          # (batch, context_size, dim)
        context_vec = embeds.mean(dim=1)               # (batch, dim)
        logits = self.output(context_vec)              # (batch, vocab_size)
        return logits
```

5.3.2 Skip-gram: الكلمة تتنبأ بالسياق

العكس: نأخذ كلمة ("جلست")، ونحاول التنبؤ بالكلمات المحيطة ("القطة"، "على"). يبدو هذا أصعب، لكنه في الواقع يعطي تمثيلات **أفضل** للكلمات النادرة، لأنه يولّد عينات أكثر لكل كلمة.

المعمارية:

- المدخل: كلمة واحدة.
- المخرج: توزيع على المفردات لكل موضع في السياق.

PYTHON
```python
class SkipGram(nn.Module):
    def __init__(self, vocab_size, embed_dim):
        super().__init__()
        self.embeddings = nn.Embedding(vocab_size, embed_dim)
        self.output = nn.Linear(embed_dim, vocab_size)

    def forward(self, center_ids, context_ids):
        # center_ids: (batch,)
        # context_ids: (batch, context_size)
        center_embeds = self.embeddings(center_ids)    # (batch, dim)
        logits = self.output(center_embeds)            # (batch, vocab_size)
        # نقارن مع كل كلمة في السياق
        loss = F.cross_entropy(
            logits.unsqueeze(1).expand(-1, context_ids.size(1), -1).reshape(-1, logits.size(-1)),
            context_ids.reshape(-1),
        )
        return loss
```

⚖️ **مقارنة**: CBOW أسرع، ومناسب للكلمات الشائعة. Skip-gram أبطأ، لكنه أفضل للكلمات النادرة والتمثيلات الدلالية الدقيقة. الورقة الأصلية توصي بـ Skip-gram كخيار افتراضي.

5.3.3 المشكلة: softmax على 50,000 كلمة مكلف

حساب softmax على المفردات كاملة في كل خطوة تدريب مكلف جدًا. إذا كانت المفردات 100,000 كلمة، فكل خطوة تحتاج إلى حساب 100,000 احتمال. ومليون خطوة تدريب تعني مئة مليار عملية. لا يمكن.

الحل: Negative Sampling

بدلًا من حساب softmax على كل المفردات، نحوّل المشكلة إلى **تصنيف ثنائي**: هل هذا الزوج (كلمة مركزية، كلمة سياق) حقيقي أم مزيف؟

- **عينات موجبة**: أزواج حقيقية من النص (مثل: "القطة"، "جلست").
- **عينات سالبة**: أزواج عشوائية (مثل: "القطة"، "محرك").
- النموذج يتعلم أن يميز الموجب من السالب.

رياضيًا، نخفض المشكلة من $O(V)$ إلى $O(k)$، حيث k عدد العينات السالبة (عادة 5–20). تسريع هائل.

```python
class SkipGramNegativeSampling(nn.Module):
    def __init__(self, vocab_size, embed_dim):
        super().__init__()
        self.center_embeddings = nn.Embedding(vocab_size, embed_dim)
        self.context_embeddings = nn.Embedding(vocab_size, embed_dim)

    def forward(self, center_ids, pos_context_ids, neg_context_ids):
        # center: (batch,)
        # pos: (batch,)
        # neg: (batch, k)
        center = self.center_embeddings(center_ids)         # (batch, dim)
        pos = self.context_embeddings(pos_context_ids)      # (batch, dim)
        neg = self.context_embeddings(neg_context_ids)      # (batch, k, dim)

        # النقاط: كلما زادت، زاد احتمال أن الزوج حقيقي
        pos_score = torch.sum(center * pos, dim=-1)         # (batch,)
        neg_score = torch.bmm(neg, center.unsqueeze(-1)).squeeze(-1)  # (batch, k)

        # نريد: pos_score عاليًا، neg_score منخفضًا
        loss = -(F.logsigmoid(pos_score).mean() + F.logsigmoid(-neg_score).mean())
        return loss
```

> **ملاحظة مهمة**: في Word2Vec الأصلية، يوجد **مصفوفتان** للتضمين: واحدة للكلمة المركزية، وواحدة للسياق. بعد التدريب، عادة ما نجمع الاثنتين، أو نستخدم الأولى فقط. هذا التفصيل الصغير يحسن الأداء بشكل ملحوظ.

5.3.4 Negative Sampling: كيف نختار العينات السالبة؟

ليس عشوائيًا بالتساوي. الكلمات الشائعة (مثل "في"، "من") تظهر كثيرًا، فلو اخترناها عشوائيًا، ستطغى على التدريب. الورقة الأصلية تقترح اختيارها باحتمال يتناسب مع **تكرارها مرفوعًا للأس 0.75**:

$$P(w) = (freq(w)^{0.75}) / (\Sigma_{w'} freq(w')^{0.75})$$

هذا يقلل هيمنة الكلمات الشائعة، ويعطي فرصة أكبر للكلمات المتوسطة والنادرة.

🤔

سؤال يستحق التفكير: لماذا 0.75 وليس 1.0 أو 0.5؟ الجواب: 1.0 يعطي الكلمات الشائعة وزنًا كبيرًا جدًا، فيصبح التدريب مركزًا على تمييز "في" من "من"، وهذا غير مفيد. 0.5 يقلل من أهمية الكلمات الشائعة أكثر من اللازم، وقد يضعف التمثيل. 0.75 قيمة تجريبية وجدت توازنًا. هذا مثال جميل على أن المعاملات الفائقة في التعلم العميق غالبًا ما تكون تجريبية، وليس نظرية بحتة.

5.3.5 كود كامل: Word2Vec من الصفر

PYTHON

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from collections import Counter
import random

# ===== 1. تجهيز البيانات =====
def build_vocab(sentences, min_count=1):
    counter = Counter()
    for sent in sentences:
        counter.update(sent)
    vocab = {w: i for i, (w, c) in enumerate(counter.items()) if c >= min_count}
    return vocab, counter

def generate_pairs(sentences, vocab, window=2):
    pairs = []
    for sent in sentences:
        ids = [vocab.get(w, -1) for w in sent]
        ids = [i for i in ids if i >= 0]
        for i, center in enumerate(ids):
            for j in range(max(0, i - window), min(len(ids), i + window + 1)):
                if j != i:
                    pairs.append((center, ids[j]))
    return pairs

# ===== 2. Dataset =====
class SkipGramDataset(Dataset):
    def __init__(self, pairs, vocab_size, n_neg=5, power=0.75):
        self.pairs = pairs
        self.vocab_size = vocab_size
        self.n_neg = n_neg
        # توزيع العينات السالبة
        self.neg_probs = torch.tensor([power], dtype=torch.float32)  # سيُحدّث لاحقًا

    def set_neg_distribution(self, counter, vocab):
        freqs = torch.zeros(self.vocab_size)
        for w, i in vocab.items():
            freqs[i] = counter[w] ** 0.75
        self.neg_probs = freqs / freqs.sum()

    def __len__(self):
        return len(self.pairs)

    def __getitem__(self, idx):
        center, pos = self.pairs[idx]
        negatives = torch.multinomial(self.neg_probs, self.n_neg, replacement=True)
        return torch.tensor(center), torch.tensor(pos), negatives

# ===== 3. النموذج =====
```

```python
class Word2Vec(nn.Module):
    def __init__(self, vocab_size, embed_dim):
        super().__init__()
        self.center = nn.Embedding(vocab_size, embed_dim)
        self.context = nn.Embedding(vocab_size, embed_dim)
        # تهيئة صغيرة
        nn.init.uniform_(self.center.weight, -0.5 / embed_dim, 0.5 / embed_dim)
        nn.init.zeros_(self.context.weight)

    def forward(self, center, pos, neg):
        c = self.center(center)                    # (B, D)
        p = self.context(pos)                      # (B, D)
        n = self.context(neg)                      # (B, K, D)

        pos_score = (c * p).sum(-1)                # (B,)
        neg_score = torch.bmm(n, c.unsqueeze(-1)).squeeze(-1)  # (B, K)

        loss = -(F.logsigmoid(pos_score).mean() + F.logsigmoid(-neg_score).mean())
        return loss

# ===== 4. التدريب =====
def train_word2vec(sentences, embed_dim=100, window=2, n_neg=5,
                   epochs=5, batch_size=256, lr=0.025):
    vocab, counter = build_vocab(sentences)
    pairs = generate_pairs(sentences, vocab, window)
    dataset = SkipGramDataset(pairs, len(vocab), n_neg)
    dataset.set_neg_distribution(counter, vocab)
    loader = DataLoader(dataset, batch_size=batch_size, shuffle=True, num_workers=2)

    model = Word2Vec(len(vocab), embed_dim)
    optimizer = optim.Adam(model.parameters(), lr=lr)

    for epoch in range(epochs):
        total_loss = 0
        for center, pos, neg in loader:
            optimizer.zero_grad()
            loss = model(center, pos, neg)
            loss.backward()
            optimizer.step()
            total_loss += loss.item()
        print(f"Epoch {epoch+1} | Loss: {total_loss / len(loader):.4f}")

    # التضمين النهائي: متوسط المصفوفتين
    embeddings = (model.center.weight + model.context.weight).detach().numpy()
    id2word = {i: w for w, i in vocab.items()}
    return embeddings, vocab, id2word

# ===== 5. البحث عن أقرب الجيران =====
def nearest_neighbors(word, embeddings, vocab, id2word, top_k=10):
    from sklearn.metrics.pairwise import cosine_similarity
    if word not in vocab:
        print(f"'{word}' not in vocab")
        return
    idx = vocab[word]
    sims = cosine_similarity([embeddings[idx]], embeddings)[0]
    top = sims.argsort()[::-1][1:top_k+1]
    return [(id2word[i], float(sims[i])) for i in top]

# ===== مثال =====
if __name__ == "__main__":
    sentences = [
        "الملك يحكم البلاد".split(),
        "الملكة تحكم البلاد".split(),
```

```
        "الرجل يعمل في الحقل".split(),
        "المرأة تعمل في الحقل".split(),
        # ... (أضف المزيد من الجمل المتنوعة)
    ]
    embeddings, vocab, id2word = train_word2vec(sentences, embed_dim=50, epochs=10)
    for w in ["الرجل" ,"الملك"]:
        print(w, "→", nearest_neighbors(w, embeddings, vocab, id2word, 3))
```

⚠ **تحذير**: هذا الكود تعليمي، وليس جاهزًا للإنتاج. Word2Vec الأصلية تستخدم **subsampling** للكلمات الشائعة (لتحسين الجودة والسرعة)، و**learning rate decay**، و**phrase detection**. للاستخدام الحقيقي، استخدم `gensim.models.Word2Vec`، التي تحتوي على كل هذه التحسينات.

5.3.6 Subsampling: التخلص من الكلمات المزعجة

الكلمات مثل "في"، "من"، "على" تظهر في كل مكان. هي ضرورية للنحو، لكنها لا تحمل معنى دلاليًا غنيًا. Word2Vec تقترح **حذفها عشوائيًا** أثناء التدريب، باحتمال يتناسب مع تكرارها:

$$P(\text{drop}(w)) = 1 - \sqrt{(t)/(f(w))}$$

حيث t عتبة (عادة 10^{-5})، و$f(w)$ تكرار الكلمة. النتيجة: الكلمات الشائعة جدًا تُحذف بنسبة عالية، والكلمات النادرة تبقى. هذا يسرّع التدريب ويحسّن جودة التمثيلات.

💡 **نصيحة**: إذا كنت تنفذ Word2Vec بنفسك، **لا تتجاهل subsampling**. الفرق في الجودة ملحوظ، والتدريب يصبح أسرع بكثير.

5.4 GloVe: نهج مختلف، نفس الهدف

الفكرة

GloVe (Global Vectors) لا تستخدم شبكة عصبية، بل تستخدم **مصفوفة التكرار المشترك** (Co-occurrence Matrix). الفكرة: بدلًا من توقع الكلمات واحدًا واحدًا، انظر إلى **الإحصاءات العالمية** لكيفية ظهور الكلمات معًا.

كيف تعمل؟

1. ابنِ مصفوفة X حيث X_{ij} هو عدد المرات التي تظهر فيها الكلمة j في سياق الكلمة i.
2. عرّف $P_{ij} = X_{ij} / \Sigma_k X_{ik}$ (احتمال ظهور j في سياق i).
3. تعلم متجهات بحيث يكون حاصل الضرب النقطي بين متجهين قريبًا من لوغاريتم التكرار المشترك:

$$w_i^T \tilde{w}_j + b_i + \tilde{b}_j \approx \log X_{ij}$$

1. دالة الخسارة تزن الأخطاء بناءً على تكرار X_{ij}، لأن الأزواج النادرة لا يجب أن تهيمن.

مقارنة: Word2Vec أم GloVe؟

	Word2Vec	GloVe
النهج	تنبؤي (Predictive)	إحصائي (Count-based)
التدريب	شبكة عصبية	تحلل مصفوفة
البيانات	تحتاج إلى corpus كبير	نفس الشيء
السرعة	أسرع في التدريب	أبطأ قليلًا
الجودة	ممتازة للكلمات النادرة (Skip-gram)	ممتازة للتمثيلات العامة
التحديث	سهل (تدريب مستمر)	صعب (يحتاج إعادة بناء المصفوفة)

📌 **الحقيقة المهمة**: في الممارسة، الفرق بين Word2Vec وGloVe غالبًا **صغير**، ويعتمد على البيانات والمهمة. لا يوجد "أفضل" مطلق. جرّب الاثنين وقس.

GloVe في الواقع

GloVe يتوفر بنماذج مدربة مسبقًا (50d، 100d، 200d، 300d) على corpora ضخمة (Wikipedia، Common Crawl). هذا يجعله سهل الاستخدام:

```python
# باستخدام gensim
import gensim.downloader as api

# تحميل GloVe (بُعد 100) مدرب مسبقًا
glove = api.load("glove-wiki-gigaword-100")
print(glove.most_similar("king", topn=5))
# [('prince', 0.77), ('monarch', 0.73), ('throne', 0.72), ...]

# العملية الشهيرة
result = glove.most_similar(positive=["king", "woman"], negative=["man"], topn=1)
print(result)  # [('queen', 0.71)]
```

💡 **تلميح**: إذا كنت تبني نظامًا عربيًا، فـ GloVe النموذج الجاهز لن يفيدك. تحتاج إلى تدريب GloVe على corpus عربي، أو استخدام Word2Vec العربية المدربة مسبقًا (مثل `aravec`). تأكد من أن التطبيع والترميز متسقان.

5.5 تقييم التضمينات: كيف نعرف أنها "جيدة"؟

التقييم الداخلي (Intrinsic)

يقيس جودة التضمينات **بمعزل عن أي مهمة**، عبر اختبارات لغوية:

1. **التشابه الدلالي (Similarity)**: هل الكلمات المتشابهة قريبة؟ تُقاس عادةً على مجموعات مثل WordSim-353.

2. **القياس التناظري (Analogy)**: "king" - "man" + "woman" = "queen"؟ تُقاس على مجموعات مثل Google Analogies.
3. **التصنيف (Categorization)**: هل الكلمات من نفس الفئة متجمعة؟

التقييم الخارجي (Extrinsic)

يقيس جودة التضمينات **من خلال مهمة حقيقية**: تصنيف مشاعر، ترجمة، NER. تُدرّب نموذجًا بسيطًا على التضمينات، وتقيس الأداء. هذا هو التقييم **الأهم** عمليًا.

> ⚠️ **تحذير**: التقييم الداخلي قد يكون مضللًا. تضمينات تتفوق على Analogy قد تكون أسوأ في مهمة حقيقية. لا تختار التضمينات بناءً على Analogy فقط. اختبر على مهمتك.

كود: تقييم التشابه الدلالي

```python
from sklearn.metrics.pairwise import cosine_similarity
from scipy.stats import spearmanr

def evaluate_similarity(embeddings, vocab, test_pairs):
    """test_pairs: list of (word1, word2, human_score)"""
    human_scores, model_scores = [], []
    for w1, w2, score in test_pairs:
        if w1 in vocab and w2 in vocab:
            v1 = embeddings[vocab[w1]].reshape(1, -1)
            v2 = embeddings[vocab[w2]].reshape(1, -1)
            model_scores.append(cosine_similarity(v1, v2)[0][0])
            human_scores.append(score)
    return spearmanr(human_scores, model_scores).correlation
```

> 📌 **لماذا Spearman وليس Pearson؟** لأننا نهتم بـ **ترتيب** التشابهات، وليس قيمها المطلقة. إذا كانت التضمينات تقول إن "قطة-كلب" أكثر تشابهًا من "قطة-سيارة"، فهذا صحيح حتى لو كانت القيم المطلقة مختلفة عن تقييم البشر.

كود: تقييم Analogy

```python
def evaluate_analogy(embeddings, vocab, id2word, analogies):
    """analogies: list of (a, b, c, expected_d)"""
    correct, total = 0, 0
    for a, b, c, expected in analogies:
        if not all(w in vocab for w in [a, b, c]):
            continue
        target = embeddings[vocab[a]] - embeddings[vocab[b]] + embeddings[vocab[c]]
        sims = cosine_similarity([target], embeddings)[0]
        # استبعد الكلمات المدخلة
        for w in [a, b, c]:
            sims[vocab[w]] = -1
        pred_idx = sims.argmax()
```

```
        total += 1
        if id2word[pred_idx] == expected:
            correct += 1
    return correct / total if total > 0 else 0
```

💡 **نصيحة**: عند تقييم Analogy، **استبعد الكلمات المدخلة** من البحث. وإلا، قد تحصل على الكلمة المدخلة نفسها (خاصة "a") كنتيجة، وهذا خطأ شائع.

5.6 حدود التضمينات الثابتة: لماذا جاءت المحوّلات؟

المشكلة الأولى: الكلمة الواحدة، متجه واحد

في Word2Vec وGloVe، كل كلمة لها **متجه واحد ثابت**، بغض النظر عن السياق. لكن "بنك" في "جلست على ضفة البنك" تختلف تمامًا عن "ذهبت إلى البنك لأسحب المال". التضمين الثابت يعطي نفس المتجه للاثنين، فيخلط المعاني.

هذه المشكلة تسمى **Polysemy** (تعدد المعاني). وهي ليست هامشية. معظم الكلمات الشائعة لها معانٍ متعددة. التضمينات الثابتة تعاني منها.

المشكلة الثانية: الكلمات الجديدة

Word2Vec وGloVe تتعلم من corpus محدد. أي كلمة جديدة (مصطلح تقني، اسم علم، كلمة عامية جديدة) لا تكون في المفردات، فتُعامل كـ `<UNK>`. حتى مع الترميز الفرعي (Subword)، التضمين الناتج قد لا يكون مثاليًا.

المشكلة الثالثة: لا يوجد تمثيل للجملة

Word2Vec تعطي متجهًا لكل كلمة، لا لكل جملة. لتضمين جملة، كنا نأخذ متوسط متجهات الكلمات (Mean Pooling). هذا يفقد الترتيب النحوي والعلاقات بين الكلمات. جملة "الكلب عض الرجل" و"الرجل عض الكلب" لهما نفس المتوسط تقريبًا، رغم أن معناهما مختلف تمامًا.

الحل: التضمينات السياقية

هذه المشاكل أدت إلى ظهور **ELMo** (2018)، ثم **BERT** و**GPT** (2018–2019). هذه النماذج تعطي **متجهًا مختلفًا للكلمة حسب سياقها**. "بنك" في سياق مالي لها متجه مختلف عن "بنك" في سياق نهري. الكلمة الجديدة يمكن تمثيلها من أجزائها (subwords). والجملة كلها يمكن تمثيلها عبر متجهات سياقية لكل كلمة، أو عبر تجميع خاص.

🤔 **سؤال يستحق التفكير**: هل التضمينات الثابتة أصبحت عديمة الفائدة؟ لا. ما زالت تستخدم في: (1) أنظمة التوصية، (2) البحث الدلالي السريع، (3) التمثيلات الأولية في بعض المهام، (4) كأداة تعليمية لفهم التمثيلات. لكنها لم تعد الخيار الأول للنماذج اللغوية. المحوّلات تفوقت عليها.

5.7 التضمينات في العالم العربي: تحديات وفرص

التحدي: الصرف الغني

العربية لغة صرفية غنية. الكلمة الواحدة قد تحمل معنى جملة. "فاستقبلناها" = "ف" + "استقبل" + "نا" + "ها". هذا يعني أن عدد الصيغ الممكنة هائل، وأن Word2Vec على مستوى الكلمة ستعاني من تناثر البيانات (data sparsity). كل صيغة تحتاج إلى عينات كافية لتعلم تمثيل جيد، وهذا غير ممكن للصيغ النادرة.

الحل: الترميز الفرعي أو التطبيع

1. **الترميز الفرعي (Subword)**: بدلًا من كلمة كاملة، استخدم BPE/SentencePiece. الكلمة النادرة تُقسّم إلى جذع ولواحق معروفة، فيتعلم النموذج من الأجزاء.
2. **التطبيع (Normalization)**: إزالة التشكيل، توحيد الهمزات، إزالة التطويل. هذا يقلل التنوع، ويزيد تكرار كل صيغة.
3. **التجذيع (Stemming) أو الاشتقاق (Lemmatization)**: تحويل الكلمة إلى جذرها أو أصلها المعجمي. "يكتب"، "كاتب"، "مكتوب"، "كتابة" كلها تعود إلى "كتب". هذا يقلل المفردات بشكل كبير، لكنه قد يفقد معلومات صرفية.

⚖️ **مقارنة**: Stemming سريع لكنه خشن (قد يخطئ). Lemmatization دقيق لكنه يحتاج إلى محالل صرفي (مثل Farasa أو MADAMIRA). في الممارسة، الترميز الفرعي غالبًا يكفي، ولا تحتاج إلى تجذيع صريح.

نماذج عربية مدربة مسبقًا

- **AraVec**: تضمينات Word2Vec وGloVe مدربة على corpora عربية كبيرة (تويتر، ويكيبيديا، نصوص حديثة).
- **FastText العربية**: يستخدم الترميز الفرعي (n-grams)، فيتعامل مع الصيغ النادرة أفضل من Word2Vec.
- **BERT العربية** (AraBERT، MARBERT، CAMeLBERT): تضمينات سياقية.

💡 **نصيحة**: إذا كنت تبني نظامًا عربيًا، فابدأ بـ **FastText** أو **BERT**، وليس Word2Vec. FastText يتعامل مع الصرف الغني بشكل أفضل، وBERT يعطي تمثيلات سياقية. Word2Vec قد يكون مفيدًا كخط أساس بسيط، لكنه ليس الخيار الأمثل.

كود: استخدام FastText للعربية

```python
from gensim.models import FastText

# تدريب
sentences = [line.split() for line in open("arabic_corpus.txt", encoding="utf-8")]
model = FastText(sentences, vector_size=100, window=5, min_count=5, epochs=10, sg=1)

# الكلمات المتشابهة
model.wv.most_similar("البرمجة", topn=5)

# كلمة غير موجودة في المفردات: FastText يمثلها من n-grams
model.wv["برمجه"]  # لها متجه، حتى لو لم تكن في corpus
```

5.8 الأخطاء الشائعة: القائمة السوداء

1. استخدام Word2Vec على corpus صغير

العرض: التضمينات لا معنى لها، والجيران عشوائيون.

الحل: Word2Vec تحتاج إلى **ملايين** الكلمات لتعطي تمثيلات جيدة. إذا كان corpus صغيرًا، استخدم نموذجًا مدربًا مسبقًا، أو FastText، أو BERT.

2. تجاهل التطبيع قبل التدريب

العرض: "القطة" و"قطة" و"القطه" لها متجهات مختلفة، والجودة متدهورة.

الحل: طبّق تطبيعًا متسقًا (إزالة التشكيل، توحيد الهمزات) قبل التدريب.

3. عدم استخدام Subsampling

العرض: التدريب بطيء، والكلمات الشائعة تطغى على التمثيل.

الحل: استخدم subsampling للكلمات الشائعة (كما في الورقة الأصلية).

4. تجاهل Negative Sampling distribution

العرض: التضمينات ضعيفة للكلمات المتوسطة.

الحل: استخدم توزيع 0.75 (وليس توزيعًا متساويًا) لاختيار العينات السالبة.

5. استخدام `model.wv[word]` لكلمة غير موجودة

العرض: `KeyError`.

الحل: استخدم `model.wv.key_to_index` للتحقق، أو `model.wv.most_similar` التي تتعامل مع الأخطاء. مع FastText، يمكنك استخدام `model.wv[word]` حتى لو لم تكن الكلمة في المفردات (بفضل n-grams).

6. تقييم على Analogy فقط

العرض: التضمينات تتفوق على Analogy، لكنها تفشل في مهمتك الحقيقية.

الحل: قيّم على **مهمتك** (تصنيف، ترجمة، NER) وليس فقط على Analogy.

7. عدم تطبيع المتجهات قبل حساب التشابه

العرض: المسافات لا تعكس التشابه بشكل صحيح.

الحل: استخدم `cosine_similarity` (التي تطبع داخليًا)، أو `sklearn.preprocessing.normalize`.

8. استخدام Embedding ثابت مع نموذج يستخدم التضمينات السياقية

العرض: النموذج يعطي نتائج سيئة رغم أن التضمينات "جيدة".

الحل: لا تخلط. BERT يحتاج إلى tokenizer وتضمينات خاصة به. Word2Vec لا تعمل مع BERT.

9. نسيان أن Word2Vec ليست حتمية

العرض: نتائج مختلفة في كل تشغيل.

الحل: اضبط البذرة (`seed=42`)، ولكن تذكر أن التوازي في التدريب قد يسبب اختلافات طفيفة.

10. تجاهل معالجة الكلمات النادرة

العرض: الكلمات التي تظهر مرة واحدة لها تمثيل عشوائي.

الحل: استخدم `min_count` (عادة 5 أو 10) لحذف الكلمات النادرة جدًا، أو استخدم FastText (n-grams) لتمثيلها من أجزائها.

5.9 أفضل الممارسات: قائمة المهندس المحترف

1. **نظّف النص أولًا**: تطبيع، إزالة تشكيل، إزالة رموز غريبة. التضمينات تعكس جودة البيانات.
2. **استخدم نموذجًا مدربًا مسبقًا** إن أمكن. تدريب Word2Vec من الصفر يحتاج إلى corpus ضخم.
3. **استخدم FastText للعربية** بدلًا من Word2Vec، لأنه يتعامل مع الصرف الغني والكلمات النادرة.
4. **لا تقيّم على Analogy فقط**. اختبر على مهمتك.
5. **استخدم Subsample + Negative Sampling** إذا كنت تدرب من الصفر.
6. **تطبيع المتجهات** قبل حساب التشابه (cosine).
7. **احفظ النموذج والمفردات** معًا. لا تفصل بينهما.
8. **راقب الكلمات غير الموجودة (OOV)**. إذا كانت النسبة عالية، فأنت بحاجة إلى FastText أو Subword.
9. **لا تخلط بين Word2Vec وBERT**. لكل منهما tokenizer وتضمينات خاصة.
10. **فكر في التكلفة**: Word2Vec رخيص وسريع. BERT أغلى. اختر بناءً على المهمة.

5.10 تمرين شامل: مختبر التضمينات

السيناريو

لديك corpus عربي (يمكنك استخدام أي مجموعة نصوص عربية). تريد بناء تضمينات، ومقارنة نماذج مختلفة.

المطلوب

1. **نظّف النص**: إزالة التشكيل، توحيد الهمزات، إزالة التطويل. احفظ نسخة نظيفة.
2. **درّب Word2Vec** (Skip-gram، بُعد 100، نافذة 5، min_count=5) باستخدام `gensim`.
3. **درّب FastText** بنفس الإعدادات.
4. **حمّل AraVec** أو أي نموذج عربي مدرب مسبقًا.
5. **قيّم**: لكل نموذج، احسب:

- متوسط التشابه بين 50 زوجًا من الكلمات المتشابهة (يدويًا أو من مجموعة تقييم).
- أداء على مهمة تصنيف مشاعر صغيرة (درّب logistic regression على متوسط التضمينات).

1. **قارن**: أي نموذج أفضل؟ لماذا؟ هل الفرق يستحق التعقيد؟
2. **اختبر OOV**: أعطِ كل نموذج 20 كلمة جديدة (مصطلحات تقنية، أسماء). أي نموذج يمثلها أفضل؟
3. **اكتب تقريرًا**: ما التطبيع الذي استخدمته؟ ما الإعدادات؟ ما النتائج؟ ما القيود؟

تمرين إضافي (متقدم)

قارن بين Word2Vec وBERT على مهمة **Word Sense Disambiguation** (تمييز معاني الكلمات). اختر 10 كلمات متعددة المعاني (مثل "بنك"، "عين"، "سوق")، واجمع 5 جمل لكل معنى. قس: هل Word2Vec تعطي نفس المتجه لكل الجمل؟ هل BERT تعطي متجهات مختلفة حسب السياق؟ أي منهما يميز المعاني بشكل أفضل؟

تمرين بحثي

ابحث عن **"bias in word embeddings"**. كيف تعكس Word2Vec وGloVe تحيزات اجتماعية (مثلًا، "طبيب" أقرب إلى "رجل" و"ممرضة" أقرب إلى "امرأة")؟ ما التقنيات المقترحة لإزالة هذا التحيز؟ اقرأ عن **"Debiasing Word Embeddings"** (Bolukbasi et al., 2016).

خلاصة الفصل: ما يجب أن يبقى في ذهنك

1. **One-Hot ضخم ولا يمثل التشابه**. التضمينات تحل هذه المشكلة.
2. **فرضية التوزيع**: الكلمة تُعرف بجيرانها. Word2Vec وGloVe يترجمان هذه الفكرة إلى خوارزميات.
3. **CBOW أسرع، Skip-gram أفضل للكلمات النادرة**. كلاهما يحتاج إلى Negative Sampling لتسريع التدريب.
4. **GloVe نهج إحصائي**، وليس تنبؤيًا. النتائج غالبًا متقاربة مع Word2Vec.
5. **التقييم الخارجي أهم من الداخلي**. اختبر على مهمتك، وليس فقط على Analogy.
6. **التضمينات الثابتة لها حدود**: تعدد المعاني، الكلمات الجديدة، لا تمثيل للجملة. المحوّلات حلت هذه المشاكل.
7. **العربية صعبة**: الصرف الغني، التشكيل، الهمزات. استخدم FastText أو BERT، وليس Word2Vec الأصلي.
8. **لا تنسَ التطبيع، ولا Subsample، ولا Negative Sampling Distribution**.
9. **الأخطاء الشائعة قاتلة**: corpus صغير، تجاهل التطبيع، تقييم على Analogy فقط. احفظ القائمة.
10. **التضمينات ليست مجرد أرقام**. إنها خريطة للمعنى. تعلم قراءتها، وستفهم النماذج اللغوية أعمق.

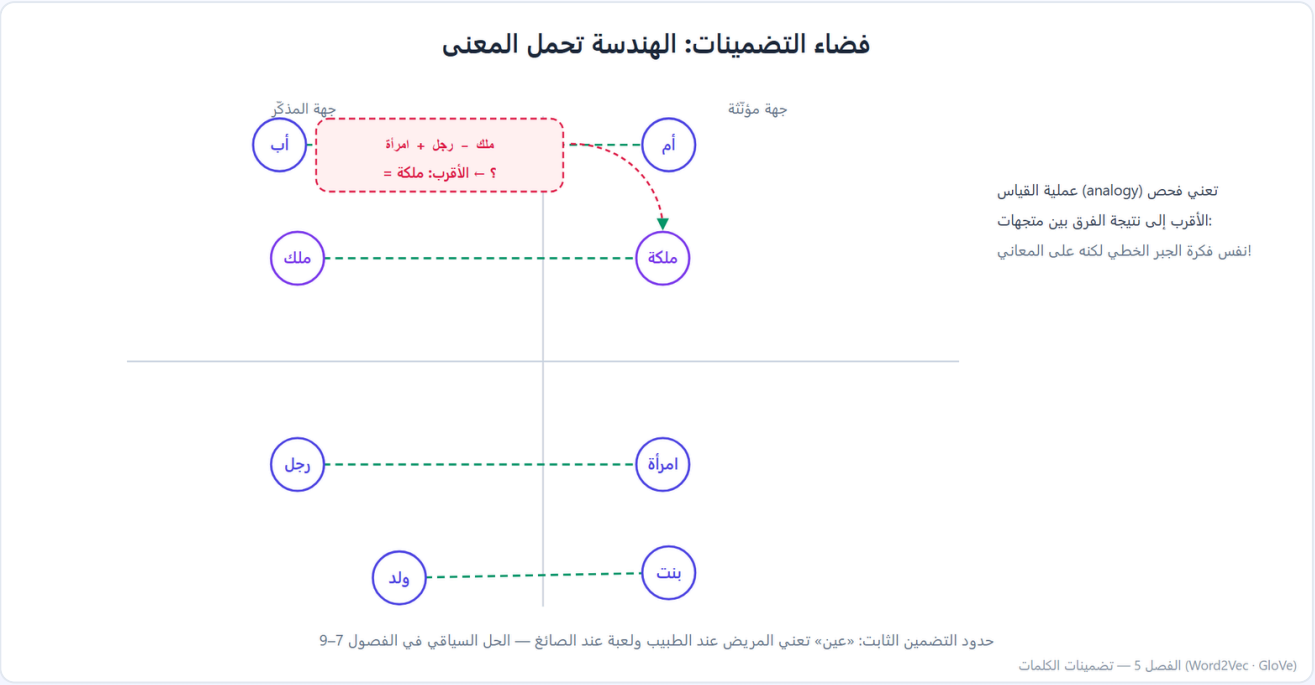
افتتاحية الفصل القادم: "تضمينات الجُمل — RNN وLSTM وGRU"

في الفصل التالي، سنغادر عالم الكلمات المفردة، وندخل عالم **التسلسلات**. كيف نمثل جملة كاملة؟ كيف يتذكر النموذج بداية الجملة عند الوصول إلى نهايتها؟ لماذا RNN البسيطة تفشل في الجمل الطويلة، وكيف حلت LSTM ذلك؟ وما هي GRU التي قدمت حلا أبسط؟

سنتعلم كيف نبني RNN وLSTM من الصفر، وكيف ندربها، ولماذا كانت هي الحدث الكبير قبل المحوّلات. وسنجيب على سؤال عميق: **هل يمكن للشبكة العصبية أن "تتذكر"؟** الإجابة ستأخذنا إلى قلب مشكلة الاعتماد طويل المدى، التي أدت مباشرة إلى اختراع الانتباه.

سؤال يفتح الشهية: هل تعلم أن LSTM صُممت في 1997، لكنها لم تنتشر إلا بعد 2013؟ ولماذا؟ ما الذي تغير؟ سنكتشف في الفصل القادم.

المخطط التوضيحي للفصل



شكل 5.1 — فضاء التضمينات: الهندسة تحمل المعنى

06

تضمينات الجُمل — RNN وLSTM وGRU: كيف تتذكر الشبكة ما نسيته؟

افتتاحية: حكاية القارئ الذي نسي بداية الجملة

تخيل أنك تقرأ رواية بوليسية. في الصفحة الأولى، يظهر شخص اسمه "سامي"، ويُوصف بأنه "رجل هادئ، يعمل في مكتبة، ويكره العنف". في الصفحة الثلاثمئة، يُقتل شخص في ظروف غامضة، ويظهر سامي في المشهد. أنت فورًا تشك فيه. لماذا؟ لأنك **تتذكر** ما قرأته منذ ثلاثمئة صفحة، وتربطه بالحاضر.

الآن تخيل أنك تقرأ نفس الرواية، لكن ذاكرتك **تُمسح كل عشر صفحات**. في الصفحة الثلاثمئة، سترى سامي كشخص غريب، ولن تشك فيه. ستفقد الخيط. ستفقد المعنى.

هذه بالضبط مشكلة **الشبكات العصبية التغذوية (Feedforward Networks)** التي استخدمناها في الفصول السابقة. كل مدخل يُعالج **بمعزل** عن الآخر. لا ذاكرة. لا سياق. لا ترابط. إذا أردنا أن نفهم جملة، أو نصًا، أو محادثة، فنحن بحاجة إلى شيء **يتذكر**.

هذا الفصل هو رحلة إلى **الذاكرة في الشبكات العصبية**. سنبدأ من RNN البسيطة، التي وعدت بالذاكرة لكنها نسيت بسرعة، ثم ننتقل إلى LSTM التي حلت المشكلة ببوابات ذكية، ثم GRU التي بسّطت الحل. وفي النهاية، سنكتشف لماذا **كل هذه المحاولات لم تكن كافية**، ولماذا كان لا بد من اختراع المحوّلات.

6.1 لماذا لا تكفي الشبكات التغذوية؟

المشكلة الأولى: الطول المتغير

في التصنيف التقليدي، كل مدخل له **حجم ثابت** (مثل صورة 224×224). لكن الجمل تختلف في الطول. جملة من 3 كلمات وجملة من 50 كلمة. كيف نمررها إلى شبكة تتوقع حجمًا ثابتًا؟

الحل الساذج: الحشو (Padding) إلى طول أقصى. لكن هذا يهدر الحساب، وقد يربك النموذج (كلمات الحشو ليست كلمات حقيقية).

المشكلة الثانية: الترتيب مهم

جملة "الكلب عض الرجل" و"الرجل عض الكلب" لهما نفس الكلمات، لكن معنى مختلف تمامًا. إذا مررنا الكلمات إلى شبكة تغذوية (حتى مع متوسط التضمينات)، فسنحصل على نفس الناتج تقريبًا. **الترتيب يضيع**.

المشكلة الثالثة: لا سياق تراكمي

عندما نقرأ "أعطى الكتاب للفتاة التي قابلها في المكتبة"، نفهم أن "ها" تعود إلى "الفتاة"، وأن "ها" الثانية (في "قابلها") تعود إلى "الفتاة" أيضًا، وأن "هو" في "قابلها" يعود إلى الشخص الذي أعطى. هذا الفهم **تراكمي**: كل كلمة تعتمد على ما قبلها.

الشبكات التغذوية لا تستطيع ذلك. تحتاج إلى **حالة (state)** تنتقل من خطوة إلى خطوة. وهذا ما تقدمه **الشبكات العصبية المتكررة (Recurrent Neural Networks)**.

6.2 RNN: الفكرة البسيطة التي فتحت الباب

الفكرة في سطر واحد

بدلًا من معالجة الجملة كوحدة، عالجها **كلمة كلمة**، واحتفظ بـ **حالة مخفية (hidden state)** تتحدث مع كل كلمة جديدة، وتُمرر إلى الخطوة التالية.

المعادلة

في كل خطوة t:

$$h_t = \tanh(W_{hh}\, h_{t-1} + W_{xh}\, x_t + b_h)$$

$$y_t = W_(hy)\, h_t + b_(y)$$

حيث:

- x_t: مدخل الخطوة t (تضمين الكلمة).
- h_{t-1}: الحالة المخفية من الخطوة السابقة (ذاكرة).
- h_t: الحالة المخفية الجديدة.
- y_t: المخرج (اختياري، حسب المهمة).

تشبيه: قراءة رواية

تخيل أنك تقرأ رواية. في كل صفحة، تتذكر ما حدث سابقًا (h_{t-1})، وتدمجه مع ما تقرأه الآن (x_t)، لتكوّن فهمًا محدثًا (h_t). هذا الفهم المحدث هو ما تحمله إلى الصفحة التالية.

RNN من الصفر

```python
import torch
import torch.nn as nn

class SimpleRNN(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super().__init__()
        self.hidden_dim = hidden_dim
        self.W_xh = nn.Linear(input_dim, hidden_dim, bias=False)
        self.W_hh = nn.Linear(hidden_dim, hidden_dim, bias=True)
```

```
        self.W_hy = nn.Linear(hidden_dim, output_dim)

    def forward(self, x, h0=None):
        # x: (batch, seq_len, input_dim)
        batch_size, seq_len, _ = x.size()
        if h0 is None:
            h0 = torch.zeros(batch_size, self.hidden_dim, device=x.device)

        h = h0
        outputs = []
        for t in range(seq_len):
            h = torch.tanh(self.W_xh(x[:, t, :]) + self.W_hh(h))
            outputs.append(h)

        outputs = torch.stack(outputs, dim=1)  # (batch, seq_len, hidden)
        logits = self.W_hy(outputs)             # (batch, seq_len, output_dim)
        return logits, h
```

📌 **ملاحظة**: في PyTorch، هناك `nn.RNN` جاهزة تستخدم `cuDNN` على GPU، وهي أسرع بكثير من الحلقة اليدوية. لكن كتابة الحلقة مرة واحدة تساعدك على فهم ما يحدث تحت الغطاء. استخدم `nn.RNN` في الإنتاج.

تدريب RNN: Backpropagation Through Time (BPTT)

كيف ندرّب RNN؟ نمرر التسلسل كاملًا، نحسب الخسارة، ثم ننتشر عكسيًا **عبر الزمن**. هذا يسمى **BPTT**. المشكلة: كل خطوة زمنية تضيف طبقة إلى الرسم البياني الحسابي، فالتسلسلات الطويلة تستهلك ذاكرة هائلة.

الحل: **Truncated BPTT**. نقسم التسلسل إلى أجزاء (مثلًا 35 خطوة)، ونحسب التدرج على كل جزء، مع الاحتفاظ بالحالة المخفية بين الأجزاء (لكن دون تدرج عبرها).

PYTHON

```
def train_rnn(model, data, epochs=10, seq_len=35, lr=1e-3):
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    criterion = nn.CrossEntropyLoss()

    for epoch in range(epochs):
        h = None
        total_loss = 0
        for batch in data:
            optimizer.zero_grad()
            logits, h = model(batch["input"], h0=h.detach() if h is not None else None)
            loss = criterion(logits.view(-1, logits.size(-1)), batch["target"].view(-1))
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            optimizer.step()
            total_loss += loss.item()
        print(f"Epoch {epoch+1} | Loss: {total_loss / len(data):.4f}")
```

⚠️ **تحذير**: `h.detach()` ضرورية. بدونها، ستحتفظ بالرسم البياني الحسابي عبر كل الدفعات، وستنفد الذاكرة. هذا خطأ شائع جدًا.

6.3 مشكلة الاعتماد طويل المدى: عندما تنسى RNN

المشكلة

RNN البسيطة **تنسى** المعلومات القديمة. إذا كانت الجملة "القطة التي كانت تجلس على السجادة الحمراء في غرفة المعيشة **كانت** متعبة"، فإن RNN قد تنسى "القطة" بحلول الوقت الذي تصل فيه إلى "كانت". والسبب: التدرج يتلاشى.

لماذا يتلاشى التدرج؟

تذكر من الفصل الأول: الانتشار العكسي يستخدم قاعدة السلسلة. في RNN، التدرج عبر T خطوة يتضمن حاصل ضرب T مشتقة. إذا كانت المشتقات أقل من 1، فحاصل الضرب يتلاشى أسيًا. إذا كانت أكبر من 1، ينفجر.

رياضيًا:

$$\partial L/\partial h_0 = (\partial L)/(\partial h_(T)) \; \Pi_{t=1}{}^{T} \; (\partial h_t)/(\partial h_{t-1})$$

إذا كان $|(\partial h_t)/(\partial h_{t-1})| < 1$، فبعد 50 خطوة، يصبح التدرج $\sim 10^{-10}$، أي صفر تقريبًا. النموذج لا يتعلم من البداية.

مشكلة الانفجار

إذا كانت المشتقات أكبر من 1، ينفجر التدرج، وتصبح الأوزان `NaN`. الحل البسيط: `clip_grad_norm_` (كما فعلنا). الحل الجذري: LSTM.

> **نصيحة**: `clip_grad_norm_` ضرورية في كل تدريب RNN/LSTM/GRU. لا تتجاهلها. القيمة الشائعة 1.0 أو 5.0.

تجربة فكرية

تخيل أنك تحاول تعلم قاعدة: "الفاعل في بداية الجملة يحدد جنس الفعل في نهايتها". إذا كانت الجملة من 5 كلمات، فقد تنجح RNN. إذا كانت من 50 كلمة، تفشل. هذا بالضبط ما أظهرته الأبحاث: RNN تتعلم الاعتماديات القصيرة (حتى 10–20 خطوة)، لكنها تفشل في الطويلة.

6.4 LSTM: البوابات التي أنقذت الذاكرة

الفكرة الثورية

في 1997، اقترح هوخريتر وشميدهوبر (Hochreiter & Schmidhuber) **Long Short-Term Memory (LSTM)**. الفكرة: بدلًا من حالة مخفية واحدة، احتفظ بـ **حالة خلوية (cell state)** تعمل كـ "طريق سريع للمعلومات"، مع **بوابات** تتحكم في ما يدخل ويخرج ويُنسى.

البوابات الثلاث

1. **بوابة النسيان (Forget Gate)**: ماذا نتجاهل من الحالة السابقة؟

$$f_t = \sigma(W_(f) \; [h_{t-1}, x_t] + b_(f))$$

1. **بوابة الإدخال (Input Gate)**: ماذا نضيف من المعلومات الجديدة؟

$$i_t = \sigma(W_i \, [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_(C) \, [h_{t-1}, x_t] + b_(C))$$

1. **بوابة الإخراج (Output Gate)**: ماذا نكشف من الحالة الخلوية؟

$$o_t = \sigma(W_o \, [h_{t-1}, x_t] + b_o)$$

تحديث الحالة

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

$$h_t = o_t \odot \tanh(C_t)$$

حيث $\odot$ هو الضرب العنصري (element-wise).

لماذا تحل LSTM مشكلة التلاشي؟

لأن الحالة الخلوية C_t تُحدّث عبر **جمع**، وليس ضربًا متسلسلًا. إذا كانت $f_t \approx 1$، فالمعلومات القديمة تمر دون تغيير. التدرج لا يتلاشى بنفس السرعة. هذا "الطريق السريع" هو ما يجعل LSTM قادرة على تعلم اعتماديات تصل إلى مئات الخطوات.

> 📌 **تشبيه**: تخيل أن RNN البسيطة مثل شخص يهمس برسالة من شخص لآخر في صف طويل. بعد 50 شخصًا، الرسالة أصبحت مشوهة. LSTM مثل شخص يكتب الرسالة على ورقة ويمررها. الورقة تصل كما هي، لأنها لا تعتمد على الذاكرة البشرية.

LSTM من الصفر

PYTHON

```python
class LSTMCell(nn.Module):
    def __init__(self, input_dim, hidden_dim):
        super().__init__()
        self.hidden_dim = hidden_dim
        # نجمع كل الأوزان في مصفوفة واحدة للكفاءة
        self.W = nn.Linear(input_dim + hidden_dim, 4 * hidden_dim)

    def forward(self, x, state):
        h_prev, c_prev = state
        combined = torch.cat([x, h_prev], dim=-1)
        gates = self.W(combined)
        i, f, g, o = gates.chunk(4, dim=-1)

        i = torch.sigmoid(i)
        f = torch.sigmoid(f)
        g = torch.tanh(g)
        o = torch.sigmoid(o)
```

```
        c = f * c_prev + i * g
        h = o * torch.tanh(c)
        return h, (h, c)

class LSTM(nn.Module):
    def __init__(self, input_dim, hidden_dim, num_layers=1, dropout=0.0):
        super().__init__()
        self.hidden_dim = hidden_dim
        self.num_layers = num_layers
        self.cells = nn.ModuleList([
            LSTMCell(input_dim if l == 0 else hidden_dim, hidden_dim)
            for l in range(num_layers)
        ])
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, state=None):
        # x: (batch, seq_len, input_dim)
        batch_size, seq_len, _ = x.size()
        if state is None:
            h = torch.zeros(batch_size, self.hidden_dim, device=x.device)
            c = torch.zeros(batch_size, self.hidden_dim, device=x.device)
            state = [(h, c) for _ in range(self.num_layers)]

        outputs = []
        for t in range(seq_len):
            inp = x[:, t, :]
            new_state = []
            for l, cell in enumerate(self.cells):
                h, c = cell(inp, state[l])
                inp = self.dropout(h) if l < self.num_layers - 1 else h
                new_state.append((h, c))
            state = new_state
            outputs.append(inp)

        outputs = torch.stack(outputs, dim=1)
        return outputs, state
```

> **نصيحة**: في الإنتاج، استخدم `nn.LSTM`، التي تستخدم `cuDNN` وتدعم `pack_padded_sequence` للتعامل مع التسلسلات متغيرة الطول بكفاءة. الحلقة اليدوية تعليمية فقط.

استخدام nn.LSTM مع الحشو

PYTHON

```
from torch.nn.utils.rnn import pack_padded_sequence, pad_packed_sequence

# lengths: (batch,) طول كل تسلسل قبل الحشو
packed = pack_padded_sequence(x, lengths.cpu(), batch_first=True, enforce_sorted=False)
output_packed, (h_n, c_n) = lstm(packed)
output, _ = pad_packed_sequence(output_packed, batch_first=True)
```

> **تحذير**: `enforce_sorted=False` مهمة إذا كانت التسلسلات غير مرتبة. بدونها، `pack_padded_sequence` ستعطي نتائج خاطئة بصمت. هذا خطأ شائع يقع فيه الكثيرون.

GRU 6.5: تبسيط LSTM دون خسارة كبيرة

الفكرة

في 2014، اقترح شو (Cho) وآخرون **Gated Recurrent Unit (GRU)**. الفكرة: LSTM رائعة، لكنها معقدة (4 مجموعات أوزان). هل يمكن تبسيطها؟

GRU تدمج بوابة النسيان والإدخال في **بوابة تحديث (update gate)** واحدة، وتضيف **بوابة إعادة تعيين (reset gate)**:

$$z_t = \sigma(W_(z) [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W_h [r_t \odot h_{t-1}, x_t])$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

مقارنة: LSTM أم GRU؟

	LSTM	GRU
البوابات	3 (نسيان، إدخال، إخراج)	2 (تحديث، إعادة تعيين)
الحالة	h_t وC_t	h_t فقط
عدد المعاملات	أكثر	أقل (~25%)
السرعة	أبطأ قليلًا	أسرع
الأداء	ممتاز في التسلسلات الطويلة جدًا	ممتاز في معظم المهام، وأحيانًا أفضل
متى تختاره	مهام معقدة، تسلسلات طويلة	مهام عامة، بيانات محدودة

📌 **حقيقة**: في كثير من المهام، GRU تتفوق على LSTM رغم بساطتها. لا يوجد "أفضل" مطلق. جرّب الاثنين.

GRU من الصفر (مختصر)

```python
class GRUCell(nn.Module):
    def __init__(self, input_dim, hidden_dim):
        super().__init__()
        self.W_z = nn.Linear(input_dim + hidden_dim, hidden_dim)
        self.W_r = nn.Linear(input_dim + hidden_dim, hidden_dim)
        self.W_h = nn.Linear(input_dim + hidden_dim, hidden_dim)

    def forward(self, x, h):
        combined = torch.cat([x, h], dim=-1)
        z = torch.sigmoid(self.W_z(combined))
        r = torch.sigmoid(self.W_r(combined))
```

```
        combined_r = torch.cat([x, r * h], dim=-1)
        h_tilde = torch.tanh(self.W_h(combined_r))
        h_new = (1 - z) * h + z * h_tilde
        return h_new
```

6.6 RNN ثنائية الاتجاه (Bidirectional)

الفكرة

في جملة "أعطى الكتابَ للفتاة التي قابلها"، الكلمة "ها" تعتمد على "الفتاة" التي **قبلها**. لكن في "قابلها في المكتبة"، "ها" تعتمد على "الفتاة" التي **قبلها بعدة كلمات**، وقد تحتاج إلى معلومات من **بعدها** أيضًا.

RNN أحادية الاتجاه ترى الماضي فقط. **BiRNN** تشغل RNN في اتجاهين: من البداية إلى النهاية، ومن النهاية إلى البداية، ثم تدمج الحالتين.

PYTHON

```python
class BiLSTM(nn.Module):
    def __init__(self, input_dim, hidden_dim, num_layers=1):
        super().__init__()
        self.lstm = nn.LSTM(
            input_dim, hidden_dim, num_layers,
            batch_first=True, bidirectional=True
        )
        self.fc = nn.Linear(hidden_dim * 2, hidden_dim)

    def forward(self, x):
        out, _ = self.lstm(x)  # (batch, seq, hidden*2)
        return self.fc(out)
```

متى تستخدم BiRNN؟

- **تصنيف النصوص**: نعم، لأن لديك الجملة كاملة.
- **NER**: نعم.
- **الترجمة**: لا (أو بحذر)، لأن التوليد يحدث من اليسار إلى اليمين، ولا يمكنك رؤية المستقبل أثناء التوليد.
- **توليد النصوص**: لا، لنفس السبب.

> ⚠
> **تحذير**: لا تستخدم BiRNN في مهام التوليد المتسلسل (مثل GPT). سترى المستقبل، وهذا "تسريب" (leakage) في التدريب، وسلوك غير صحيح في الاستدلال.

6.7 Encoder-Decoder: الجد الذي مهد للمحوّلات

الفكرة

لحل مشكلة الترجمة (مدخل ومخرج بتسلسلين مختلفين)، اقترح سوتسكفر (Sutskever) وآخرون في 2014 معمارية **Encoder-Decoder**:

1. **Encoder**: يقرأ الجملة المصدر كاملة، وينتج **متجه سياق (context vector)** يمثل المعنى.

2. **Decoder**: يأخذ متجه السياق، ويولّد الجملة الهدف كلمة كلمة.

```
"القطة تجلس" → [Encoder LSTM] → [c] → [Decoder LSTM] → "The cat is sitting"
```
TEXT

المشكلة: عنق الزجاجة

كل المعلومات يجب أن تُضغط في **متجه واحد** (c). الجمل الطويلة تفقد معلومات. هذا "عنق الزجاجة" أدى إلى **آلية الانتباه** (Bahdanau et al., 2014)، حيث يرى Decoder **كل** حالات Encoder، وليس فقط الأخيرة.

وهذه الآلية هي الجد المباشر لـ **Self-Attention** في المحوّلات. سنراها بالتفصيل في الفصل القادم.

> **درس تاريخي**: المحوّلات لم تأتِ من العدم. جاءت من رحم مشاكل RNN. فهم RNN يجعل فهم المحوّلات أعمق بكثير.

كود مبسّط لـ Encoder-Decoder

```python
class Encoder(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True)

    def forward(self, x):
        emb = self.embed(x)
        outputs, (h, c) = self.lstm(emb)
        return outputs, (h, c)

class Decoder(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, x, state):
        emb = self.embed(x)
        outputs, state = self.lstm(emb, state)
        logits = self.fc(outputs)
        return logits, state
```
PYTHON

6.8 التضمينات التسلسلية للجمل: كيف نحصل على متجه واحد؟

بعد تدريب RNN/LSTM، كيف نحصل على **متجه واحد** يمثل الجملة؟

الطرق الشائعة

1. **آخر حالة مخفية (h_T)**: بسيط، لكنه يفقد المعلومات المبكرة.
2. **متوسط الحالات المخفية**: يحافظ على المعلومات، لكنه يوزع الوزن بالتساوي.
3. **Max Pooling**: يأخذ أقصى قيمة في كل بُعد. يفيد في التقاط الميزات البارزة.

4. **Attention Pooling**: يتعلم أوزانًا لكل خطوة. الأفضل، لكنه أكثر تعقيدًا.
5. **حالة LSTM الخلوية**: في بعض المهام، C_T أفضل من h_T.

> **نصيحة**: لا تفترض أن h_T هو الأفضل. جرّب المتوسط وMax Pooling. في مهام التصنيف، المتوسط غالبًا يتفوق. في مهام التسلسل، قد تحتاج إلى الحالات كلها.

كود: Average Pooling مع الحشو

```python
def mean_pooling(hidden_states, attention_mask):
    """
    hidden_states: (batch, seq, hidden)
    attention_mask: (batch, seq) - 1 للكلمات الحقيقية، 0 للحشو
    """
    mask = attention_mask.unsqueeze(-1).float()
    summed = (hidden_states * mask).sum(dim=1)
    counts = mask.sum(dim=1).clamp(min=1e-9)
    return summed / counts
```

> **تحذير**: بدون `attention_mask`، الحشو سيؤثر على المتوسط. هذا خطأ شائع جدًا ويؤدي إلى تمثيلات مشوهة.

6.9 تقييم نماذج التسلسل

المقاييس

- **تصنيف الجملة**: Accuracy، F1.
- **NER**: F1 على مستوى الكيان (entity-level)، وليس الرمز (token-level).
- **الترجمة**: BLEU، ROUGE، METEOR، COMET.
- **توليد النص**: Perplexity، BLEU، وتقييم بشري.

> **تحذير**: BLEU لها حدود معروفة (لا تلتقط المعنى، تعتمد على n-grams). لا تعتمد عليها وحدها في الترجمة. استخدم COMET أو BERTScore أو تقييمًا بشريًا.

كود: تقييم NER بـ seqeval

```python
from seqeval.metrics import classification_report, f1_score

y_true = [["B-PER", "I-PER", "O", "B-LOC"]]
y_pred = [["B-PER", "I-PER", "O", "B-LOC"]]
print(classification_report(y_true, y_pred))
```

6.10 الأخطاء الشائعة: القائمة السوداء

1. نسيان `h.detach()` في Truncated BPTT

العرض: نفاد الذاكرة بعد عدة دفعات.

الحل: `h.detach()` عند تمرير الحالة بين الدفعات.

2. عدم استخدام `pack_padded_sequence`

العرض: الحشو يؤثر على الحالة النهائية، والنتائج غير دقيقة.

الحل: استخدم `pack_padded_sequence` مع `enforce_sorted=False` .

3. تجاهل `clip_grad_norm_`

العرض: انفجار التدرج و `NaN` .

الحل: `clip_grad_norm_(model.parameters(), 1.0)` في كل خطوة.

4. استخدام RNN البسيطة للتسلسلات الطويلة

العرض: النموذج يفشل في تعلم الاعتماديات البعيدة.

الحل: استخدم LSTM/GRU، أو الأفضل: المحوّلات.

5. BiRNN في مهام التوليد

العرض: النموذج "يرى المستقبل"، والتدريب يبدو ممتازًا، لكن الاستدلال يفشل.

الحل: استخدم RNN أحادية الاتجاه للتوليد.

6. نسيان `attention_mask` في Pooling

العرض: الحشو يشوه التمثيل النهائي.

الحل: اضرب في `attention_mask` قبل التجميع.

7. تهيئة الحالة المخفية بشكل خاطئ

العرض: سلوك غير مستقر في البداية.

الحل: استخدم أصفارًا (افتراضي)، أو حالة قابلة للتعلم (أفضل في بعض الحالات).

8. عدم ترتيب التسلسلات حسب الطول

العرض: `pack_padded_sequence` يعطي نتائج خاطئة أو أخطاء.

الحل: رتب حسب الطول تنازليًا، أو استخدم `enforce_sorted=False` .

9. استخدام Dropout في RNN دون `DropoutWrapper`

العرض: Dropout على المدخلات فقط، وليس على الحالة المتكررة.

الحل: استخدم `nn.LSTM(..., dropout=0.3)` (يطبق على الطبقات، وليس الحالة)، أو `DropoutWrapper` المخصص (variational dropout) لتطبيق نفس القناع على كل الخطوات.

10. تجاهل مشكلة "النسيان" في RNN

العرض: النموذج يعمل جيدًا في التدريب، لكنه يفشل في الجمل الطويلة.

الحل: لا تستخدم RNN البسيطة في المهام التي تحتاج إلى ذاكرة طويلة. استخدم LSTM/GRU، أو انتقل إلى المحوّلات.

6.11 أفضل الممارسات: قائمة المهندس المحترف

1. **استخدم** `nn.LSTM` / `nn.GRU` في الإنتاج، وليس الحلقة اليدوية.
2. `clip_grad_norm_` **دائمًا**. لا استثناء.
3. `pack_padded_sequence` مع `enforce_sorted=False` للتعامل مع الحشو.
4. `h.detach()` في Truncated BPTT.
5. **اختر LSTM أو GRU** بناءً على التجربة، وليس الافتراض. GRU أسرع وأحيانًا أفضل.
6. **BiRNN للتصنيف، UniRNN للتوليد**. لا تخلط.
7. **متوسط الحالات مع** `attention_mask` أفضل من h_T في معظم مهام التصنيف.
8. **راقب** `grad_norm`. الانفجار المفاجئ إشارة إلى مشكلة.
9. **لا تستخدم RNN للتسلسلات الطويلة جدًا** (> 500 خطوة). المحوّلات أفضل.
10. **وثّق اختياراتك**: LSTM أم GRU؟ لماذا؟ ما الإعدادات؟ "لماذا هذا النموذج أفضل؟" سؤال ستحتاج الإجابة عليه.

6.12 تمرين شامل: مختبر التسلسلات

السيناريو

لديك مجموعة بيانات لتصنيف مشاعر الجمل العربية (إيجابي/سلبي). تريد بناء نموذج تسلسلي، ومقارنة معماريات مختلفة.

المطلوب

1. **جهّز البيانات**: ترميز، حشو، `attention_mask`.
2. **ابنِ نموذجًا بـ RNN بسيطة**: بُعد 128، طبقة واحدة. درّب 10 حقب.
3. **ابنِ نموذجًا بـ LSTM**: نفس الإعدادات. قارن.
4. **ابنِ نموذجًا بـ GRU**: نفس الإعدادات. قارن.
5. **ابنِ نموذجًا بـ BiLSTM**: قارن.
6. **جرّب Pooling مختلفًا**: آخر حالة، متوسط، Max. أيها أفضل؟
7. **اختبر على جمل طويلة** (أكثر من 50 كلمة). أي نموذج يحافظ على الأداء؟

8. **اكتب تقريرًا**: ما الأسرع؟ الأفضل دقة؟ الأكثر استقرارًا؟ ماذا تعلمت؟

تمرين إضافي (متقدم)

أضف **آلية انتباه بسيطة** (Bahdanau) فوق LSTM. قارن أداءها مع LSTM بدون انتباه. هل تحسنت النتائج على الجمل الطويلة؟ (تلميح: هذا تمهيد لفصل المحوّلات.)

تمرين بحثي

اقرأ ورقة **"An Empirical Exploration of Recurrent Network Architectures"** (Jozefowicz et al., 2015). جرّب 3 معماريات مختلفة (مثلًا: LSTM مع/بدون peephole connections، GRU مع تعديلات). قارن على مهمة صغيرة. ماذا تلاحظ؟

خلاصة الفصل: ما يجب أن يبقى في ذهنك

1. **RNN قدمت الذاكرة**، لكنها نسيت بسرعة بسبب تلاشي التدرج.
2. **LSTM حلت المشكلة بالبوابات**، وفتحت الباب للتسلسلات الطويلة.
3. **GRU بسّطت LSTM**، وأحيانًا تتفوق عليها. لا تفترض، جرّب.
4. **BiRNN للتصنيف، UniRNN للتوليد**. لا تخلط.
5. **Encoder-Decoder** مهدت للمحوّلات. عنق الزجاجة أدى إلى الانتباه.
6. `pack_padded_sequence` ضرورية للحشو. بدونها، النتائج مشوهة.
7. `clip_grad_norm_` ضرورية. الانفجار قاتل.
8. **Pooling** ليس تفصيلًا. جرّب آخر حالة، متوسط، Max.
9. **الأخطاء الشائعة قاتلة**: نسيان `detach`، BiRNN في التوليد، تجاهل الحشو. احفظ القائمة.
10. **RNN ليست النهاية**. المحوّلات جاءت لحل مشاكلها. لكن فهم RNN يجعل فهم المحوّلات أعمق.

افتتاحية الفصل القادم: "الدافع وراء آلية الانتباه الذاتي"

في الفصل التالي، سنغادر عالم التكرار، وندخل عالم **الانتباه**. كيف يمكن لشبكة أن تنظر إلى كل الكلمات في وقت واحد، دون حلقة؟ ما هي المعادلة التي غيّرت كل شيء؟ ولماذا كانت "Attention is All You Need" نقطة تحول في تاريخ الذكاء الاصطناعي؟

سنتعلم كيف يعمل **Scaled Dot-Product Attention** من الصفر، وكيف نبني **Multi-Head Attention**، وكيف نتجنب الأخطاء الشائعة (مثل نسيان القناع السببي). وسنجيب على سؤال بسيط: **إذا كانت RNN تتذكر، فماذا يفعل الانتباه؟** الإجابة ستغير فهمك للنماذج اللغوية.

> ❓ **سؤال يفتح الشهية**: هل تعلم أن ورقة "Attention is All You Need" (2017) لم تكن الأولى التي استخدمت الانتباه؟ سبقتها Bahdanau (2014) وLuong (2015). فما الجديد فيها؟ سنكتشف في الفصل القادم.

المخطط التوضيحي للفصل



شكل 6.1 — LSTM: بوابة إدارة الذاكرة